

Appendix J: ResNet50V2 Tests 5

In [1]: `!nvidia-smi`

```
Tue Mar 18 10:25:55 2025
+-----+
+-----+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version:
12.4    |
|-----+-----+-----+-----+
+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Unc
orr. ECC |
| Fan   Temp   Perf           Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Co
mpute M. |
|              |                       |                      |
MIG M. |
|=====+=====+=====+=====+
=====|
|    0   NVIDIA A100-SXM4-40GB                Off | 00000000:00:04.0 Off |
0|
| N/A    31C    P0              47W / 400W |      0MiB / 40960MiB |      0%
Default |
|              |                       |                      |
Disabled |
+-----+-----+-----+-----+
+-----+
+-----+
| Processes:
|
| GPU   GI    CI          PID    Type    Process name                        GP
U Memory |
|       ID    ID                                   |
age      |
|=====+=====+=====+=====+
=====|
| No running processes found
|
+-----+-----+-----+-----+
+-----+
```

In [2]: `#!/pip install tensorflow`
`import tensorflow as tf`
`print(tf.config.list_physical_devices('GPU'))`

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Import libraries

In [3]: `import tensorflow as tf # Access to models, datasets and training`
`from tensorflow.keras.datasets import cifar100 # Access to CIFAR-100`
`from tensorflow.keras.applications import ResNet50 # Access to pre-trained model`
`from tensorflow.keras import layers, models, optimizers # Access to building blo`

```

from tensorflow.keras.models import load_model # Ability to load best_saved mode
from tensorflow import keras # Access to stuff for model
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
from sklearn.model_selection import train_test_split # To help split dataset up
from sklearn.metrics import confusion_matrix, classification_report # Analysis of model
import numpy as np # Manipulate data
import pandas as pd # Statistical analysis of data
import seaborn as sns
import matplotlib.pyplot as plt # Plot data

```

Function definitions

```

In [4]: def display_imgs(imgs, labels):
        plt.subplots(figsize=(10,10))
        for i in range(16):
            plt.subplot(4, 4, i+1)
            k = np.random.randint(0, imgs.shape[0])
            if i == 0:
                print(f"labels[{k}].shape: {labels[k].shape}")
                print(f"imgs[{k}].shape: {imgs[k].shape}")
            plt.imshow(imgs[k])
            #plt.title(labels[k])
            plt.axis('off')
        plt.tight_layout()
        plt.show()

def class_distrib(y, labels_names, dataset_name):
    counts = pd.DataFrame(data=y).value_counts().sort_index()
    #print(f"counts:\n{counts}")
    fig, ax = plt.subplots(figsize=(20,10))
    ax.bar(labels_names, counts)
    ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
    plt.title(f"Distribution of '{dataset_name}' Dataset", fontsize=25)
    plt.grid()
    plt.tight_layout()
    plt.show()

def augment_dataset(x, y):
    x = tf.image.resize(x, (224, 224)) # Resize images
    x = tf.image.random_flip_left_right(x) # Random horizontal flip
    x = tf.image.random_brightness(x, max_delta=0.2) # Adjust brightness
    x = tf.image.random_contrast(x, lower=0.8, upper=1.2) # Adjust contrast
    y = tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32)) # One-hot encoding
    return x, y

def plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc):
    # Plot Training and Validation Loss
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_loss, label='Training Loss')
    plt.plot(epochs, val_loss, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()
    plt.grid()

```

```

# Plot Training and Validation Accuracy
plt.subplot(1, 2, 2)
plt.plot(epochs, train_acc, label='Training Accuracy')
plt.plot(epochs, val_acc, label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Training and Validation Accuracy')
plt.legend()
plt.grid()

plt.tight_layout()
plt.show()

```

Dropout(0.7), L2Reg(3e-1), weight_decay(3e-1), train_augmentation

```

In [5]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
        X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_size=0.1)
        print(f"X_temp.shape: {X_temp.shape}\n")

        # Split temp data into equal validation (4000) and testing (4000) data
        X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp, y_temp, test_size=0.5)
        print(f"X_temp_val.shape: {X_temp_val.shape}")
        print(f"y_temp_val.shape: {y_temp_val.shape}")
        print(f"X_temp_test.shape: {X_temp_test.shape}")
        print(f"y_temp_test.shape: {y_temp_test.shape}\n")

        # Split test data into validation (5000) and testing (5000)
        X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.5)

        # Add temp_val to validation (9000) and temp_test to testing (9000) to get a
        X_val = np.concatenate((X_val, X_temp_val), axis=0)
        y_val = np.concatenate((y_val, y_temp_val), axis=0)
        X_test = np.concatenate((X_test, X_temp_test), axis=0)
        y_test = np.concatenate((y_test, y_temp_test), axis=0)

        print(f"X_train.shape: {X_train.shape}")
        print(f"y_train.shape: {y_train.shape}")
        print(f"X_val.shape: {X_val.shape}")
        print(f"y_val.shape: {y_val.shape}")
        print(f"X_test.shape: {X_test.shape}")
        print(f"y_test.shape: {y_test.shape}\n")

        display_imgs(X_train, y_train)

        # Normalise images (scale to range [0, 1]) - Improves convergence speed & accuracy
        X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
        display_imgs(X_train, y_train)

        labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'fish', 'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', 'bears', 'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock']

```

```

        'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
        'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
        'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
        'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
        'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
        'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
        'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
        'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                    .map(augment_dataset, num_parallel_calls=tf.data.experiment
                    .batch(batch_size)
                    .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
              .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                tf.squeeze(tf.one_hot(y, depth=100, dtype=
              .batch(batch_size)
              .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3

```

```

print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-1) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.7), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample)) # Should be <class 'tuple'>
    print(len(sample)) # Should be 2
    print(type(sample[0]), type(sample[1])) # Both should be <class 'tensorflow.Tensor'>
    print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
    print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=3e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)
#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                               factor=0.1, # Reduce lr by a factor
#                               patience=3, # Num epochs w/ no improvement
#                               min_lr=1e-6, # Min lr
#                               verbose=1)

```

```

#tensorboard = TensorBoard(Log_dir='./Logs', # Logs directory
#                               histogram_freq=1, # Logs histograms for weights/ac
#                               write_graph=True, # Logs graph of model
#                               write_images=True) # Log images like weight histog
checkpoint = ModelCheckpoint('best_model_v3.keras', # Path where the best mo
                             monitor='val_loss', # What to monitor (validati
                             save_best_only=True, # Only save model when val
                             mode='min', # Looking to minimise the validation
                             verbose=1) # Print out message when saving mode

checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v3.keras', # Pa
                                       monitor='val_loss', # What to monitor (validati
                                       save_best_only=True, # Only save model when val
                                       mode='min', # Looking to minimise the validation
                                       verbose=1) # Print out message when saving mode

#csv_logger = CSVLogger('training_log.csv', seperator=',', append=True) # Sa

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                    batch_size=batch_size, callbacks=[early_stopping, checkp

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract Loss and accuracy
epochs = range(1,len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v3.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'

```

```

plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<Fine-Tune>>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Modify Dataset>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                          .batch(batch_size)
                          .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                             batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>>

# Extract Loss and accuracy
epochs = range(1,len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>>

# Evaluate on test data

```

```

model_fine_tune = load_model('best_model_fine_tune_v3.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n")

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist

```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz>

169001437/169001437 ————— 4s 0us/step

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

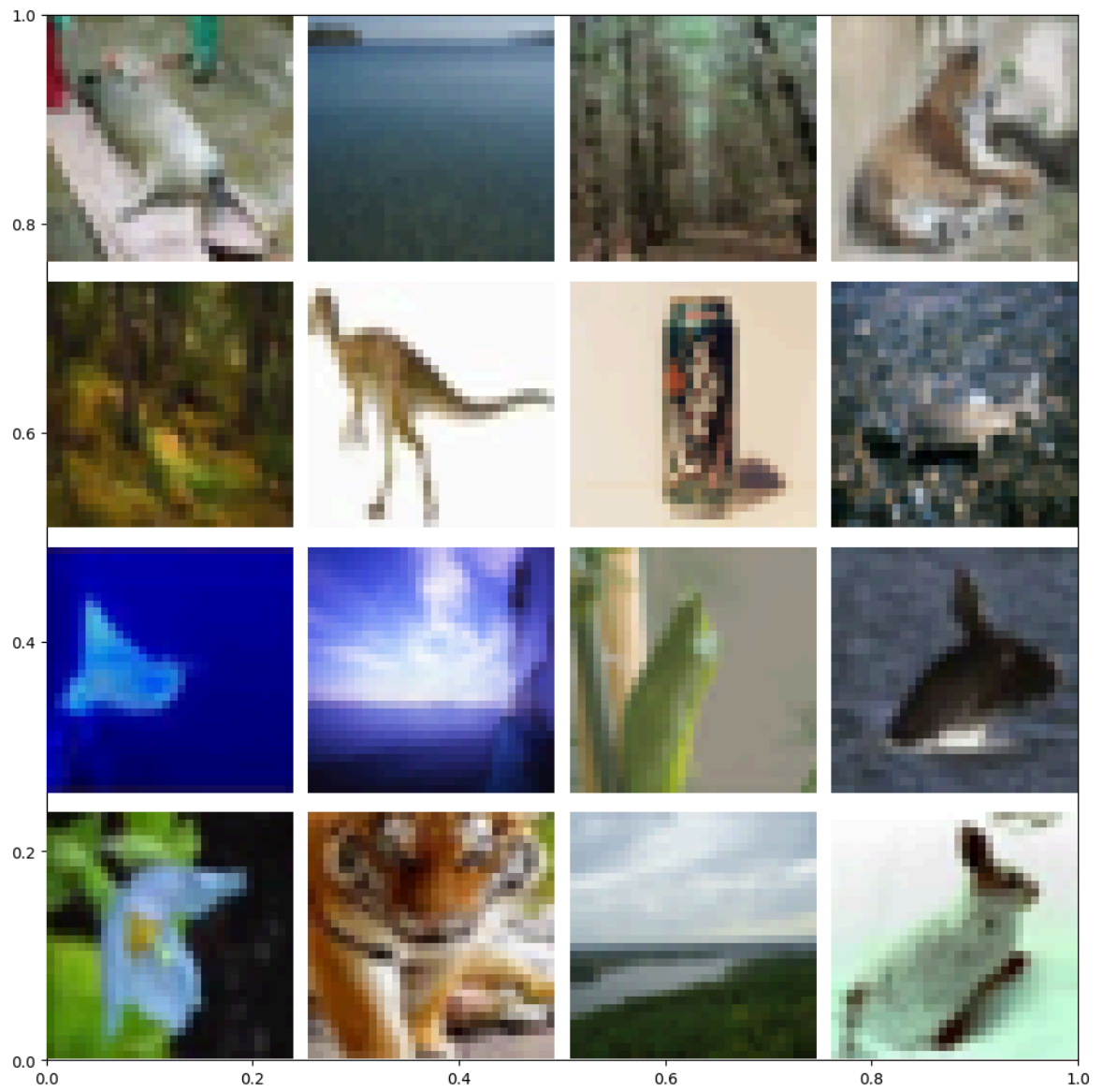
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

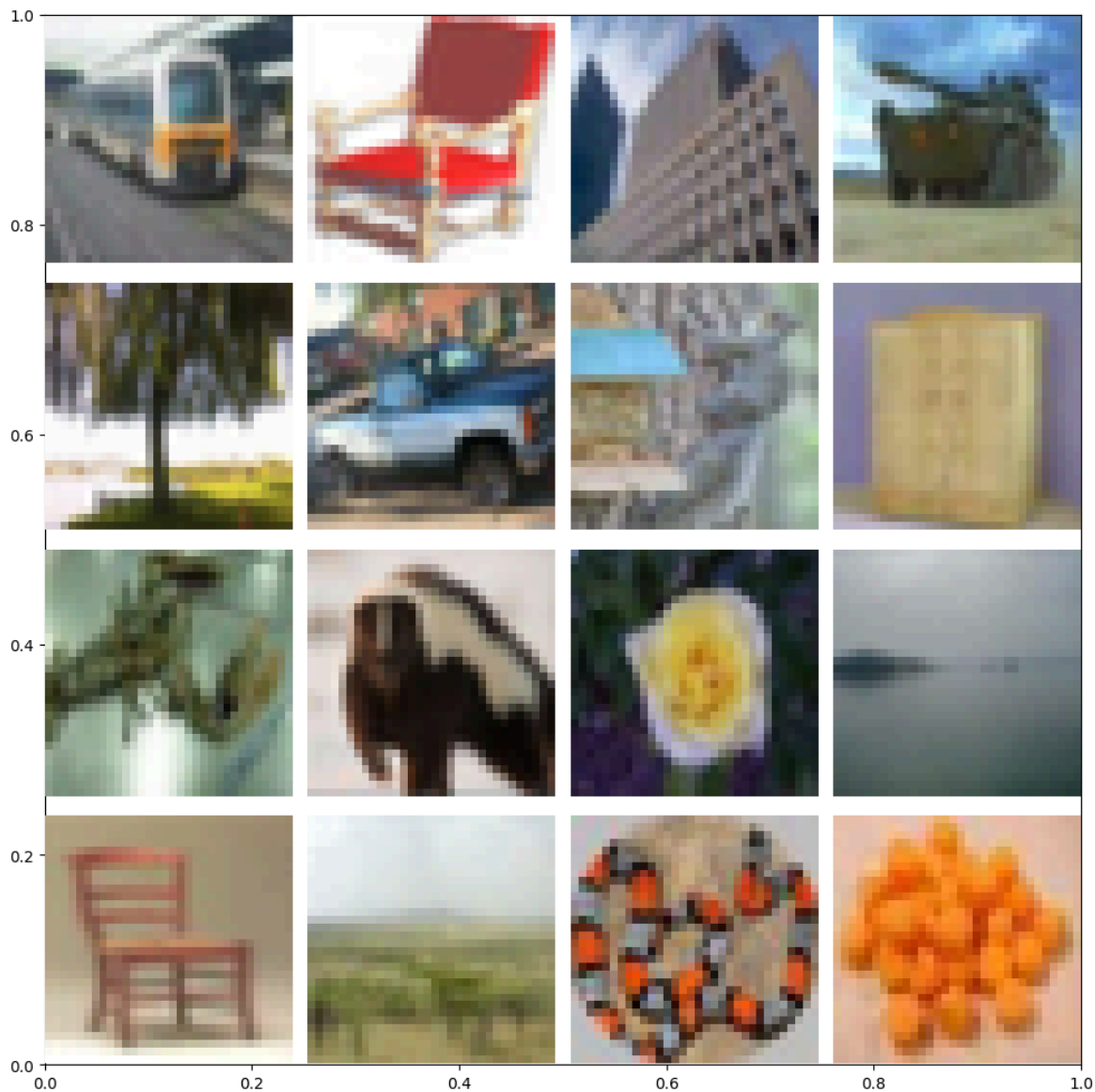
labels[17666].shape: (1,)

imgs[17666].shape: (32, 32, 3)



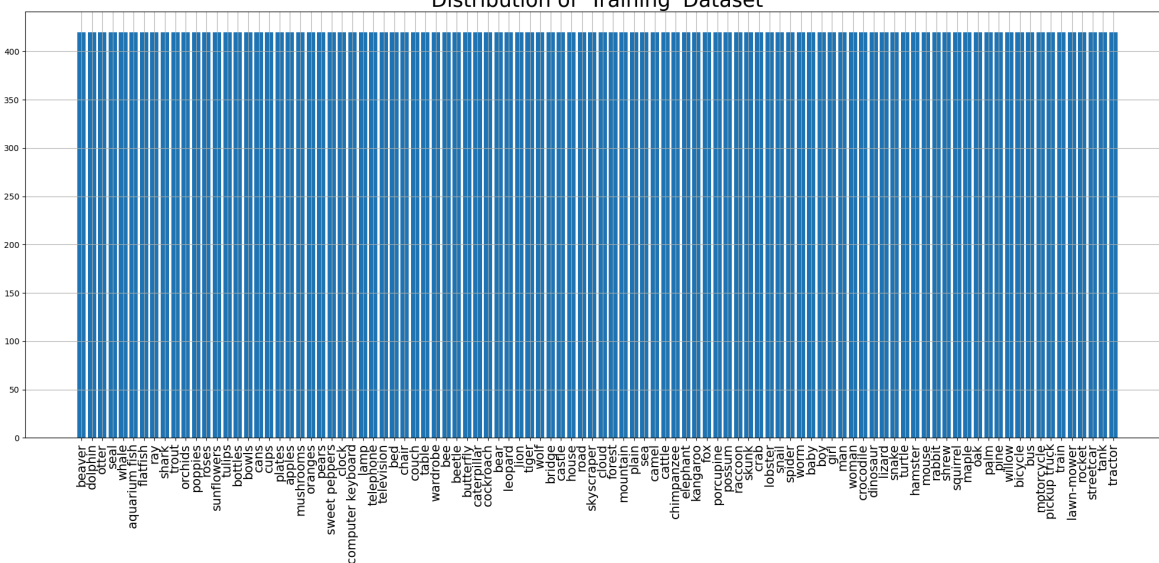
labels[28743].shape: (1,)

imgs[28743].shape: (32, 32, 3)

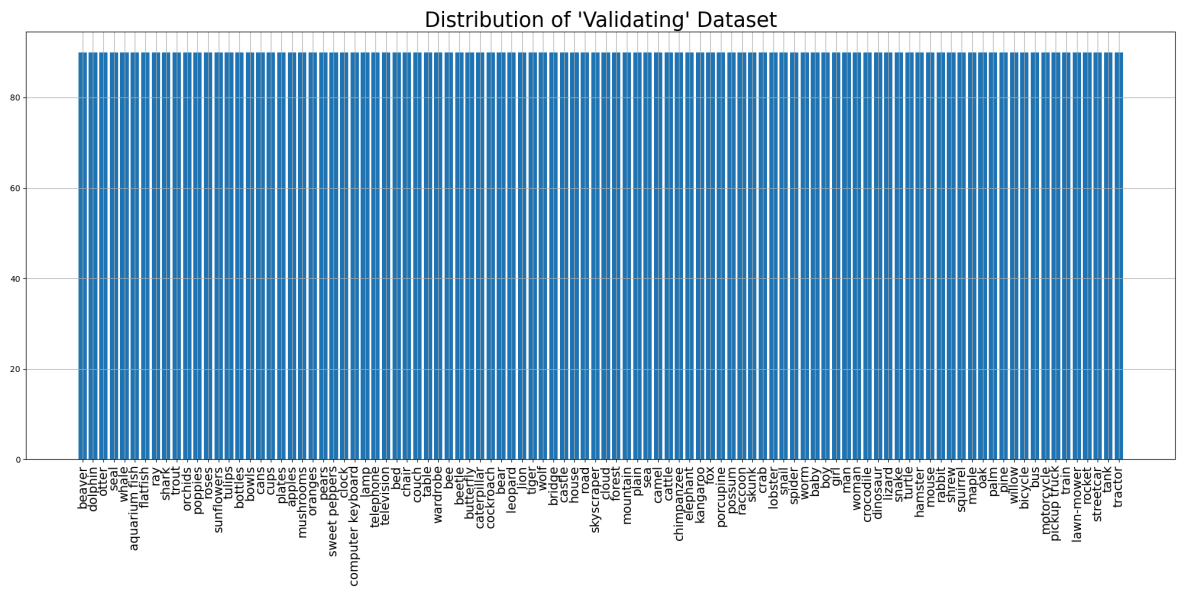


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

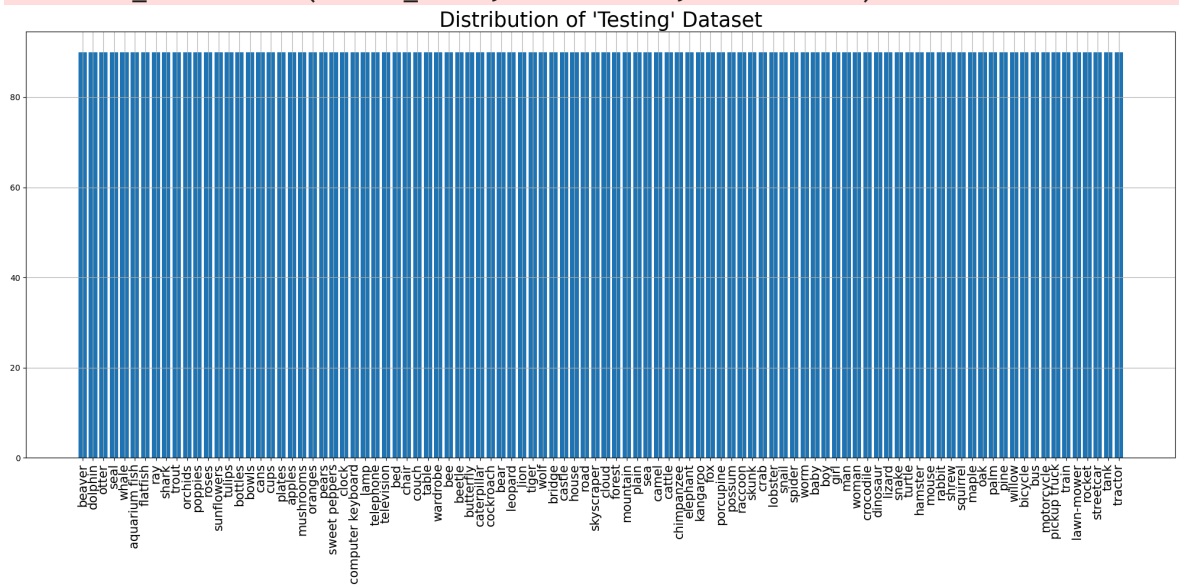
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_ConcatenateDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50v2_weights_tf_dim_ordering_tf_kernels_notop.h5

94668760/94668760 ————— 0s 0us/step

```
<class 'tuple'>
```

2

```
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
```

```
(64, 224, 224, 3)
```

```
(64, 100)
```

Model input shape: (None, 224, 224, 3)

Model output shape: (None, 100)

2

```
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
```

```
(64, 224, 224, 3) (64, 100)
```

Model: "sequential"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d (GlobalAveragePooling2D)	(None, 2048)
dense (Dense)	(None, 512)
dropout (Dropout)	(None, 512)
dense_1 (Dense)	(None, 100)

◀ ————— ▶

Total params: 24,665,188 (94.09 MB)

Trainable params: 1,100,388 (4.20 MB)

Non-trainable params: 23,564,800 (89.89 MB)

Epoch 1/25
1313/1314 ————— **0s** 28ms/step - accuracy: 0.3347 - f1_score: 0.3295
- loss: 2.7661 - precision: 0.6693
Epoch 1: val_loss improved from inf to 1.47019, saving model to best_model_v3.keras
1314/1314 ————— **71s** 40ms/step - accuracy: 0.3348 - f1_score: 0.3296
- loss: 2.7653 - precision: 0.6694 - val_accuracy: 0.5957 - val_f1_score: 0.5900
- val_loss: 1.4702 - val_precision: 0.8346
Epoch 2/25
1312/1314 ————— **0s** 23ms/step - accuracy: 0.5124 - f1_score: 0.5080
- loss: 1.7733 - precision: 0.7504
Epoch 2: val_loss improved from 1.47019 to 1.38370, saving model to best_model_v3.keras
1314/1314 ————— **35s** 26ms/step - accuracy: 0.5124 - f1_score: 0.5081
- loss: 1.7732 - precision: 0.7504 - val_accuracy: 0.6103 - val_f1_score: 0.6061
- val_loss: 1.3837 - val_precision: 0.8297
Epoch 3/25
1313/1314 ————— **0s** 23ms/step - accuracy: 0.5355 - f1_score: 0.5319
- loss: 1.6632 - precision: 0.7579
Epoch 3: val_loss improved from 1.38370 to 1.36993, saving model to best_model_v3.keras
1314/1314 ————— **34s** 26ms/step - accuracy: 0.5355 - f1_score: 0.5319
- loss: 1.6632 - precision: 0.7579 - val_accuracy: 0.6166 - val_f1_score: 0.6118
- val_loss: 1.3699 - val_precision: 0.8346
Epoch 4/25
1313/1314 ————— **0s** 23ms/step - accuracy: 0.5466 - f1_score: 0.5428
- loss: 1.6194 - precision: 0.7653
Epoch 4: val_loss improved from 1.36993 to 1.35583, saving model to best_model_v3.keras
1314/1314 ————— **34s** 26ms/step - accuracy: 0.5466 - f1_score: 0.5428
- loss: 1.6194 - precision: 0.7653 - val_accuracy: 0.6202 - val_f1_score: 0.6168
- val_loss: 1.3558 - val_precision: 0.8209
Epoch 5/25
1314/1314 ————— **0s** 23ms/step - accuracy: 0.5536 - f1_score: 0.5506
- loss: 1.5770 - precision: 0.7693
Epoch 5: val_loss improved from 1.35583 to 1.32755, saving model to best_model_v3.keras
1314/1314 ————— **34s** 26ms/step - accuracy: 0.5536 - f1_score: 0.5506
- loss: 1.5770 - precision: 0.7693 - val_accuracy: 0.6238 - val_f1_score: 0.6204
- val_loss: 1.3275 - val_precision: 0.8292
Epoch 6/25
1314/1314 ————— **0s** 23ms/step - accuracy: 0.5611 - f1_score: 0.5580
- loss: 1.5465 - precision: 0.7722
Epoch 6: val_loss did not improve from 1.32755
1314/1314 ————— **33s** 25ms/step - accuracy: 0.5611 - f1_score: 0.5580
- loss: 1.5465 - precision: 0.7722 - val_accuracy: 0.6201 - val_f1_score: 0.6166
- val_loss: 1.3368 - val_precision: 0.8239
Epoch 7/25
1314/1314 ————— **0s** 23ms/step - accuracy: 0.5639 - f1_score: 0.5606
- loss: 1.5362 - precision: 0.7703
Epoch 7: val_loss did not improve from 1.32755
1314/1314 ————— **33s** 25ms/step - accuracy: 0.5639 - f1_score: 0.5606
- loss: 1.5362 - precision: 0.7703 - val_accuracy: 0.6150 - val_f1_score: 0.6107
- val_loss: 1.3433 - val_precision: 0.8158
Epoch 8/25
1313/1314 ————— **0s** 23ms/step - accuracy: 0.5681 - f1_score: 0.5650
- loss: 1.5219 - precision: 0.7764
Epoch 8: val_loss improved from 1.32755 to 1.31252, saving model to best_model_v3.keras
1314/1314 ————— **34s** 26ms/step - accuracy: 0.5681 - f1_score: 0.565

0 - loss: 1.5220 - precision: 0.7764 - val_accuracy: 0.6297 - val_f1_score: 0.626
1 - val_loss: 1.3125 - val_precision: 0.8264
Epoch 9/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.5707 - f1_score: 0.5673
- loss: 1.5091 - precision: 0.7733
Epoch 9: val_loss did not improve from 1.31252
1314/1314 ————— 33s 25ms/step - accuracy: 0.5707 - f1_score: 0.567
3 - loss: 1.5091 - precision: 0.7733 - val_accuracy: 0.6273 - val_f1_score: 0.624
6 - val_loss: 1.3139 - val_precision: 0.8306
Epoch 10/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.5725 - f1_score: 0.5693
- loss: 1.4974 - precision: 0.7760
Epoch 10: val_loss did not improve from 1.31252
1314/1314 ————— 33s 25ms/step - accuracy: 0.5725 - f1_score: 0.569
3 - loss: 1.4974 - precision: 0.7760 - val_accuracy: 0.6287 - val_f1_score: 0.625
3 - val_loss: 1.3143 - val_precision: 0.8249
Epoch 11/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.5733 - f1_score: 0.5699
- loss: 1.4983 - precision: 0.7772
Epoch 11: val_loss improved from 1.31252 to 1.31181, saving model to best_model_v
3.keras
1314/1314 ————— 34s 26ms/step - accuracy: 0.5733 - f1_score: 0.569
9 - loss: 1.4983 - precision: 0.7772 - val_accuracy: 0.6273 - val_f1_score: 0.623
5 - val_loss: 1.3118 - val_precision: 0.8251
Epoch 12/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.5727 - f1_score: 0.5691
- loss: 1.4994 - precision: 0.7735
Epoch 12: val_loss did not improve from 1.31181
1314/1314 ————— 33s 25ms/step - accuracy: 0.5727 - f1_score: 0.569
1 - loss: 1.4994 - precision: 0.7735 - val_accuracy: 0.6208 - val_f1_score: 0.618
2 - val_loss: 1.3253 - val_precision: 0.8176
Epoch 13/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.5729 - f1_score: 0.5695
- loss: 1.4956 - precision: 0.7779
Epoch 13: val_loss did not improve from 1.31181
1314/1314 ————— 33s 25ms/step - accuracy: 0.5729 - f1_score: 0.569
5 - loss: 1.4956 - precision: 0.7779 - val_accuracy: 0.6253 - val_f1_score: 0.622
5 - val_loss: 1.3277 - val_precision: 0.8246
Epoch 14/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.5736 - f1_score: 0.5705
- loss: 1.4902 - precision: 0.7732
Epoch 14: val_loss did not improve from 1.31181
1314/1314 ————— 33s 25ms/step - accuracy: 0.5736 - f1_score: 0.570
5 - loss: 1.4902 - precision: 0.7732 - val_accuracy: 0.6280 - val_f1_score: 0.624
1 - val_loss: 1.3174 - val_precision: 0.8228
Epoch 15/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.5748 - f1_score: 0.5720
- loss: 1.4854 - precision: 0.7744
Epoch 15: val_loss improved from 1.31181 to 1.30151, saving model to best_model_v
3.keras
1314/1314 ————— 34s 26ms/step - accuracy: 0.5748 - f1_score: 0.572
0 - loss: 1.4854 - precision: 0.7744 - val_accuracy: 0.6312 - val_f1_score: 0.629
3 - val_loss: 1.3015 - val_precision: 0.8334
Epoch 16/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.5782 - f1_score: 0.5750
- loss: 1.4778 - precision: 0.7771
Epoch 16: val_loss did not improve from 1.30151
1314/1314 ————— 33s 25ms/step - accuracy: 0.5782 - f1_score: 0.575
0 - loss: 1.4778 - precision: 0.7771 - val_accuracy: 0.6279 - val_f1_score: 0.625
7 - val_loss: 1.3058 - val_precision: 0.8299

Epoch 17/25

1314/1314 ————— **0s** 23ms/step - accuracy: 0.5775 - f1_score: 0.5742
- loss: 1.4775 - precision: 0.7764

Epoch 17: val_loss did not improve from 1.30151

1314/1314 ————— **33s** 25ms/step - accuracy: 0.5775 - f1_score: 0.5742
- loss: 1.4775 - precision: 0.7764 - val_accuracy: 0.6319 - val_f1_score: 0.6289
- val_loss: 1.3018 - val_precision: 0.8241

Epoch 18/25

1314/1314 ————— **0s** 23ms/step - accuracy: 0.5791 - f1_score: 0.5762
- loss: 1.4750 - precision: 0.7751

Epoch 18: val_loss did not improve from 1.30151

1314/1314 ————— **33s** 25ms/step - accuracy: 0.5791 - f1_score: 0.5761
- loss: 1.4750 - precision: 0.7751 - val_accuracy: 0.6303 - val_f1_score: 0.6265
- val_loss: 1.3094 - val_precision: 0.8220

Epoch 19/25

1313/1314 ————— **0s** 23ms/step - accuracy: 0.5752 - f1_score: 0.5722
- loss: 1.4810 - precision: 0.7725

Epoch 19: val_loss did not improve from 1.30151

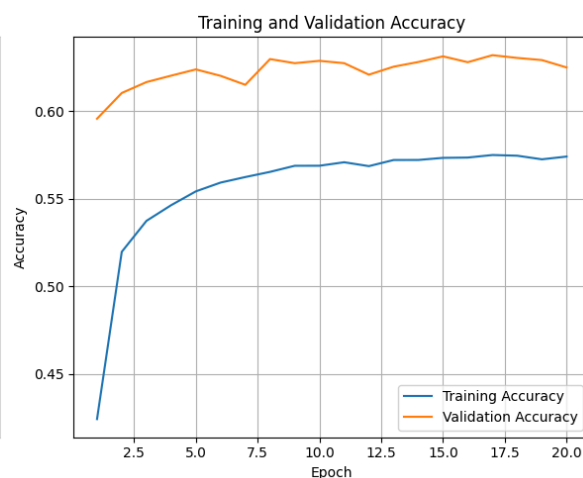
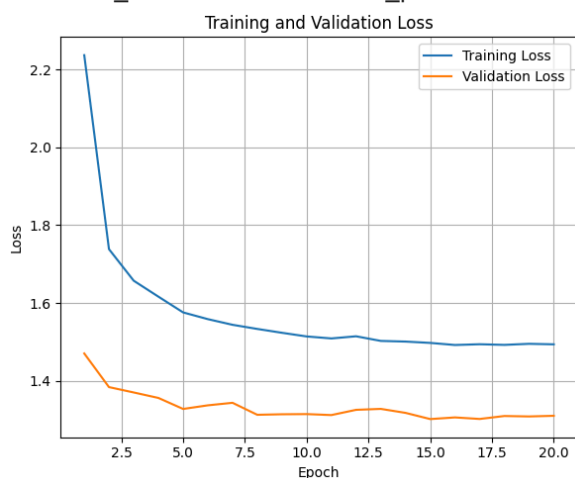
1314/1314 ————— **33s** 25ms/step - accuracy: 0.5752 - f1_score: 0.5722
- loss: 1.4810 - precision: 0.7725 - val_accuracy: 0.6291 - val_f1_score: 0.6237
- val_loss: 1.3082 - val_precision: 0.8225

Epoch 20/25

1314/1314 ————— **0s** 23ms/step - accuracy: 0.5783 - f1_score: 0.5751
- loss: 1.4776 - precision: 0.7747

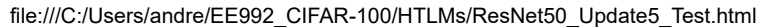
Epoch 20: val_loss did not improve from 1.30151

1314/1314 ————— **33s** 25ms/step - accuracy: 0.5783 - f1_score: 0.5751
- loss: 1.4776 - precision: 0.7747 - val_accuracy: 0.6249 - val_f1_score: 0.6215
- val_loss: 1.3100 - val_precision: 0.8262



141/141  **11s** 37ms/step - accuracy: 0.6150 - f1_score: 0.6081
- loss: 1.3336 - precision: 0.8202
Test Accuracy: 62.68%
Test Loss: 1.305716633796692
Test Precision: 83.13%
Test F1 Scores (Per Class): [82.63472 78.918915 53.107338 49.673195 32.258057 63.687145 57.00482
61.928932 79.28994 80.43478 54.421764 47.533627 66.242035 51.81818
65.359474 53.153145 72.53885 68.722466 49.999992 57.66871 77.41935
74.73684 77.83783 78.30688 76.92307 61.224483 47.368412 41.666664
73.26733 65.517235 60.130722 62.29508 44.585983 51.101315 64.676605
20.96774 71.33758 57.317066 48.520706 87.95179 67.90123 78.65168
60.869556 69.76743 44.761898 48.837204 54.794514 50.22421 80.43478
83.90804 44.324318 73.291916 58.883244 85.549126 64.13043 37.914684
78.85714 67.980286 70.66666 51.094883 78.12499 70.526306 57.142853
66.26506 38.80597 53.521126 62.275444 57.60869 84.57711 85.22727
64.13043 71.1111 39.52095 52.873566 36.2573 80.22598 85.057465
56.470577 64.15094 67.09677 38.582672 47.290634 87.83068 70.520226
54.335255 73.11827 78.07486 77.456635 58.100555 66.27218 61.842094
68.42105 52.173912 55.26315 81.40703 66.34146 39.455776 62.06896
35.22012 78.61271]
Average Test F1 Scores:62.66

282/282  **18s** 37ms/step



	precision	recall	f1-score	support
beaver	0.90	0.77	0.83	90
dolphin	0.77	0.81	0.79	90
otter	0.54	0.52	0.53	90
seal	0.60	0.42	0.50	90
whale	0.38	0.28	0.32	90
aquarium fish	0.64	0.63	0.64	90
flatfish	0.50	0.66	0.57	90
ray	0.57	0.68	0.62	90
shark	0.85	0.74	0.79	90
trout	0.79	0.83	0.81	90
orchids	0.70	0.44	0.54	90
poppies	0.40	0.59	0.48	90
roses	0.78	0.58	0.66	90
sunflowers	0.44	0.63	0.52	90
tulips	0.79	0.56	0.65	90
bottles	0.45	0.66	0.53	90
bowls	0.69	0.78	0.73	90
cans	0.57	0.87	0.69	90
cups	0.39	0.69	0.50	90
plates	0.64	0.52	0.58	90
apples	0.75	0.80	0.77	90
mushrooms	0.71	0.79	0.75	90
oranges	0.76	0.80	0.78	90
pears	0.75	0.82	0.78	90
sweet peppers	0.82	0.72	0.77	90
clock	0.57	0.67	0.61	90
computer keyboard	0.45	0.50	0.47	90
lamp	0.36	0.50	0.42	90
telephone	0.66	0.82	0.73	90
television	0.68	0.63	0.66	90
bed	0.73	0.51	0.60	90
chair	0.61	0.63	0.62	90
couch	0.52	0.39	0.45	90
table	0.42	0.64	0.51	90
wardrobe	0.59	0.72	0.65	90
bee	0.38	0.14	0.21	90
beetle	0.84	0.62	0.71	90
butterfly	0.64	0.52	0.57	90
caterpillar	0.52	0.46	0.49	90
cockroach	0.96	0.81	0.88	90
bear	0.76	0.61	0.68	90
leopard	0.80	0.78	0.79	90
lion	0.86	0.47	0.60	90
tiger	0.73	0.67	0.70	90
wolf	0.39	0.52	0.45	90
bridge	0.51	0.47	0.49	90
castle	0.47	0.67	0.55	90
house	0.42	0.62	0.50	90
road	0.79	0.82	0.80	90
skyscraper	0.87	0.81	0.84	90
cloud	0.43	0.46	0.44	90
forest	0.83	0.66	0.73	90
mountain	0.54	0.64	0.59	90
plain	0.89	0.82	0.86	90
sea	0.63	0.66	0.64	90
camel	0.33	0.44	0.38	90
cattle	0.81	0.77	0.79	90
chimpanzee	0.61	0.77	0.68	90

elephant	0.88	0.59	0.71	90
kangaroo	0.74	0.39	0.51	90
fox	0.74	0.83	0.78	90
porcupine	0.67	0.74	0.71	90
possum	0.65	0.51	0.57	90
raccoon	0.72	0.60	0.65	90
skunk	0.59	0.29	0.39	90
crab	0.73	0.42	0.54	90
lobster	0.68	0.58	0.62	90
snail	0.56	0.59	0.58	90
spider	0.77	0.94	0.85	90
worm	0.87	0.83	0.85	90
baby	0.63	0.66	0.64	90
boy	0.71	0.71	0.71	90
girl	0.43	0.37	0.40	90
man	0.55	0.51	0.53	90
woman	0.38	0.34	0.36	90
crocodile	0.82	0.79	0.80	90
dinosaur	0.88	0.82	0.85	90
lizard	0.60	0.53	0.56	90
snake	0.74	0.57	0.64	90
turtle	0.80	0.58	0.67	90
hamster	0.30	0.54	0.39	90
mouse	0.42	0.53	0.47	90
rabbit	0.84	0.92	0.88	90
shrew	0.73	0.68	0.71	90
squirrel	0.57	0.52	0.54	90
maple	0.71	0.76	0.73	90
oak	0.75	0.81	0.78	90
palm	0.81	0.74	0.77	90
pine	0.58	0.58	0.58	90
willow	0.71	0.62	0.66	90
bicycle	0.76	0.52	0.62	90
bus	0.65	0.72	0.68	90
motorcycle	0.46	0.60	0.52	90
pickup truck	0.68	0.47	0.55	90
train	0.74	0.90	0.81	90
lawn-mower	0.59	0.76	0.66	90
rocket	0.51	0.32	0.39	90
streetcar	0.64	0.60	0.62	90
tank	0.41	0.31	0.35	90
tractor	0.82	0.76	0.79	90
accuracy			0.63	9000
macro avg	0.65	0.63	0.63	9000
weighted avg	0.65	0.63	0.63	9000

Model: "sequential"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d (GlobalAveragePooling2D)	(None , 2048)
dense (Dense)	(None , 512)
dropout (Dropout)	(None , 512)
dense_1 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1970/1971 ————— **0s** 25ms/step - accuracy: 0.5992 - f1_score: 0.5966
- loss: 1.3871 - precision: 0.7946
Epoch 1: val_loss improved from inf to 1.23884, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **72s** 30ms/step - accuracy: 0.5992 - f1_score: 0.5967 - loss: 1.3871 - precision: 0.7946 - val_accuracy: 0.6458 - val_f1_score: 0.6438 - val_loss: 1.2388 - val_precision: 0.8297
Epoch 2/15
1969/1971 ————— **0s** 24ms/step - accuracy: 0.6236 - f1_score: 0.6209 - loss: 1.2915 - precision: 0.8090
Epoch 2: val_loss improved from 1.23884 to 1.22756, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **50s** 26ms/step - accuracy: 0.6236 - f1_score: 0.6209 - loss: 1.2915 - precision: 0.8090 - val_accuracy: 0.6472 - val_f1_score: 0.6452 - val_loss: 1.2276 - val_precision: 0.8268
Epoch 3/15
1971/1971 ————— **0s** 23ms/step - accuracy: 0.6364 - f1_score: 0.6336 - loss: 1.2499 - precision: 0.8162
Epoch 3: val_loss improved from 1.22756 to 1.22060, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **50s** 25ms/step - accuracy: 0.6364 - f1_score: 0.6336 - loss: 1.2499 - precision: 0.8162 - val_accuracy: 0.6498 - val_f1_score: 0.6478 - val_loss: 1.2206 - val_precision: 0.8260
Epoch 4/15
1971/1971 ————— **0s** 23ms/step - accuracy: 0.6408 - f1_score: 0.6378 - loss: 1.2296 - precision: 0.8213
Epoch 4: val_loss improved from 1.22060 to 1.21468, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **50s** 25ms/step - accuracy: 0.6408 - f1_score: 0.6378 - loss: 1.2296 - precision: 0.8213 - val_accuracy: 0.6511 - val_f1_score: 0.6493 - val_loss: 1.2147 - val_precision: 0.8270
Epoch 5/15
1971/1971 ————— **0s** 23ms/step - accuracy: 0.6434 - f1_score: 0.6406 - loss: 1.2130 - precision: 0.8220
Epoch 5: val_loss improved from 1.21468 to 1.21021, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **50s** 25ms/step - accuracy: 0.6434 - f1_score: 0.6406 - loss: 1.2130 - precision: 0.8220 - val_accuracy: 0.6517 - val_f1_score: 0.6499 - val_loss: 1.2102 - val_precision: 0.8266
Epoch 6/15
1970/1971 ————— **0s** 23ms/step - accuracy: 0.6523 - f1_score: 0.6496 - loss: 1.1932 - precision: 0.8240
Epoch 6: val_loss improved from 1.21021 to 1.20581, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **50s** 25ms/step - accuracy: 0.6523 - f1_score: 0.6496 - loss: 1.1932 - precision: 0.8240 - val_accuracy: 0.6549 - val_f1_score: 0.6530 - val_loss: 1.2058 - val_precision: 0.8267
Epoch 7/15
1970/1971 ————— **0s** 23ms/step - accuracy: 0.6563 - f1_score: 0.6536 - loss: 1.1773 - precision: 0.8276
Epoch 7: val_loss improved from 1.20581 to 1.20204, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **50s** 25ms/step - accuracy: 0.6563 - f1_score: 0.6536 - loss: 1.1773 - precision: 0.8276 - val_accuracy: 0.6554 - val_f1_score: 0.6536 - val_loss: 1.2020 - val_precision: 0.8249
Epoch 8/15
1969/1971 ————— **0s** 23ms/step - accuracy: 0.6586 - f1_score: 0.6559 - loss: 1.1650 - precision: 0.8277
Epoch 8: val_loss improved from 1.20204 to 1.19843, saving model to best_model_fi

ne_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.6586 - f1_score: 0.6559 - loss: 1.1650 - precision: 0.8277 - val_accuracy: 0.6568 - val_f1_score: 0.6549 - val_loss: 1.1984 - val_precision: 0.8255

Epoch 9/15

1971/1971 ————— 0s 23ms/step - accuracy: 0.6628 - f1_score: 0.6602 - loss: 1.1512 - precision: 0.8357

Epoch 9: val_loss improved from 1.19843 to 1.19542, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.6628 - f1_score: 0.6602 - loss: 1.1512 - precision: 0.8357 - val_accuracy: 0.6577 - val_f1_score: 0.6558 - val_loss: 1.1954 - val_precision: 0.8268

Epoch 10/15

1971/1971 ————— 0s 23ms/step - accuracy: 0.6625 - f1_score: 0.6599 - loss: 1.1419 - precision: 0.8334

Epoch 10: val_loss improved from 1.19542 to 1.19282, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.6625 - f1_score: 0.6599 - loss: 1.1419 - precision: 0.8334 - val_accuracy: 0.6561 - val_f1_score: 0.6543 - val_loss: 1.1928 - val_precision: 0.8261

Epoch 11/15

1971/1971 ————— 0s 23ms/step - accuracy: 0.6678 - f1_score: 0.6653 - loss: 1.1305 - precision: 0.8392

Epoch 11: val_loss improved from 1.19282 to 1.19029, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.6678 - f1_score: 0.6653 - loss: 1.1305 - precision: 0.8392 - val_accuracy: 0.6581 - val_f1_score: 0.6562 - val_loss: 1.1903 - val_precision: 0.8264

Epoch 12/15

1971/1971 ————— 0s 23ms/step - accuracy: 0.6708 - f1_score: 0.6683 - loss: 1.1199 - precision: 0.8371

Epoch 12: val_loss improved from 1.19029 to 1.18787, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.6708 - f1_score: 0.6683 - loss: 1.1199 - precision: 0.8371 - val_accuracy: 0.6591 - val_f1_score: 0.6574 - val_loss: 1.1879 - val_precision: 0.8267

Epoch 13/15

1970/1971 ————— 0s 23ms/step - accuracy: 0.6727 - f1_score: 0.6702 - loss: 1.1115 - precision: 0.8411

Epoch 13: val_loss improved from 1.18787 to 1.18549, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.6727 - f1_score: 0.6702 - loss: 1.1115 - precision: 0.8411 - val_accuracy: 0.6596 - val_f1_score: 0.6579 - val_loss: 1.1855 - val_precision: 0.8250

Epoch 14/15

1971/1971 ————— 0s 23ms/step - accuracy: 0.6764 - f1_score: 0.6740 - loss: 1.1005 - precision: 0.8411

Epoch 14: val_loss improved from 1.18549 to 1.18291, saving model to best_model_fine_tune_v3.keras

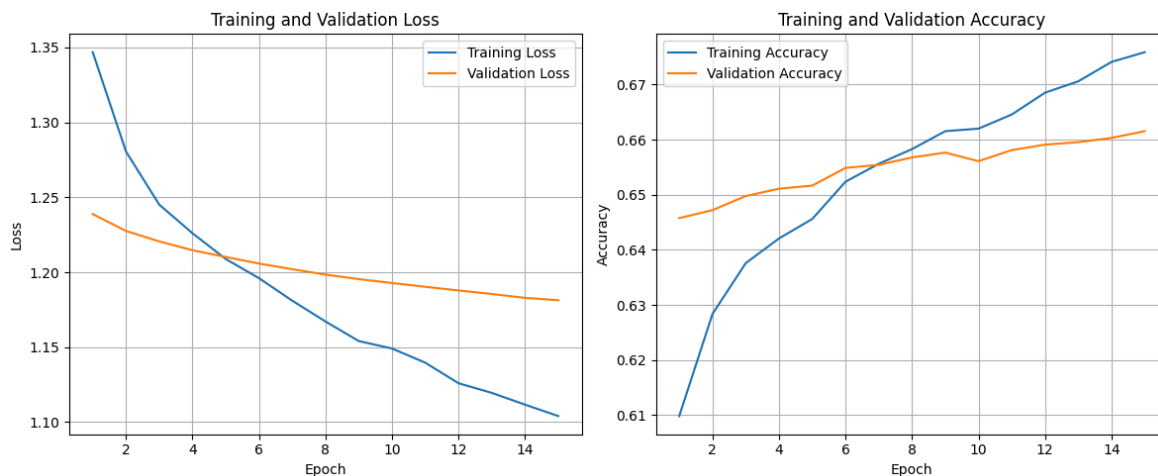
1971/1971 ————— 50s 25ms/step - accuracy: 0.6764 - f1_score: 0.6740 - loss: 1.1005 - precision: 0.8411 - val_accuracy: 0.6603 - val_f1_score: 0.6587 - val_loss: 1.1829 - val_precision: 0.8252

Epoch 15/15

1970/1971 ————— 0s 23ms/step - accuracy: 0.6779 - f1_score: 0.6754 - loss: 1.0940 - precision: 0.8433

Epoch 15: val_loss improved from 1.18291 to 1.18133, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.6779 - f1_score: 0.6754 - loss: 1.0940 - precision: 0.8433 - val_accuracy: 0.6616 - val_f1_score: 0.6599 - val_loss: 1.1813 - val_precision: 0.8242



141/141 ————— **11s 40ms/step** - accuracy: 0.6517 - f1_score: 0.6436

- loss: 1.1959 - precision: 0.8205

Test Accuracy: 66.22%

Test Loss: 1.1718369722366333

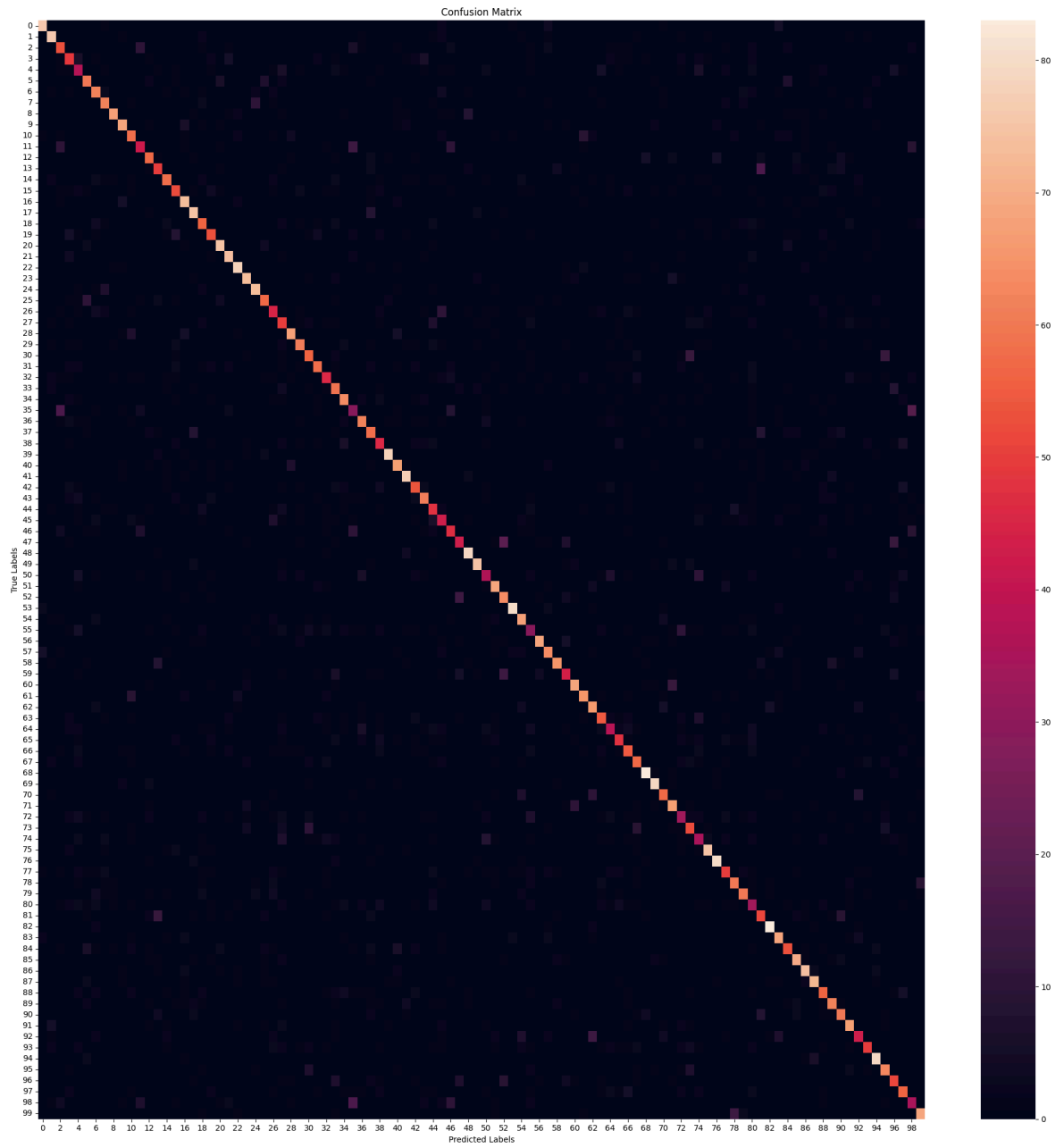
Test Precision: 82.86%

Test F1 Scores (Per Class): [85.39326 79.99999 55.208332 53.551907 39.3617 64.51612 63.917522

68.888885	81.437126	78.40909	62.702698	51.219505	65.89595	57.142853
68.23529	59.428566	74.87179	77.31957	60.540535	60.919537	80.87432
78.30688	84.32432	80.21977	79.78141	66.66666	51.136356	47.846886
78.16091	70.93022	61.202175	69.461075	55.75757	58.41583	69.18918
32.768353	68.888885	62.032085	52.272724	89.65517	71.65774	82.35293
62.06896	69.31818	50.793648	48.087425	52.51396	52.760727	83.59788
84.74575	43.37349	74.999985	63.999992	89.887634	68.68686	34.939754
82.142845	70.71823	77.8443	53.987724	77.09496	74.28571	67.6923
62.14689	46.341457	59.036137	60.439552	62.63736	87.83068	85.405396
67.05882	71.276596	41.212116	56.830597	38.67403	82.872925	87.43169
59.880234	67.415726	68.965515	35.78947	53.191483	89.72973	73.40425
62.352936	73.68421	80.87432	79.12087	62.222218	68.13186	65.217384
71.65774	52.439022	62.5	85.405396	68.47825	54.255314	62.222218
41.176464	77.9661					

Average Test F1 Scores:66.10

282/282 ————— **11s 27ms/step**



	precision	recall	f1-score	support
beaver	0.86	0.84	0.85	90
dolphin	0.76	0.84	0.80	90
otter	0.52	0.59	0.55	90
seal	0.53	0.54	0.54	90
whale	0.38	0.41	0.39	90
aquarium fish	0.62	0.67	0.65	90
flatfish	0.60	0.69	0.64	90
ray	0.69	0.69	0.69	90
shark	0.88	0.76	0.81	90
trout	0.80	0.77	0.78	90
orchids	0.61	0.64	0.63	90
poppies	0.57	0.47	0.51	90
roses	0.69	0.63	0.66	90
sunflowers	0.59	0.56	0.57	90
tulips	0.72	0.64	0.68	90
bottles	0.61	0.58	0.59	90
bowls	0.70	0.81	0.75	90
cans	0.72	0.83	0.77	90
cups	0.59	0.62	0.61	90
plates	0.63	0.59	0.61	90
apples	0.80	0.83	0.82	90
mushrooms	0.75	0.82	0.78	90
oranges	0.82	0.87	0.84	90
pears	0.79	0.81	0.80	90
sweet peppers	0.78	0.81	0.80	90
clock	0.70	0.63	0.67	90
computer keyboard	0.52	0.50	0.51	90
lamp	0.42	0.56	0.48	90
telephone	0.81	0.76	0.78	90
television	0.74	0.68	0.71	90
bed	0.60	0.62	0.61	90
chair	0.75	0.64	0.69	90
couch	0.61	0.51	0.56	90
table	0.52	0.66	0.58	90
wardrobe	0.67	0.71	0.69	90
bee	0.33	0.32	0.33	90
beetle	0.69	0.69	0.69	90
butterfly	0.60	0.64	0.62	90
caterpillar	0.53	0.51	0.52	90
cockroach	0.93	0.87	0.90	90
bear	0.70	0.74	0.72	90
leopard	0.79	0.86	0.82	90
lion	0.64	0.60	0.62	90
tiger	0.71	0.68	0.69	90
wolf	0.48	0.53	0.51	90
bridge	0.47	0.48	0.47	90
castle	0.53	0.52	0.53	90
house	0.59	0.48	0.53	90
road	0.80	0.88	0.84	90
skyscraper	0.86	0.83	0.85	90
cloud	0.47	0.40	0.43	90
forest	0.73	0.77	0.75	90
mountain	0.58	0.71	0.64	90
plain	0.91	0.89	0.90	90
sea	0.63	0.76	0.69	90
camel	0.38	0.32	0.35	90
cattle	0.88	0.77	0.82	90
chimpanzee	0.70	0.71	0.71	90

elephant	0.84	0.72	0.78	90
kangaroo	0.60	0.49	0.54	90
fox	0.78	0.77	0.77	90
porcupine	0.76	0.72	0.74	90
possum	0.63	0.73	0.68	90
raccoon	0.63	0.61	0.62	90
skunk	0.51	0.42	0.46	90
crab	0.64	0.53	0.58	90
lobster	0.60	0.61	0.60	90
snail	0.62	0.63	0.63	90
spider	0.84	0.92	0.88	90
worm	0.83	0.88	0.85	90
baby	0.71	0.63	0.67	90
boy	0.68	0.74	0.71	90
girl	0.45	0.38	0.41	90
man	0.56	0.58	0.57	90
woman	0.38	0.39	0.39	90
crocodile	0.82	0.83	0.83	90
dinosaur	0.86	0.89	0.87	90
lizard	0.65	0.56	0.60	90
snake	0.68	0.67	0.67	90
turtle	0.71	0.67	0.69	90
hamster	0.34	0.38	0.36	90
mouse	0.52	0.57	0.54	90
rabbit	0.87	0.92	0.90	90
shrew	0.70	0.77	0.73	90
squirrel	0.66	0.59	0.62	90
maple	0.70	0.78	0.74	90
oak	0.80	0.82	0.81	90
palm	0.78	0.80	0.79	90
pine	0.62	0.62	0.62	90
willow	0.67	0.69	0.68	90
bicycle	0.65	0.67	0.66	90
bus	0.69	0.74	0.72	90
motorcycle	0.58	0.48	0.52	90
pickup truck	0.71	0.56	0.62	90
train	0.83	0.88	0.85	90
lawn-mower	0.67	0.70	0.68	90
rocket	0.52	0.57	0.54	90
streetcar	0.62	0.62	0.62	90
tank	0.43	0.39	0.41	90
tractor	0.79	0.77	0.78	90
accuracy			0.66	9000
macro avg	0.66	0.66	0.66	9000
weighted avg	0.66	0.66	0.66	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

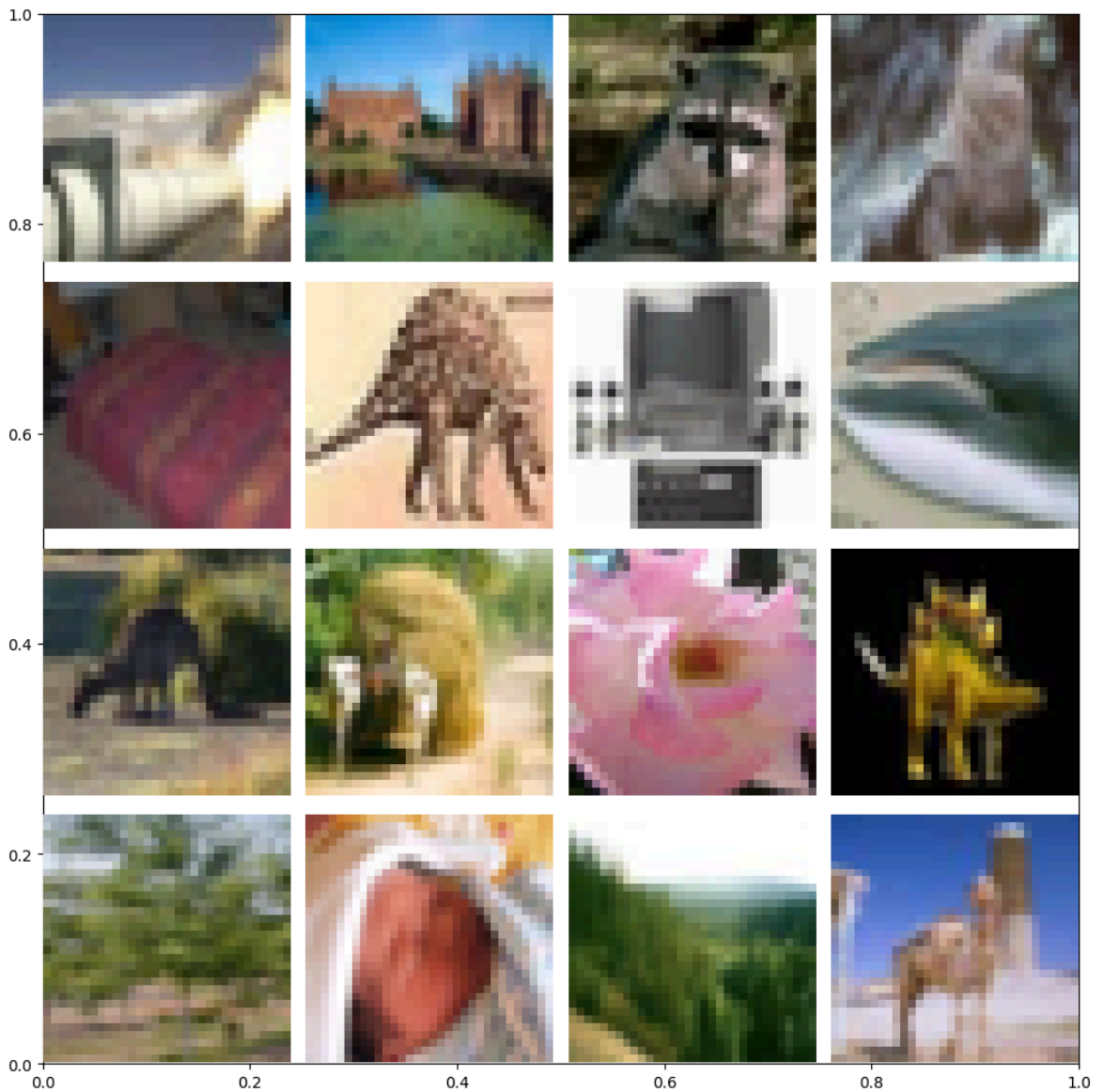
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

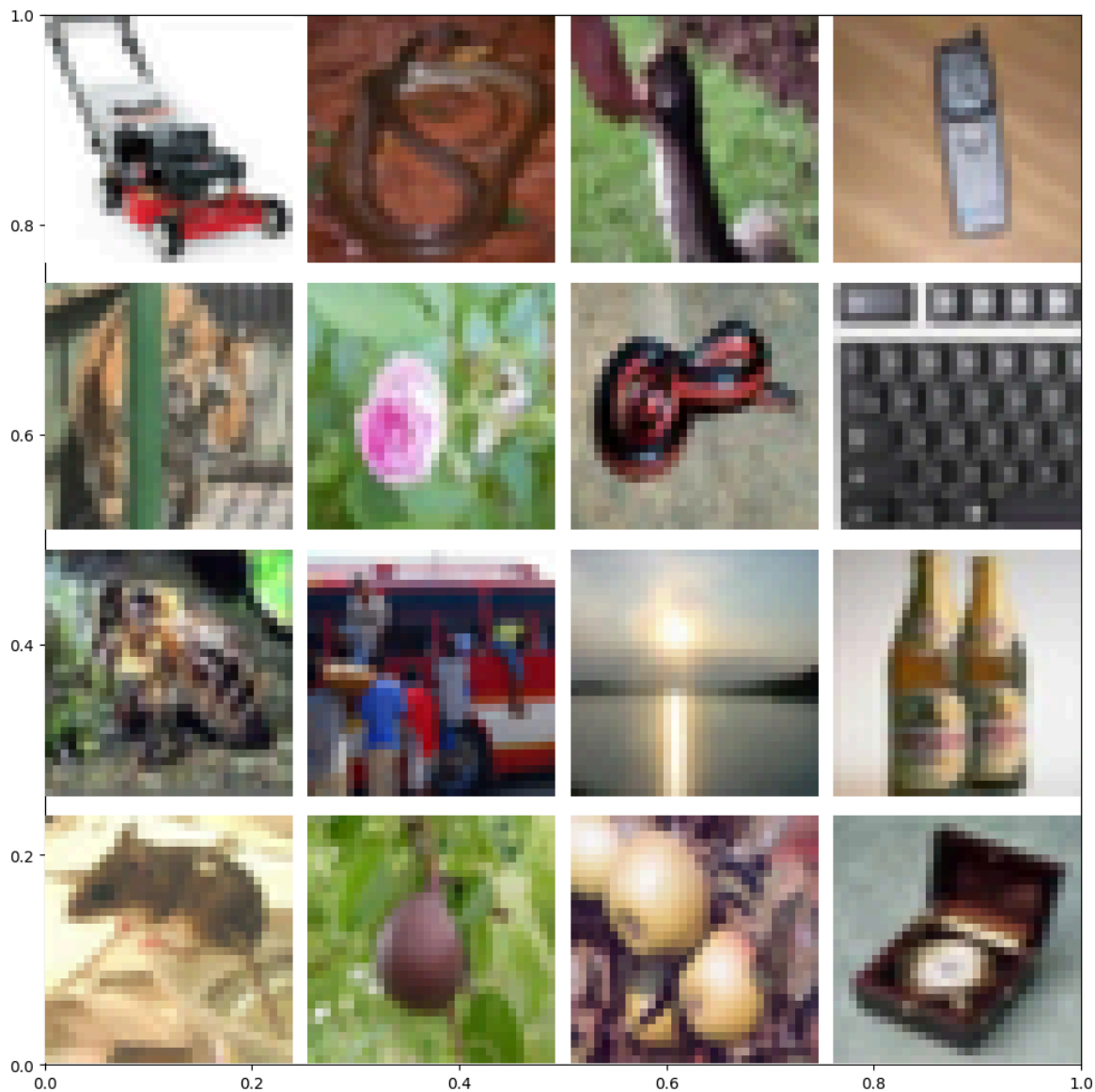
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[41376].shape: (1,)  
imgs[41376].shape: (32, 32, 3)
```

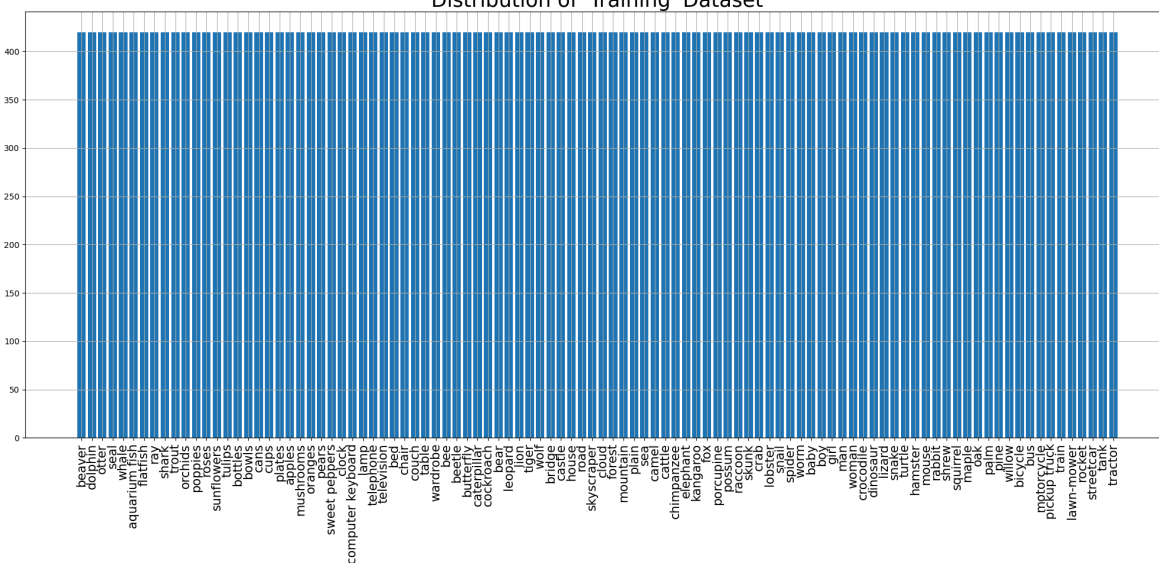


```
labels[28387].shape: (1,)  
imgs[28387].shape: (32, 32, 3)
```

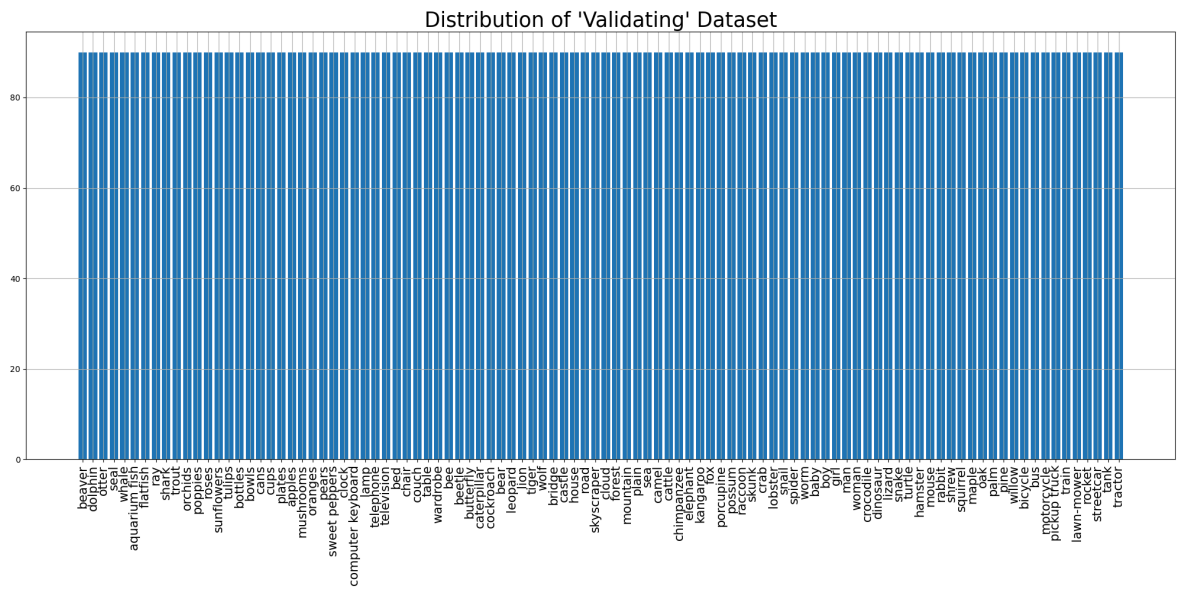


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

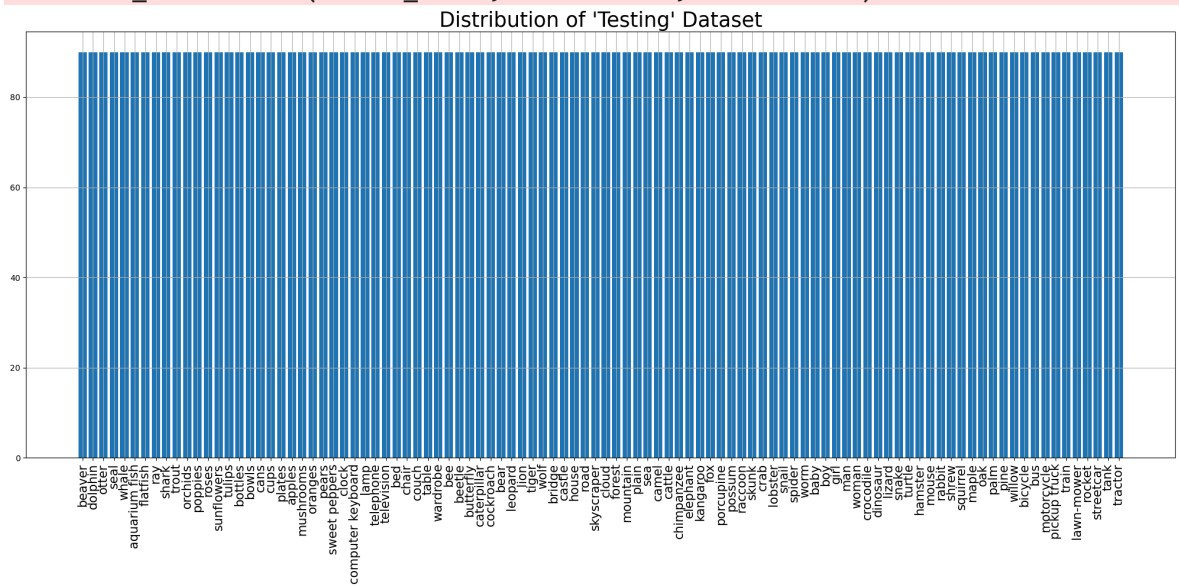
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_ConcatenateDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

```
<class 'tuple'>
```

2

```
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
```

(64, 224, 224, 3)

(64, 100)

Model input shape: (None, 224, 224, 3)

Model output shape: (None, 100)

2

```
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
```

(64, 224, 224, 3) (64, 100)

Model: "sequential_1"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None , 2048)
dense_2 (Dense)	(None , 512)
dropout_1 (Dropout)	(None , 512)
dense_3 (Dense)	(None , 100)

◀  ▶

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
1313/1314 ————— 0s 26ms/step - accuracy: 0.3308 - f1_score: 0.3253
- loss: 2.7982 - precision: 0.6726
Epoch 1: val_loss improved from inf to 1.47552, saving model to best_model_v3.keras
1314/1314 ————— 55s 34ms/step - accuracy: 0.3309 - f1_score: 0.3255
- loss: 2.7974 - precision: 0.6726 - val_accuracy: 0.5954 - val_f1_score: 0.5909
- val_loss: 1.4755 - val_precision: 0.8350
Epoch 2/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.5103 - f1_score: 0.5057
- loss: 1.7821 - precision: 0.7487
Epoch 2: val_loss improved from 1.47552 to 1.39231, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.5103 - f1_score: 0.5057
- loss: 1.7820 - precision: 0.7487 - val_accuracy: 0.6112 - val_f1_score: 0.6085
- val_loss: 1.3923 - val_precision: 0.8314
Epoch 3/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.5339 - f1_score: 0.5302
- loss: 1.6712 - precision: 0.7602
Epoch 3: val_loss improved from 1.39231 to 1.36564, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.5339 - f1_score: 0.5302
- loss: 1.6711 - precision: 0.7602 - val_accuracy: 0.6186 - val_f1_score: 0.6146
- val_loss: 1.3656 - val_precision: 0.8411
Epoch 4/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.5448 - f1_score: 0.5413
- loss: 1.6168 - precision: 0.7634
Epoch 4: val_loss improved from 1.36564 to 1.34421, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.5448 - f1_score: 0.5413
- loss: 1.6168 - precision: 0.7634 - val_accuracy: 0.6174 - val_f1_score: 0.6131
- val_loss: 1.3442 - val_precision: 0.8223
Epoch 5/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.5549 - f1_score: 0.5508
- loss: 1.5811 - precision: 0.7662
Epoch 5: val_loss improved from 1.34421 to 1.33874, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.5549 - f1_score: 0.5508
- loss: 1.5811 - precision: 0.7662 - val_accuracy: 0.6216 - val_f1_score: 0.6183
- val_loss: 1.3387 - val_precision: 0.8333
Epoch 6/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.5585 - f1_score: 0.5553
- loss: 1.5536 - precision: 0.7710
Epoch 6: val_loss improved from 1.33874 to 1.33050, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.5585 - f1_score: 0.5553
- loss: 1.5536 - precision: 0.7710 - val_accuracy: 0.6264 - val_f1_score: 0.6248
- val_loss: 1.3305 - val_precision: 0.8305
Epoch 7/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.5644 - f1_score: 0.5611
- loss: 1.5385 - precision: 0.7676
Epoch 7: val_loss improved from 1.33050 to 1.31451, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.5644 - f1_score: 0.5611
- loss: 1.5385 - precision: 0.7676 - val_accuracy: 0.6274 - val_f1_score: 0.6236
- val_loss: 1.3145 - val_precision: 0.8226
Epoch 8/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.5662 - f1_score: 0.5628
- loss: 1.5290 - precision: 0.7711
Epoch 8: val_loss did not improve from 1.31451

1314/1314 ————— 34s 26ms/step - accuracy: 0.5662 - f1_score: 0.5628 - loss: 1.5291 - precision: 0.7711 - val_accuracy: 0.6212 - val_f1_score: 0.6170 - val_loss: 1.3231 - val_precision: 0.8222
Epoch 9/25

1313/1314 ————— 0s 23ms/step - accuracy: 0.5686 - f1_score: 0.5656 - loss: 1.5139 - precision: 0.7713
Epoch 9: val_loss did not improve from 1.31451

1314/1314 ————— 34s 26ms/step - accuracy: 0.5686 - f1_score: 0.5656 - loss: 1.5139 - precision: 0.7712 - val_accuracy: 0.6227 - val_f1_score: 0.6198 - val_loss: 1.3374 - val_precision: 0.8241
Epoch 10/25

1314/1314 ————— 0s 23ms/step - accuracy: 0.5712 - f1_score: 0.5678 - loss: 1.5060 - precision: 0.7756
Epoch 10: val_loss did not improve from 1.31451

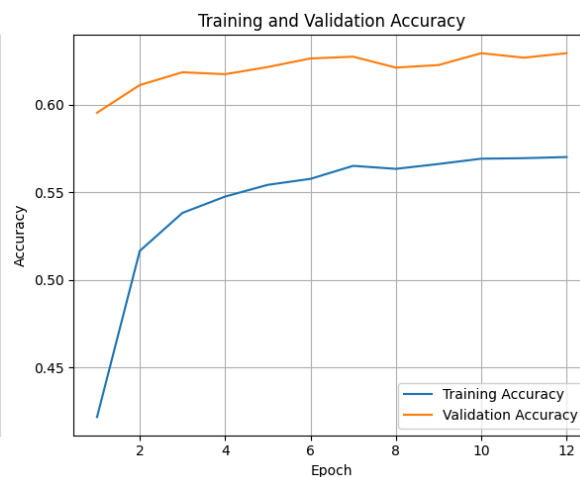
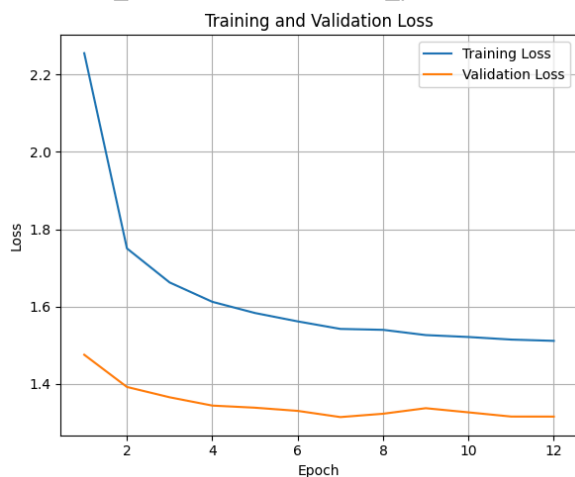
1314/1314 ————— 34s 26ms/step - accuracy: 0.5712 - f1_score: 0.5678 - loss: 1.5060 - precision: 0.7756 - val_accuracy: 0.6294 - val_f1_score: 0.6256 - val_loss: 1.3267 - val_precision: 0.8203
Epoch 11/25

1312/1314 ————— 0s 23ms/step - accuracy: 0.5705 - f1_score: 0.5671 - loss: 1.5022 - precision: 0.7723
Epoch 11: val_loss did not improve from 1.31451

1314/1314 ————— 34s 26ms/step - accuracy: 0.5705 - f1_score: 0.5671 - loss: 1.5022 - precision: 0.7723 - val_accuracy: 0.6269 - val_f1_score: 0.6229 - val_loss: 1.3159 - val_precision: 0.8281
Epoch 12/25

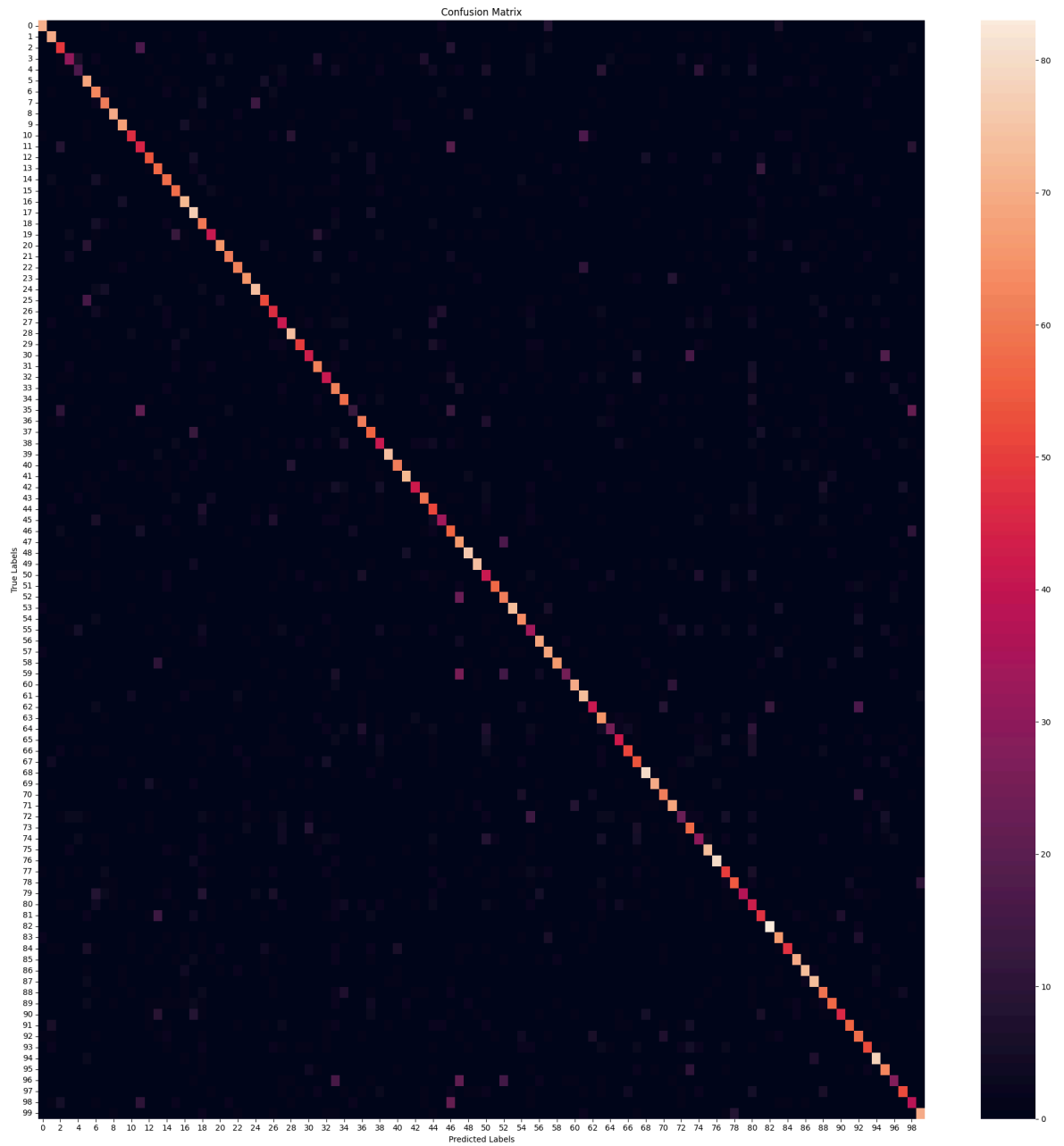
1313/1314 ————— 0s 23ms/step - accuracy: 0.5726 - f1_score: 0.5692 - loss: 1.5000 - precision: 0.7728
Epoch 12: val_loss did not improve from 1.31451

1314/1314 ————— 34s 26ms/step - accuracy: 0.5726 - f1_score: 0.5692 - loss: 1.5000 - precision: 0.7728 - val_accuracy: 0.6294 - val_f1_score: 0.6271 - val_loss: 1.3158 - val_precision: 0.8283



141/141  **11s** 37ms/step - accuracy: 0.6110 - f1_score: 0.6016
- loss: 1.3410 - precision: 0.8140
Test Accuracy: 62.82%
Test Loss: 1.3076353073120117
Test Precision: 82.98%
Test F1 Scores (Per Class): [83.5294 74.46808 53.551907 42.465748 25.563904 6
4.48598 58.878494
68.926544 82.352936 77.27273 60.645157 46.63212 65.85365 58.461536
62.032085 62.365585 76.59574 69.36936 52.173912 52.229298 76.92307
73.93939 73.809525 76.02339 78.723404 63.803684 51.366108 46.857132
77.89473 64.93506 54.43037 63.2653 49.397583 50.819664 61.052628
20.869562 68.53932 59.574467 47.12643 88.09524 68.53932 79.55801
58.333324 69.005844 49.514557 46.258495 47.45762 53.012043 81.72042
82.22221 40.999996 70.80744 60.78431 86.904755 68.449196 36.756752
73.01586 70.40816 79.51807 40.9836 78.65168 71.219505 53.947365
61.90476 37.593983 53.947365 63.75 55.384613 83.76962 80.92485
67.77777 71.87499 32.167828 56.999992 35.502953 76.43979 87.43169
54.05405 66.66666 54.814808 32.82442 53.93258 84.263954 69.10995
59.25926 75.82417 78.918915 74.87179 58.999996 65.89595 61.33333
66.666664 52.777767 60.1156 81.675385 63.316578 37.837833 62.275444
41.304344 77.77777]
Average Test F1 Scores:62.53

282/282  **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.89	0.79	0.84	90
dolphin	0.71	0.78	0.74	90
otter	0.53	0.54	0.54	90
seal	0.55	0.34	0.42	90
whale	0.40	0.19	0.26	90
aquarium fish	0.56	0.77	0.64	90
flatfish	0.51	0.70	0.59	90
ray	0.70	0.68	0.69	90
shark	0.88	0.78	0.82	90
trout	0.79	0.76	0.77	90
orchids	0.72	0.52	0.61	90
poppies	0.44	0.50	0.47	90
roses	0.73	0.60	0.66	90
sunflowers	0.54	0.63	0.58	90
tulips	0.60	0.64	0.62	90
bottles	0.60	0.64	0.62	90
bowls	0.73	0.80	0.77	90
cans	0.58	0.86	0.69	90
cups	0.43	0.67	0.52	90
plates	0.61	0.46	0.52	90
apples	0.82	0.72	0.77	90
mushrooms	0.81	0.68	0.74	90
oranges	0.79	0.69	0.74	90
pears	0.80	0.72	0.76	90
sweet peppers	0.76	0.82	0.79	90
clock	0.71	0.58	0.64	90
computer keyboard	0.51	0.52	0.52	90
lamp	0.48	0.46	0.47	90
telephone	0.74	0.82	0.78	90
television	0.78	0.56	0.65	90
bed	0.63	0.48	0.54	90
chair	0.58	0.69	0.63	90
couch	0.54	0.46	0.49	90
table	0.40	0.69	0.51	90
wardrobe	0.58	0.64	0.61	90
bee	0.48	0.13	0.21	90
beetle	0.69	0.68	0.69	90
butterfly	0.57	0.62	0.60	90
caterpillar	0.49	0.46	0.47	90
cockroach	0.95	0.82	0.88	90
bear	0.69	0.68	0.69	90
leopard	0.79	0.80	0.80	90
lion	0.78	0.47	0.58	90
tiger	0.73	0.66	0.69	90
wolf	0.44	0.57	0.50	90
bridge	0.60	0.38	0.46	90
castle	0.38	0.62	0.47	90
house	0.41	0.73	0.53	90
road	0.79	0.84	0.82	90
skyscraper	0.82	0.82	0.82	90
cloud	0.38	0.46	0.41	90
forest	0.80	0.63	0.71	90
mountain	0.54	0.68	0.60	90
plain	0.94	0.81	0.87	90
sea	0.66	0.71	0.68	90
camel	0.36	0.38	0.37	90
cattle	0.70	0.77	0.73	90
chimpanzee	0.65	0.77	0.70	90

elephant	0.88	0.73	0.80	90
kangaroo	0.78	0.28	0.41	90
fox	0.80	0.78	0.79	90
porcupine	0.63	0.81	0.71	90
possum	0.67	0.46	0.54	90
raccoon	0.54	0.72	0.62	90
skunk	0.58	0.28	0.38	90
crab	0.67	0.47	0.55	90
lobster	0.73	0.57	0.64	90
snail	0.51	0.60	0.55	90
spider	0.79	0.89	0.84	90
worm	0.84	0.78	0.81	90
baby	0.68	0.68	0.68	90
boy	0.68	0.77	0.72	90
girl	0.43	0.26	0.32	90
man	0.52	0.63	0.57	90
woman	0.38	0.33	0.36	90
crocodile	0.72	0.81	0.76	90
dinosaur	0.86	0.89	0.87	90
lizard	0.53	0.56	0.54	90
snake	0.73	0.61	0.67	90
turtle	0.82	0.41	0.55	90
hamster	0.25	0.48	0.33	90
mouse	0.55	0.53	0.54	90
rabbit	0.78	0.92	0.84	90
shrew	0.65	0.73	0.69	90
squirrel	0.67	0.53	0.59	90
maple	0.75	0.78	0.77	90
oak	0.77	0.81	0.79	90
palm	0.70	0.81	0.75	90
pine	0.54	0.66	0.59	90
willow	0.69	0.63	0.66	90
bicycle	0.77	0.51	0.61	90
bus	0.72	0.62	0.67	90
motorcycle	0.46	0.64	0.53	90
pickup truck	0.62	0.58	0.60	90
train	0.77	0.87	0.82	90
lawn-mower	0.58	0.70	0.63	90
rocket	0.48	0.31	0.38	90
streetcar	0.68	0.58	0.62	90
tank	0.40	0.42	0.41	90
tractor	0.78	0.78	0.78	90
accuracy			0.63	9000
macro avg	0.64	0.63	0.63	9000
weighted avg	0.64	0.63	0.63	9000

Model: "sequential_1"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None , 2048)
dense_2 (Dense)	(None , 512)
dropout_1 (Dropout)	(None , 512)
dense_3 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15

1971/1971  **0s** 25ms/step - accuracy: 0.5917 - f1_score: 0.5879
- loss: 1.4286 - precision: 0.7876

Epoch 1: val_loss improved from inf to 1.25412, saving model to best_model_fine_tune_v3.keras

1971/1971  **71s** 30ms/step - accuracy: 0.5917 - f1_score: 0.5879 - loss: 1.4286 - precision: 0.7876 - val_accuracy: 0.6451 - val_f1_score: 0.6425 - val_loss: 1.2541 - val_precision: 0.8280

Epoch 2/15

1970/1971  **0s** 23ms/step - accuracy: 0.6153 - f1_score: 0.6122
- loss: 1.3337 - precision: 0.8018

Epoch 2: val_loss improved from 1.25412 to 1.24230, saving model to best_model_fine_tune_v3.keras

1971/1971  **50s** 25ms/step - accuracy: 0.6154 - f1_score: 0.6122 - loss: 1.3337 - precision: 0.8018 - val_accuracy: 0.6487 - val_f1_score: 0.6463 - val_loss: 1.2423 - val_precision: 0.8266

Epoch 3/15

1969/1971  **0s** 23ms/step - accuracy: 0.6225 - f1_score: 0.6196
- loss: 1.2997 - precision: 0.8084

Epoch 3: val_loss improved from 1.24230 to 1.23498, saving model to best_model_fine_tune_v3.keras

1971/1971  **50s** 25ms/step - accuracy: 0.6225 - f1_score: 0.6196 - loss: 1.2997 - precision: 0.8084 - val_accuracy: 0.6500 - val_f1_score: 0.6477 - val_loss: 1.2350 - val_precision: 0.8258

Epoch 4/15

1971/1971  **0s** 23ms/step - accuracy: 0.6311 - f1_score: 0.6281
- loss: 1.2752 - precision: 0.8168

Epoch 4: val_loss improved from 1.23498 to 1.22863, saving model to best_model_fine_tune_v3.keras

1971/1971  **50s** 25ms/step - accuracy: 0.6311 - f1_score: 0.6281 - loss: 1.2752 - precision: 0.8168 - val_accuracy: 0.6534 - val_f1_score: 0.6510 - val_loss: 1.2286 - val_precision: 0.8275

Epoch 5/15

1971/1971  **0s** 23ms/step - accuracy: 0.6363 - f1_score: 0.6332
- loss: 1.2530 - precision: 0.8177

Epoch 5: val_loss improved from 1.22863 to 1.22309, saving model to best_model_fine_tune_v3.keras

1971/1971  **50s** 25ms/step - accuracy: 0.6363 - f1_score: 0.6332 - loss: 1.2530 - precision: 0.8177 - val_accuracy: 0.6544 - val_f1_score: 0.6523 - val_loss: 1.2231 - val_precision: 0.8263

Epoch 6/15

1970/1971  **0s** 23ms/step - accuracy: 0.6399 - f1_score: 0.6371
- loss: 1.2444 - precision: 0.8192

Epoch 6: val_loss improved from 1.22309 to 1.21794, saving model to best_model_fine_tune_v3.keras

1971/1971  **50s** 25ms/step - accuracy: 0.6399 - f1_score: 0.6371 - loss: 1.2444 - precision: 0.8192 - val_accuracy: 0.6551 - val_f1_score: 0.6531 - val_loss: 1.2179 - val_precision: 0.8277

Epoch 7/15

1970/1971  **0s** 23ms/step - accuracy: 0.6425 - f1_score: 0.6396
- loss: 1.2256 - precision: 0.8234

Epoch 7: val_loss improved from 1.21794 to 1.21425, saving model to best_model_fine_tune_v3.keras

1971/1971  **50s** 25ms/step - accuracy: 0.6425 - f1_score: 0.6396 - loss: 1.2256 - precision: 0.8234 - val_accuracy: 0.6567 - val_f1_score: 0.6544 - val_loss: 1.2142 - val_precision: 0.8275

Epoch 8/15

1970/1971  **0s** 23ms/step - accuracy: 0.6465 - f1_score: 0.6438
- loss: 1.2185 - precision: 0.8226

Epoch 8: val_loss improved from 1.21425 to 1.21036, saving model to best_model_fine_tune_v3.keras

ne_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.6465 - f1_score: 0.6438 - loss: 1.2185 - precision: 0.8226 - val_accuracy: 0.6566 - val_f1_score: 0.6545 - val_loss: 1.2104 - val_precision: 0.8281

Epoch 9/15

1969/1971 ————— 0s 23ms/step - accuracy: 0.6534 - f1_score: 0.6508 - loss: 1.1976 - precision: 0.8298

Epoch 9: val_loss improved from 1.21036 to 1.20706, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 26ms/step - accuracy: 0.6534 - f1_score: 0.6508 - loss: 1.1976 - precision: 0.8298 - val_accuracy: 0.6574 - val_f1_score: 0.6553 - val_loss: 1.2071 - val_precision: 0.8285

Epoch 10/15

1970/1971 ————— 0s 23ms/step - accuracy: 0.6570 - f1_score: 0.6542 - loss: 1.1818 - precision: 0.8327

Epoch 10: val_loss improved from 1.20706 to 1.20413, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.6570 - f1_score: 0.6542 - loss: 1.1818 - precision: 0.8327 - val_accuracy: 0.6573 - val_f1_score: 0.6551 - val_loss: 1.2041 - val_precision: 0.8280

Epoch 11/15

1969/1971 ————— 0s 23ms/step - accuracy: 0.6561 - f1_score: 0.6534 - loss: 1.1813 - precision: 0.8292

Epoch 11: val_loss improved from 1.20413 to 1.20090, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.6561 - f1_score: 0.6534 - loss: 1.1813 - precision: 0.8292 - val_accuracy: 0.6591 - val_f1_score: 0.6568 - val_loss: 1.2009 - val_precision: 0.8270

Epoch 12/15

1970/1971 ————— 0s 23ms/step - accuracy: 0.6598 - f1_score: 0.6570 - loss: 1.1692 - precision: 0.8320

Epoch 12: val_loss improved from 1.20090 to 1.19802, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.6598 - f1_score: 0.6570 - loss: 1.1692 - precision: 0.8320 - val_accuracy: 0.6609 - val_f1_score: 0.6589 - val_loss: 1.1980 - val_precision: 0.8287

Epoch 13/15

1969/1971 ————— 0s 23ms/step - accuracy: 0.6628 - f1_score: 0.6603 - loss: 1.1564 - precision: 0.8357

Epoch 13: val_loss improved from 1.19802 to 1.19579, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.6628 - f1_score: 0.6603 - loss: 1.1564 - precision: 0.8357 - val_accuracy: 0.6600 - val_f1_score: 0.6579 - val_loss: 1.1958 - val_precision: 0.8283

Epoch 14/15

1971/1971 ————— 0s 23ms/step - accuracy: 0.6634 - f1_score: 0.6606 - loss: 1.1510 - precision: 0.8371

Epoch 14: val_loss improved from 1.19579 to 1.19348, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.6634 - f1_score: 0.6606 - loss: 1.1511 - precision: 0.8371 - val_accuracy: 0.6593 - val_f1_score: 0.6572 - val_loss: 1.1935 - val_precision: 0.8294

Epoch 15/15

1969/1971 ————— 0s 23ms/step - accuracy: 0.6655 - f1_score: 0.6628 - loss: 1.1482 - precision: 0.8378

Epoch 15: val_loss improved from 1.19348 to 1.19080, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.6655 - f1_score: 0.6628 - loss: 1.1482 - precision: 0.8378 - val_accuracy: 0.6620 - val_f1_score: 0.6600 - val_loss: 1.1908 - val_precision: 0.8303



141/141 ————— **11s** 37ms/step - accuracy: 0.6466 - f1_score: 0.6388

- loss: 1.2127 - precision: 0.8229

Test Accuracy: 65.84%

Test Loss: 1.1833933591842651

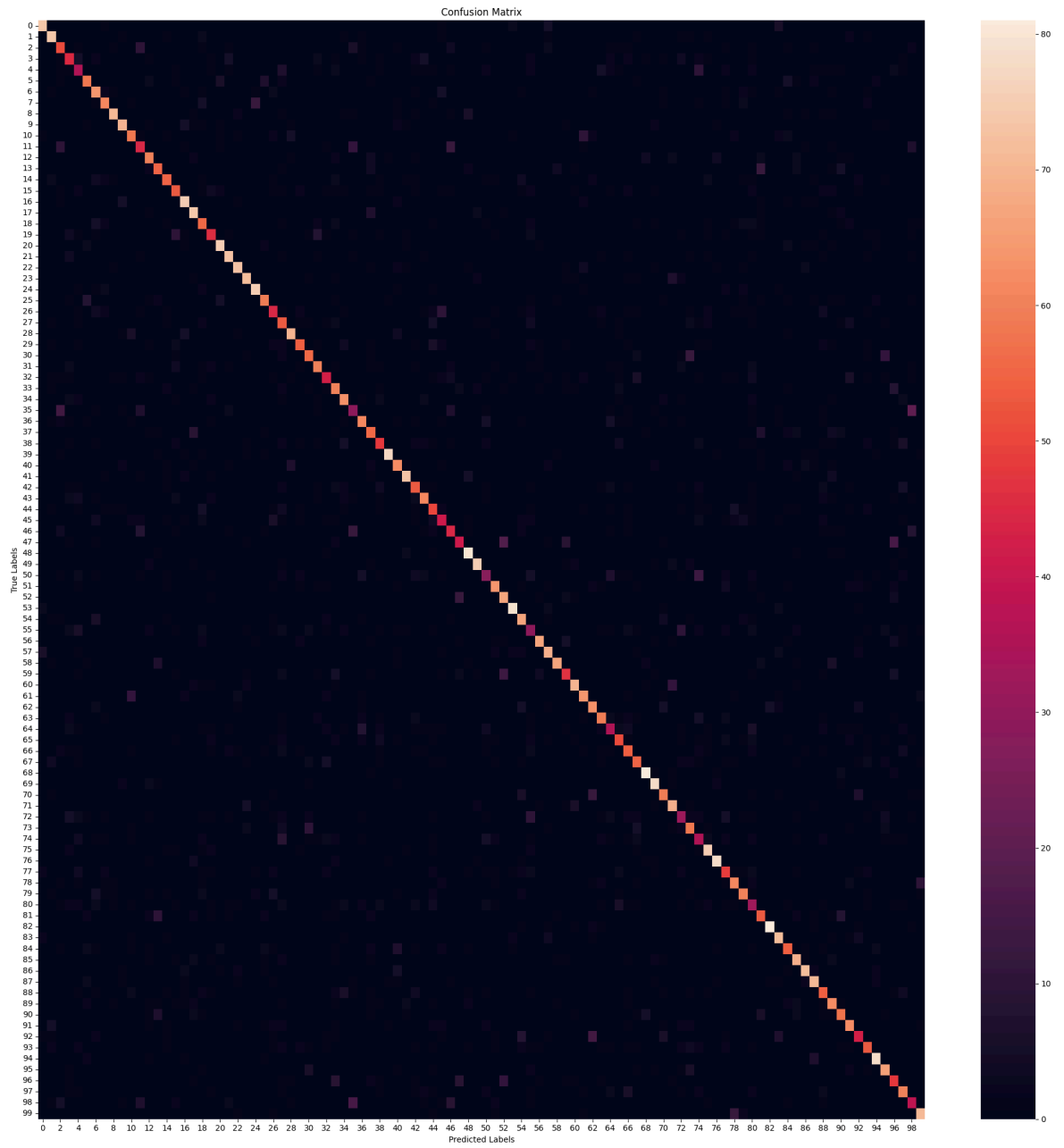
Test Precision: 83.34%

Test F1 Scores (Per Class): [82.68156 77.48691 53.68421 49.999992 39.106144 6
6.28571 62.745094

70.93022	83.04093	79.09604	65.16853	50.28571	65.92178	62.499992
66.26506	59.21787	78.12499	74.747475	57.731956	55.421684	81.08108
78.49462	81.56424	80.446915	82.41757	68.604645	51.461983	48.84792
79.09604	66.666664	63.276833	68.181816	53.416145	58.252422	68.1081
34.319523	68.15642	61.202175	56.140347	89.534874	65.957436	82.02247
58.56353	70.11494	52.910053	47.953213	49.438198	51.572315	82.90154
84.74575	35.89743	73.56321	66.33662	87.77777	67.33668	32.584263
78.82352	74.19354	80.722885	56.969696	79.09604	71.50838	65.968575
67.42856	42.94478	59.64912	58.064507	61.452507	86.63101	83.422455
66.666664	73.40425	38.787876	61.702126	38.043476	78.947365	86.666664
55.681812	68.15642	67.796616	34.408596	58.24175	91.52542	74.6114
62.06896	73.40425	80.89887	77.59562	59.66851	65.60846	63.687145
68.13186	53.08641	62.352936	84.32432	70.588234	51.578945	65.217384
43.181812	78.453026]					

Average Test F1 Scores:65.71

282/282 ————— **11s** 27ms/step



	precision	recall	f1-score	support
beaver	0.83	0.82	0.83	90
dolphin	0.73	0.82	0.77	90
otter	0.51	0.57	0.54	90
seal	0.50	0.50	0.50	90
whale	0.39	0.39	0.39	90
aquarium fish	0.68	0.64	0.66	90
flatfish	0.57	0.71	0.63	90
ray	0.74	0.68	0.71	90
shark	0.88	0.79	0.83	90
trout	0.80	0.78	0.79	90
orchids	0.66	0.64	0.65	90
poppies	0.52	0.49	0.50	90
roses	0.67	0.67	0.67	90
sunflowers	0.64	0.61	0.62	90
tulips	0.72	0.61	0.66	90
bottles	0.60	0.59	0.59	90
bowls	0.74	0.83	0.78	90
cans	0.69	0.82	0.75	90
cups	0.54	0.62	0.58	90
plates	0.61	0.51	0.55	90
apples	0.79	0.83	0.81	90
mushrooms	0.76	0.81	0.78	90
oranges	0.82	0.81	0.82	90
pears	0.81	0.80	0.80	90
sweet peppers	0.82	0.83	0.82	90
clock	0.72	0.66	0.69	90
computer keyboard	0.54	0.49	0.51	90
lamp	0.42	0.59	0.49	90
telephone	0.80	0.78	0.79	90
television	0.75	0.60	0.67	90
bed	0.64	0.62	0.63	90
chair	0.70	0.67	0.68	90
couch	0.61	0.48	0.53	90
table	0.52	0.67	0.58	90
wardrobe	0.66	0.70	0.68	90
bee	0.37	0.32	0.34	90
beetle	0.69	0.68	0.68	90
butterfly	0.60	0.62	0.61	90
caterpillar	0.59	0.53	0.56	90
cockroach	0.94	0.86	0.90	90
bear	0.63	0.69	0.66	90
leopard	0.83	0.81	0.82	90
lion	0.58	0.59	0.59	90
tiger	0.73	0.68	0.70	90
wolf	0.51	0.56	0.53	90
bridge	0.51	0.46	0.48	90
castle	0.50	0.49	0.49	90
house	0.59	0.46	0.52	90
road	0.78	0.89	0.83	90
skyscraper	0.86	0.83	0.85	90
cloud	0.42	0.31	0.36	90
forest	0.76	0.71	0.74	90
mountain	0.60	0.74	0.66	90
plain	0.88	0.88	0.88	90
sea	0.61	0.74	0.67	90
camel	0.33	0.32	0.33	90
cattle	0.84	0.74	0.79	90
chimpanzee	0.72	0.77	0.74	90

elephant	0.88	0.74	0.81	90
kangaroo	0.63	0.52	0.57	90
fox	0.80	0.78	0.79	90
porcupine	0.72	0.71	0.72	90
possum	0.62	0.70	0.66	90
raccoon	0.70	0.66	0.68	90
skunk	0.48	0.39	0.43	90
crab	0.63	0.57	0.60	90
lobster	0.56	0.60	0.58	90
snail	0.62	0.61	0.61	90
spider	0.84	0.90	0.87	90
worm	0.80	0.87	0.83	90
baby	0.68	0.66	0.67	90
boy	0.70	0.77	0.73	90
girl	0.43	0.36	0.39	90
man	0.59	0.64	0.62	90
woman	0.37	0.39	0.38	90
crocodile	0.75	0.83	0.79	90
dinosaur	0.87	0.87	0.87	90
lizard	0.57	0.54	0.56	90
snake	0.69	0.68	0.68	90
turtle	0.69	0.67	0.68	90
hamster	0.34	0.37	0.35	90
mouse	0.58	0.59	0.58	90
rabbit	0.93	0.90	0.92	90
shrew	0.70	0.80	0.75	90
squirrel	0.64	0.60	0.62	90
maple	0.70	0.77	0.73	90
oak	0.82	0.80	0.81	90
palm	0.76	0.79	0.78	90
pine	0.59	0.60	0.59	90
willow	0.63	0.69	0.66	90
bicycle	0.64	0.63	0.64	90
bus	0.67	0.69	0.68	90
motorcycle	0.60	0.48	0.53	90
pickup truck	0.66	0.59	0.62	90
train	0.82	0.87	0.84	90
lawn-mower	0.68	0.73	0.71	90
rocket	0.48	0.53	0.51	90
streetcar	0.64	0.67	0.65	90
tank	0.44	0.42	0.43	90
tractor	0.78	0.79	0.78	90
accuracy			0.66	9000
macro avg	0.66	0.66	0.66	9000
weighted avg	0.66	0.66	0.66	9000

Dropout(0.8), weight_decay(0.9), l2(0.9)

```
In [6]: for repeat_2_times in range(2):
        ##### <<<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
        X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
        print(f"X_temp.shape: {X_temp.shape}\n")
```

```

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
#                    .map(augment_dataset, num_parallel_calls=tf.data.experimen
#                    .batch(batch_size))

```

```

# .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset#.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(0.9) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.8), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample)) # Should be <class 'tuple'>
    print(len(sample)) # Should be 2

```

```

        print(type(sample[0]), type(sample[1])) # Both should be <class 'tensorflow.Tensor'>
        print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
        print(sample[1].shape) # Should be (batch_size, 100)
    print(f"Model input shape: {model.input_shape}")
    print(f"Model output shape: {model.output_shape}")
    sample = next(iter(test_dataset.as_numpy_iterator()))
    print(len(sample)) # Should be 2
    print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
    print(sample[0].shape, sample[1].shape) # Should match model input and output shape
    print("\n")
    #for x, y in test_dataset.take(1):
    #    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
    #for x_batch, y_batch in test_dataset.take(1):
    #    test_loss, test_acc = model.evaluate(x_batch, y_batch)
    #    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

    # Compile the model
    #tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./logs")
    model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=0.0001),
                  loss='categorical_crossentropy',
                  metrics=['accuracy', 'precision', 'f1_score'])

    model.summary()

    ##### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
    early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                                   patience=5, # Num. epochs with no improvement
                                   restore_best_weights=True)

    #reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
    #                               factor=0.1, # Reduce lr by a factor
    #                               patience=3, # Num epochs w/ no improvement
    #                               min_lr=1e-6, # Min lr
    #                               verbose=1)

    #tensorboard = TensorBoard(log_dir='./logs', # Logs directory
    #                           histogram_freq=1, # Logs histograms for weights/acc
    #                           write_graph=True, # Logs graph of model
    #                           write_images=True) # Log images like weight histograms

    checkpoint = ModelCheckpoint('best_model_v2.keras', # Path where the best model will be saved
                                monitor='val_loss', # What to monitor (validation loss)
                                save_best_only=True, # Only save model when val loss is best
                                mode='min', # Looking to minimize the validation loss
                                verbose=1) # Print out message when saving model

    checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v2.keras', # Path where the best model will be saved
                                            monitor='val_loss', # What to monitor (validation loss)
                                            save_best_only=True, # Only save model when val loss is best
                                            mode='min', # Looking to minimize the validation loss
                                            verbose=1) # Print out message when saving model

    #cvs_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

    # Train the model
    history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                        batch_size=batch_size, callbacks=[early_stopping, checkpoint])

    ##### <<<<<<<<Plot Training & Validation Error>>>>>>>>

    # Extract loss and accuracy
    epochs = range(1, len(history.history['loss'])+1)
    train_loss = history.history['loss']
    val_loss = history.history['val_loss']
    train_acc = history.history['accuracy']

```

```

val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v2.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<Fine-Tune>>>>>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

```

```

#### <<<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                          .batch(batch_size)
                          .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v2.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')

```



```
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

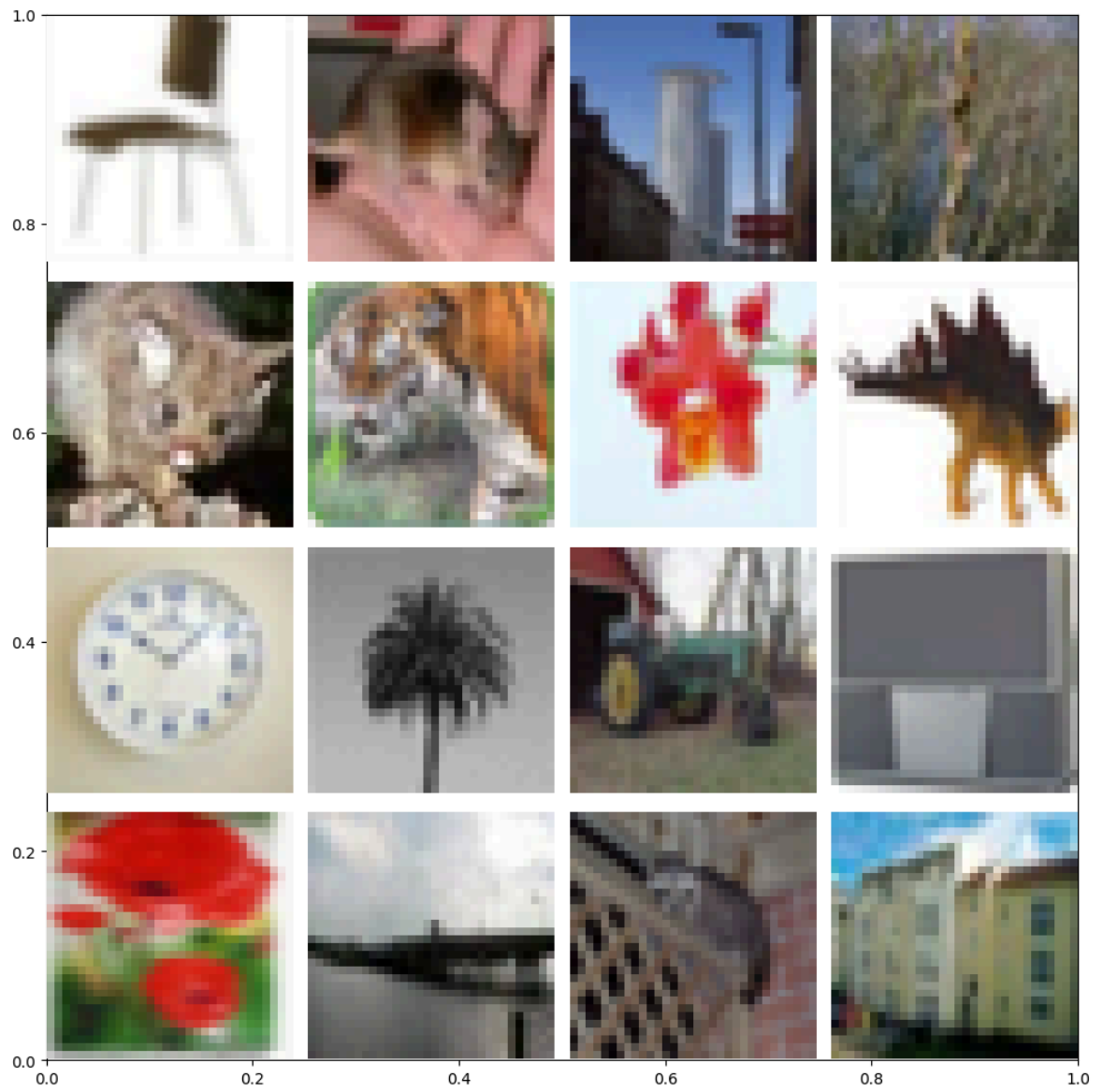
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

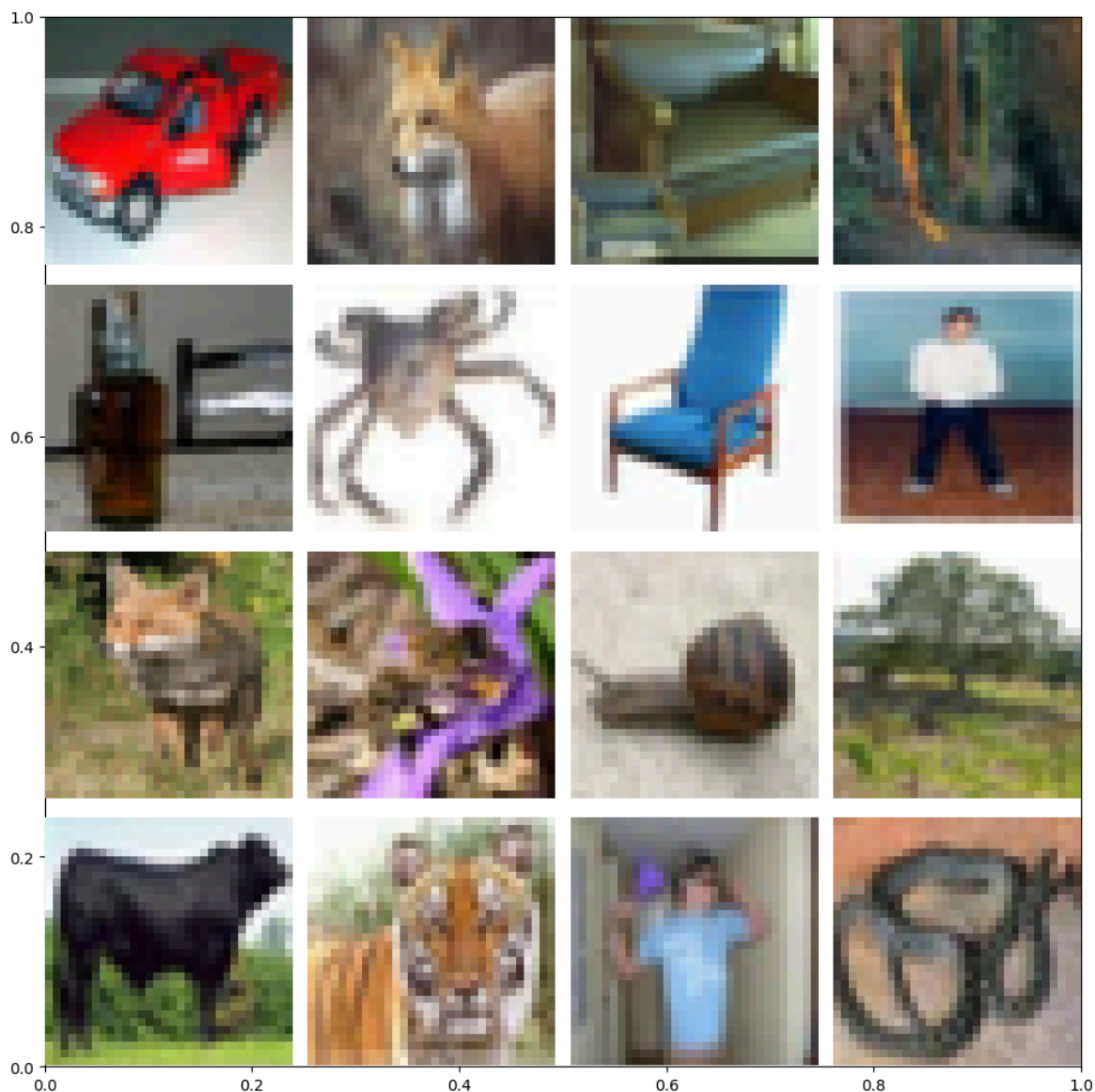
labels[30859].shape: (1,)

imgs[30859].shape: (32, 32, 3)



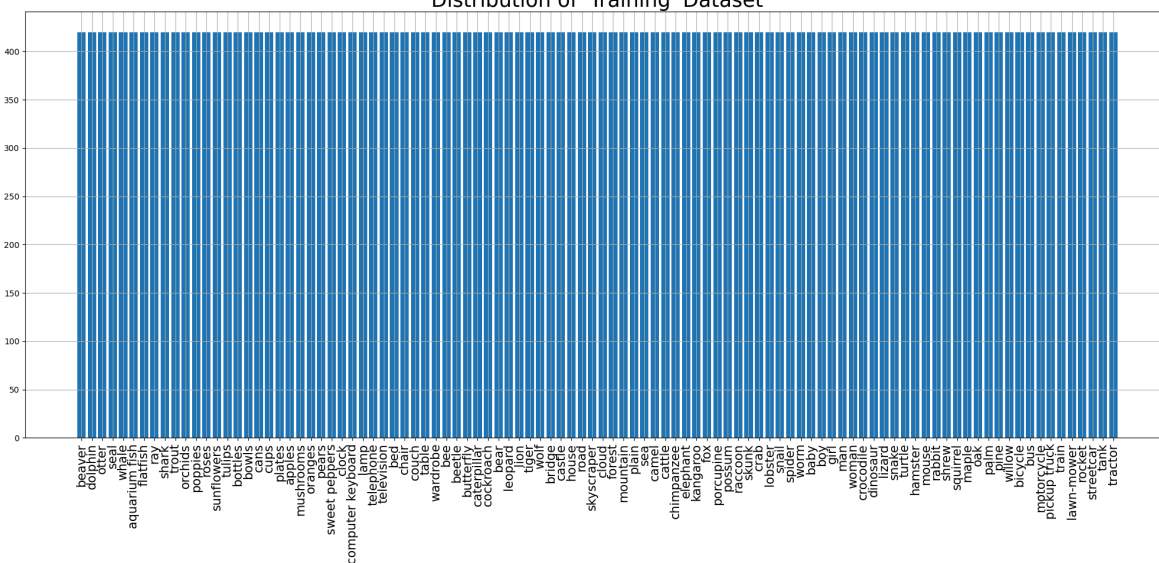
labels[37842].shape: (1,)

imgs[37842].shape: (32, 32, 3)

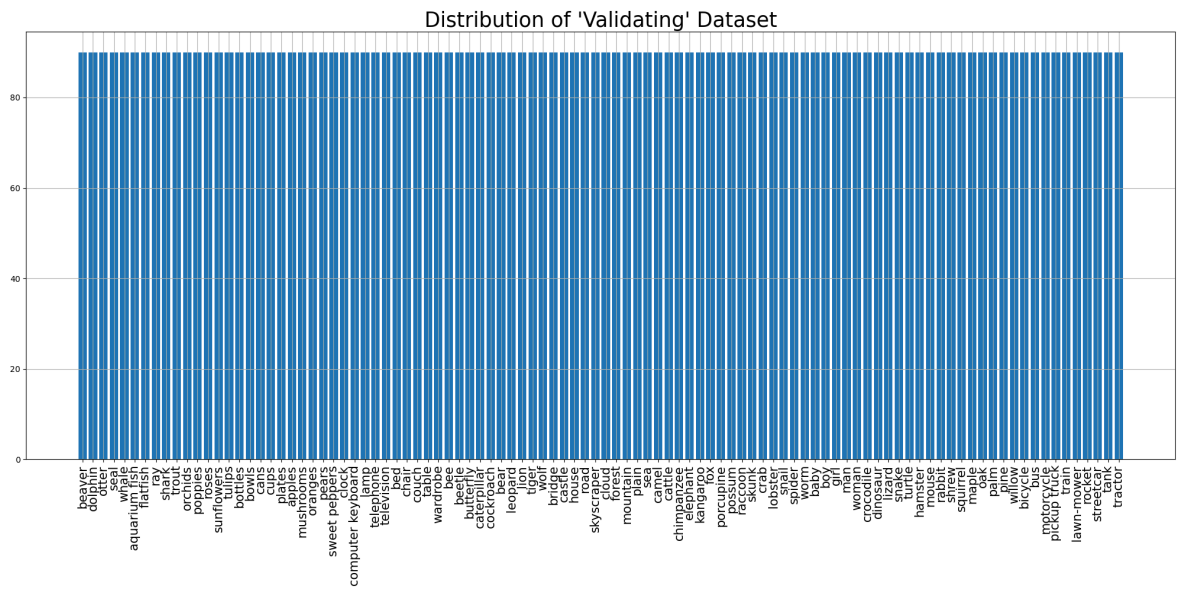


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

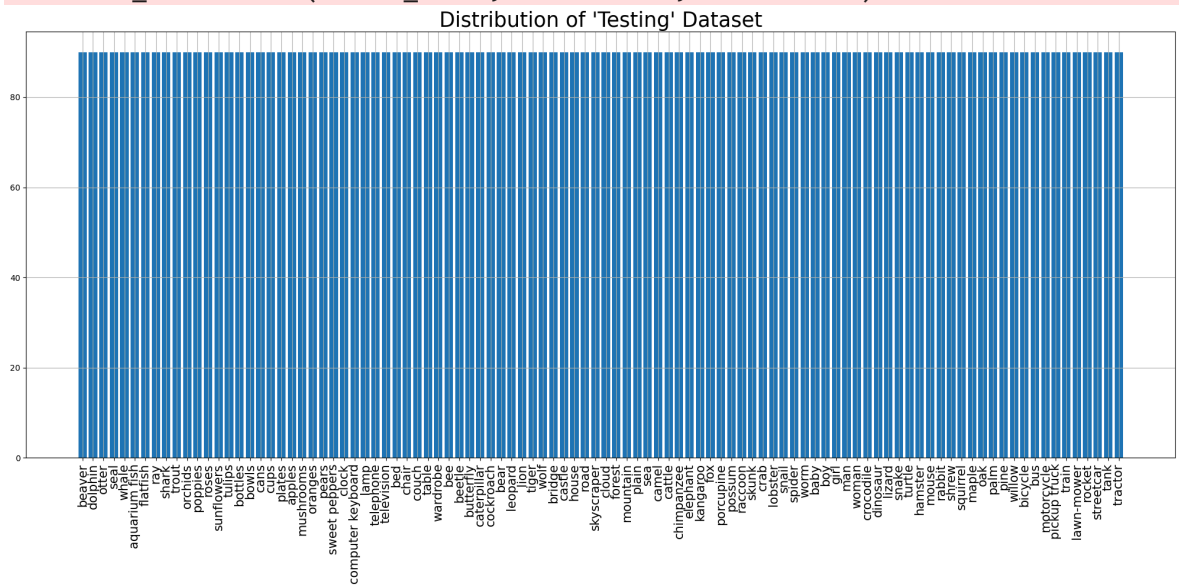
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_2"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None , 2048)
dense_4 (Dense)	(None , 512)
dropout_2 (Dropout)	(None , 512)
dense_5 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— **0s** 29ms/step - accuracy: 0.1877 - f1_score: 0.1795 - loss: 3.6425 - precision: 0.5631
Epoch 1: val_loss improved from inf to 1.78734, saving model to best_model_v2.keras as
657/657 ————— **39s** 44ms/step - accuracy: 0.1879 - f1_score: 0.1797 - loss: 3.6415 - precision: 0.5633 - val_accuracy: 0.5410 - val_f1_score: 0.5302 - val_loss: 1.7873 - val_precision: 0.8693
Epoch 2/25
656/657 ————— **0s** 24ms/step - accuracy: 0.3897 - f1_score: 0.3820 - loss: 2.3149 - precision: 0.7015
Epoch 2: val_loss improved from 1.78734 to 1.67195, saving model to best_model_v2.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.3897 - f1_score: 0.3820 - loss: 2.3147 - precision: 0.7015 - val_accuracy: 0.5587 - val_f1_score: 0.5491 - val_loss: 1.6719 - val_precision: 0.8618
Epoch 3/25
657/657 ————— **0s** 23ms/step - accuracy: 0.4152 - f1_score: 0.4072 - loss: 2.1740 - precision: 0.7186
Epoch 3: val_loss improved from 1.67195 to 1.64858, saving model to best_model_v2.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.4153 - f1_score: 0.4072 - loss: 2.1739 - precision: 0.7186 - val_accuracy: 0.5639 - val_f1_score: 0.5571 - val_loss: 1.6486 - val_precision: 0.8661
Epoch 4/25
657/657 ————— **0s** 23ms/step - accuracy: 0.4285 - f1_score: 0.4223 - loss: 2.1149 - precision: 0.7333
Epoch 4: val_loss improved from 1.64858 to 1.62482, saving model to best_model_v2.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.4285 - f1_score: 0.4223 - loss: 2.1148 - precision: 0.7333 - val_accuracy: 0.5642 - val_f1_score: 0.5546 - val_loss: 1.6248 - val_precision: 0.8594
Epoch 5/25
656/657 ————— **0s** 23ms/step - accuracy: 0.4325 - f1_score: 0.4251 - loss: 2.0826 - precision: 0.7309
Epoch 5: val_loss improved from 1.62482 to 1.61989, saving model to best_model_v2.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.4325 - f1_score: 0.4251 - loss: 2.0826 - precision: 0.7309 - val_accuracy: 0.5729 - val_f1_score: 0.5608 - val_loss: 1.6199 - val_precision: 0.8599
Epoch 6/25
657/657 ————— **0s** 23ms/step - accuracy: 0.4348 - f1_score: 0.4277 - loss: 2.0724 - precision: 0.7276
Epoch 6: val_loss improved from 1.61989 to 1.58545, saving model to best_model_v2.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.4348 - f1_score: 0.4277 - loss: 2.0723 - precision: 0.7277 - val_accuracy: 0.5730 - val_f1_score: 0.5645 - val_loss: 1.5855 - val_precision: 0.8603
Epoch 7/25
657/657 ————— **0s** 23ms/step - accuracy: 0.4448 - f1_score: 0.4372 - loss: 2.0501 - precision: 0.7344
Epoch 7: val_loss did not improve from 1.58545
657/657 ————— **18s** 28ms/step - accuracy: 0.4448 - f1_score: 0.4372 - loss: 2.0501 - precision: 0.7344 - val_accuracy: 0.5678 - val_f1_score: 0.5580 - val_loss: 1.5948 - val_precision: 0.8582
Epoch 8/25
656/657 ————— **0s** 23ms/step - accuracy: 0.4479 - f1_score: 0.4413 - loss: 2.0485 - precision: 0.7328
Epoch 8: val_loss improved from 1.58545 to 1.58275, saving model to best_model_v2.keras

657/657  **19s** 30ms/step - accuracy: 0.4479 - f1_score: 0.4413
- loss: 2.0485 - precision: 0.7328 - val_accuracy: 0.5731 - val_f1_score: 0.5646
- val_loss: 1.5828 - val_precision: 0.8532
Epoch 9/25

655/657  **0s** 23ms/step - accuracy: 0.4436 - f1_score: 0.4363 -
loss: 2.0413 - precision: 0.7321
Epoch 9: val_loss improved from 1.58275 to 1.57073, saving model to best_model_v
2.keras

657/657  **19s** 30ms/step - accuracy: 0.4436 - f1_score: 0.4364
- loss: 2.0412 - precision: 0.7321 - val_accuracy: 0.5757 - val_f1_score: 0.5666
- val_loss: 1.5707 - val_precision: 0.8582
Epoch 10/25

656/657  **0s** 23ms/step - accuracy: 0.4492 - f1_score: 0.4424 -
loss: 2.0297 - precision: 0.7343
Epoch 10: val_loss did not improve from 1.57073

657/657  **18s** 28ms/step - accuracy: 0.4492 - f1_score: 0.4424
- loss: 2.0296 - precision: 0.7343 - val_accuracy: 0.5697 - val_f1_score: 0.5607
- val_loss: 1.5775 - val_precision: 0.8458
Epoch 11/25

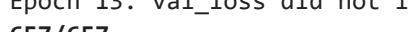
656/657  **0s** 23ms/step - accuracy: 0.4443 - f1_score: 0.4381 -
loss: 2.0295 - precision: 0.7252
Epoch 11: val_loss did not improve from 1.57073

657/657  **18s** 28ms/step - accuracy: 0.4443 - f1_score: 0.4381
- loss: 2.0294 - precision: 0.7253 - val_accuracy: 0.5744 - val_f1_score: 0.5634
- val_loss: 1.5911 - val_precision: 0.8695
Epoch 12/25

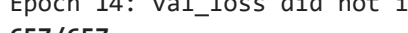
655/657  **0s** 23ms/step - accuracy: 0.4470 - f1_score: 0.4395 -
loss: 2.0285 - precision: 0.7318
Epoch 12: val_loss did not improve from 1.57073

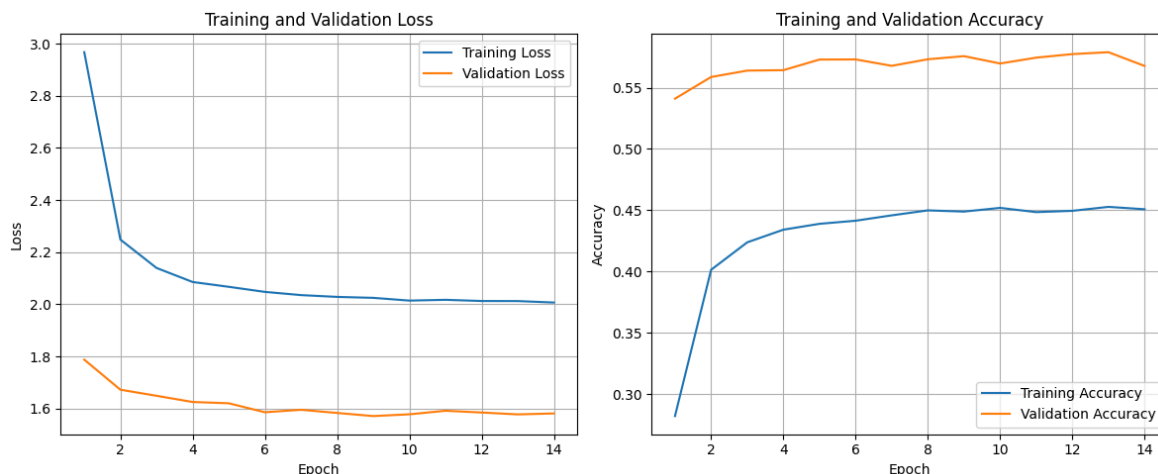
657/657  **18s** 28ms/step - accuracy: 0.4470 - f1_score: 0.4395
- loss: 2.0284 - precision: 0.7318 - val_accuracy: 0.5773 - val_f1_score: 0.5680
- val_loss: 1.5844 - val_precision: 0.8605
Epoch 13/25

656/657  **0s** 23ms/step - accuracy: 0.4483 - f1_score: 0.4416 -
loss: 2.0232 - precision: 0.7328
Epoch 13: val_loss did not improve from 1.57073

657/657  **18s** 28ms/step - accuracy: 0.4483 - f1_score: 0.4416
- loss: 2.0232 - precision: 0.7328 - val_accuracy: 0.5789 - val_f1_score: 0.5709
- val_loss: 1.5772 - val_precision: 0.8582
Epoch 14/25

657/657  **0s** 23ms/step - accuracy: 0.4458 - f1_score: 0.4396 -
loss: 2.0224 - precision: 0.7339
Epoch 14: val_loss did not improve from 1.57073

657/657  **19s** 28ms/step - accuracy: 0.4458 - f1_score: 0.4396
- loss: 2.0223 - precision: 0.7339 - val_accuracy: 0.5677 - val_f1_score: 0.5588
- val_loss: 1.5808 - val_precision: 0.8426



141/141 ————— **11s** 37ms/step - accuracy: 0.5628 - f1_score: 0.5499

- loss: 1.5834 - precision: 0.8503

Test Accuracy: 57.27%

Test Loss: 1.5580322742462158

Test Precision: 86.16%

Test F1 Scores (Per Class): [80.24691 72.625694 54.08804 27.199995 28.77697
4 55.27637

49.606293	58.510624	77.10842	81.81818	59.999996	45.71428
61.538452	49.137924	62.33766	47.111107	73.224045	57.564575
51.396645	52.702694	76.756744	72.98578	77.38094	71.428566
70.15707	58.24175	42.051273	27.672949	72.99999	59.016384
32.061066	67.05882	34.567894	40.776695	61.290314	4.1237106
67.46988	33.33333	38.94736	85.	60.689648	80.
52.307682	62.264145	48.958324	30.534348	45.814972	48.249023
72.46376	78.57143	34.693874	64.44444	50.86206	81.52173
56.41025	18.309856	71.33758	64.	66.666664	27.027023
72.53885	70.243904	55.999992	63.101597	38.32335	36.496346
56.57894	50.67873	79.61165	74.35896	62.559242	67.75956
33.684204	51.219505	35.999996	75.97765	71.36563	49.246223
62.72189	61.03895	28.220854	43.92523	78.97436	64.739876
48.648647	68.74999	70.93597	75.4491	50.48543	61.176468
45.66929	60.63829	48.27586	50.574715	80.38277	55.9322
19.354836	57.297295	40.723976	75.97765	]	

Average Test F1 Scores:56.50

282/282 ————— **11s** 27ms/step



	precision	recall	f1-score	support
beaver	0.90	0.72	0.80	90
dolphin	0.73	0.72	0.73	90
otter	0.61	0.48	0.54	90
seal	0.49	0.19	0.27	90
whale	0.41	0.22	0.29	90
aquarium fish	0.50	0.61	0.55	90
flatfish	0.38	0.70	0.50	90
ray	0.56	0.61	0.59	90
shark	0.84	0.71	0.77	90
trout	0.84	0.80	0.82	90
orchids	0.75	0.50	0.60	90
poppies	0.40	0.52	0.45	90
roses	0.83	0.49	0.62	90
sunflowers	0.40	0.63	0.49	90
tulips	0.75	0.53	0.62	90
bottles	0.39	0.59	0.47	90
bowls	0.72	0.74	0.73	90
cans	0.43	0.87	0.58	90
cups	0.52	0.51	0.51	90
plates	0.66	0.43	0.52	90
apples	0.75	0.79	0.77	90
mushrooms	0.64	0.86	0.73	90
oranges	0.83	0.72	0.77	90
pears	0.77	0.67	0.71	90
sweet peppers	0.66	0.74	0.70	90
clock	0.58	0.59	0.58	90
computer keyboard	0.39	0.46	0.42	90
lamp	0.31	0.23	0.27	90
telephone	0.66	0.81	0.73	90
television	0.58	0.60	0.59	90
bed	0.51	0.23	0.32	90
chair	0.71	0.63	0.67	90
couch	0.39	0.31	0.35	90
table	0.29	0.70	0.41	90
wardrobe	0.59	0.63	0.61	90
bee	0.29	0.02	0.04	90
beetle	0.74	0.62	0.67	90
butterfly	0.58	0.23	0.33	90
caterpillar	0.37	0.41	0.39	90
cockroach	0.97	0.76	0.85	90
bear	0.80	0.49	0.61	90
leopard	0.82	0.78	0.80	90
lion	0.85	0.38	0.52	90
tiger	0.54	0.73	0.62	90
wolf	0.47	0.53	0.50	90
bridge	0.49	0.22	0.31	90
castle	0.38	0.58	0.46	90
house	0.37	0.69	0.48	90
road	0.64	0.83	0.72	90
skyscraper	0.85	0.73	0.79	90
cloud	0.32	0.38	0.35	90
forest	0.64	0.64	0.64	90
mountain	0.42	0.66	0.51	90
plain	0.80	0.83	0.82	90
sea	0.67	0.49	0.56	90
camel	0.25	0.14	0.18	90
cattle	0.84	0.62	0.71	90
chimpanzee	0.53	0.80	0.64	90

elephant	0.86	0.54	0.67	90
kangaroo	0.71	0.17	0.27	90
fox	0.69	0.78	0.73	90
porcupine	0.63	0.80	0.70	90
possum	0.70	0.47	0.56	90
raccoon	0.61	0.66	0.63	90
skunk	0.42	0.36	0.38	90
crab	0.54	0.28	0.37	90
lobster	0.69	0.48	0.57	90
snail	0.43	0.62	0.51	90
spider	0.71	0.91	0.80	90
worm	0.88	0.64	0.74	90
baby	0.55	0.73	0.63	90
boy	0.67	0.70	0.68	90
girl	0.32	0.36	0.34	90
man	0.57	0.47	0.51	90
woman	0.45	0.30	0.36	90
crocodile	0.76	0.76	0.76	90
dinosaur	0.59	0.90	0.71	90
lizard	0.45	0.54	0.49	90
snake	0.67	0.59	0.63	90
turtle	0.75	0.52	0.61	90
hamster	0.32	0.26	0.28	90
mouse	0.38	0.52	0.44	90
rabbit	0.73	0.86	0.79	90
shrew	0.67	0.62	0.65	90
squirrel	0.62	0.40	0.49	90
maple	0.79	0.61	0.69	90
oak	0.64	0.80	0.71	90
palm	0.82	0.69	0.75	90
pine	0.45	0.58	0.50	90
willow	0.65	0.58	0.61	90
bicycle	0.78	0.32	0.46	90
bus	0.58	0.63	0.61	90
motorcycle	0.44	0.56	0.49	90
pickup truck	0.52	0.49	0.51	90
train	0.71	0.93	0.80	90
lawn-mower	0.45	0.73	0.56	90
rocket	0.35	0.13	0.19	90
streetcar	0.56	0.59	0.57	90
tank	0.34	0.50	0.41	90
tractor	0.76	0.76	0.76	90
accuracy			0.57	9000
macro avg	0.60	0.57	0.57	9000
weighted avg	0.60	0.57	0.57	9000

Model: "sequential_2"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None , 2048)
dense_4 (Dense)	(None , 512)
dropout_2 (Dropout)	(None , 512)
dense_5 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1314/1314 ————— 0s 26ms/step - accuracy: 0.4699 - f1_score: 0.4628
- loss: 1.9326 - precision: 0.7463
Epoch 1: val_loss improved from inf to 1.48361, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 55s 34ms/step - accuracy: 0.4700 - f1_score: 0.4628 - loss: 1.9325 - precision: 0.7463 - val_accuracy: 0.5981 - val_f1_score: 0.5924 - val_loss: 1.4836 - val_precision: 0.8636
Epoch 2/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.4932 - f1_score: 0.4866 - loss: 1.8242 - precision: 0.7660
Epoch 2: val_loss improved from 1.48361 to 1.46176, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.4932 - f1_score: 0.4866 - loss: 1.8241 - precision: 0.7661 - val_accuracy: 0.6027 - val_f1_score: 0.5981 - val_loss: 1.4618 - val_precision: 0.8602
Epoch 3/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.5062 - f1_score: 0.5004 - loss: 1.7914 - precision: 0.7671
Epoch 3: val_loss improved from 1.46176 to 1.44891, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5062 - f1_score: 0.5004 - loss: 1.7913 - precision: 0.7671 - val_accuracy: 0.6058 - val_f1_score: 0.6016 - val_loss: 1.4489 - val_precision: 0.8589
Epoch 4/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.5113 - f1_score: 0.5058 - loss: 1.7622 - precision: 0.7759
Epoch 4: val_loss improved from 1.44891 to 1.43903, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5113 - f1_score: 0.5058 - loss: 1.7622 - precision: 0.7759 - val_accuracy: 0.6078 - val_f1_score: 0.6036 - val_loss: 1.4390 - val_precision: 0.8569
Epoch 5/15
1312/1314 ————— 0s 23ms/step - accuracy: 0.5193 - f1_score: 0.5135 - loss: 1.7368 - precision: 0.7811
Epoch 5: val_loss improved from 1.43903 to 1.42922, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5193 - f1_score: 0.5135 - loss: 1.7367 - precision: 0.7811 - val_accuracy: 0.6086 - val_f1_score: 0.6046 - val_loss: 1.4292 - val_precision: 0.8554
Epoch 6/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.5224 - f1_score: 0.5169 - loss: 1.7304 - precision: 0.7874
Epoch 6: val_loss improved from 1.42922 to 1.42091, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5224 - f1_score: 0.5169 - loss: 1.7304 - precision: 0.7874 - val_accuracy: 0.6100 - val_f1_score: 0.6059 - val_loss: 1.4209 - val_precision: 0.8559
Epoch 7/15
1313/1314 ————— 0s 23ms/step - accuracy: 0.5264 - f1_score: 0.5209 - loss: 1.7048 - precision: 0.7895
Epoch 7: val_loss improved from 1.42091 to 1.41444, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5264 - f1_score: 0.5209 - loss: 1.7048 - precision: 0.7895 - val_accuracy: 0.6097 - val_f1_score: 0.6058 - val_loss: 1.4144 - val_precision: 0.8546
Epoch 8/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.5300 - f1_score: 0.5245 - loss: 1.6877 - precision: 0.7895
Epoch 8: val_loss improved from 1.41444 to 1.40678, saving model to best_model_fine_tune_v2.keras

ne_tune_v2.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.5300 - f1_score: 0.5246 - loss: 1.6877 - precision: 0.7895 - val_accuracy: 0.6121 - val_f1_score: 0.6082 - val_loss: 1.4068 - val_precision: 0.8541

Epoch 9/15

1312/1314 ————— 0s 23ms/step - accuracy: 0.5373 - f1_score: 0.5317 - loss: 1.6612 - precision: 0.7967

Epoch 9: val_loss improved from 1.40678 to 1.40094, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.5373 - f1_score: 0.5318 - loss: 1.6612 - precision: 0.7967 - val_accuracy: 0.6133 - val_f1_score: 0.6093 - val_loss: 1.4009 - val_precision: 0.8542

Epoch 10/15

1314/1314 ————— 0s 23ms/step - accuracy: 0.5405 - f1_score: 0.5350 - loss: 1.6578 - precision: 0.7925

Epoch 10: val_loss improved from 1.40094 to 1.39495, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 35s 26ms/step - accuracy: 0.5405 - f1_score: 0.5350 - loss: 1.6578 - precision: 0.7926 - val_accuracy: 0.6121 - val_f1_score: 0.6085 - val_loss: 1.3950 - val_precision: 0.8536

Epoch 11/15

1312/1314 ————— 0s 23ms/step - accuracy: 0.5434 - f1_score: 0.5386 - loss: 1.6391 - precision: 0.8005

Epoch 11: val_loss improved from 1.39495 to 1.39006, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 35s 26ms/step - accuracy: 0.5434 - f1_score: 0.5386 - loss: 1.6391 - precision: 0.8005 - val_accuracy: 0.6133 - val_f1_score: 0.6097 - val_loss: 1.3901 - val_precision: 0.8522

Epoch 12/15

1314/1314 ————— 0s 23ms/step - accuracy: 0.5481 - f1_score: 0.5435 - loss: 1.6266 - precision: 0.8020

Epoch 12: val_loss improved from 1.39006 to 1.38399, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 35s 26ms/step - accuracy: 0.5481 - f1_score: 0.5435 - loss: 1.6266 - precision: 0.8020 - val_accuracy: 0.6143 - val_f1_score: 0.6105 - val_loss: 1.3840 - val_precision: 0.8514

Epoch 13/15

1314/1314 ————— 0s 23ms/step - accuracy: 0.5509 - f1_score: 0.5458 - loss: 1.6153 - precision: 0.8009

Epoch 13: val_loss improved from 1.38399 to 1.37869, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.5509 - f1_score: 0.5458 - loss: 1.6153 - precision: 0.8009 - val_accuracy: 0.6167 - val_f1_score: 0.6130 - val_loss: 1.3787 - val_precision: 0.8513

Epoch 14/15

1312/1314 ————— 0s 23ms/step - accuracy: 0.5526 - f1_score: 0.5473 - loss: 1.6095 - precision: 0.8022

Epoch 14: val_loss improved from 1.37869 to 1.37513, saving model to best_model_fine_tune_v2.keras

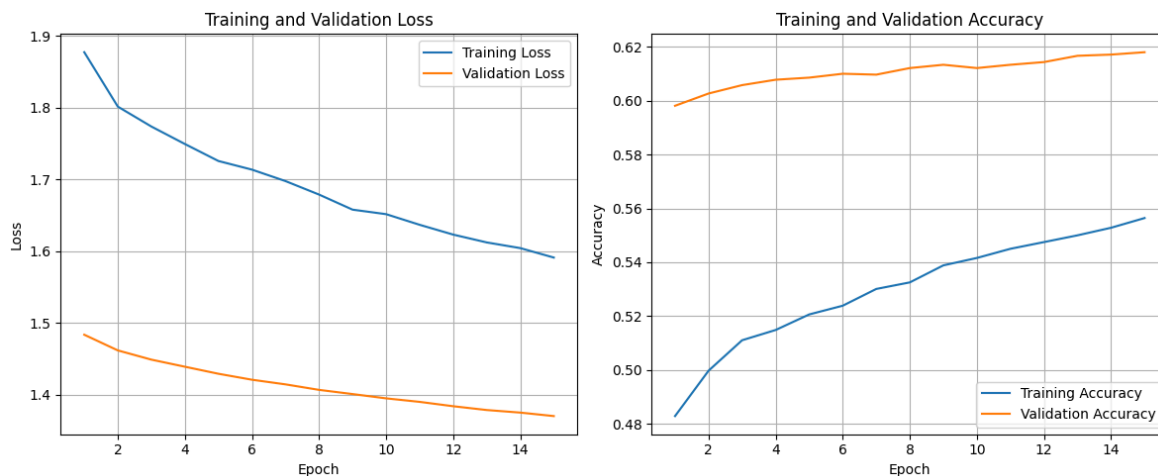
1314/1314 ————— 35s 26ms/step - accuracy: 0.5526 - f1_score: 0.5473 - loss: 1.6095 - precision: 0.8022 - val_accuracy: 0.6171 - val_f1_score: 0.6137 - val_loss: 1.3751 - val_precision: 0.8517

Epoch 15/15

1312/1314 ————— 0s 23ms/step - accuracy: 0.5539 - f1_score: 0.5488 - loss: 1.5956 - precision: 0.8061

Epoch 15: val_loss improved from 1.37513 to 1.37028, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 35s 26ms/step - accuracy: 0.5539 - f1_score: 0.5488 - loss: 1.5956 - precision: 0.8061 - val_accuracy: 0.6180 - val_f1_score: 0.6145 - val_loss: 1.3703 - val_precision: 0.8501



141/141 ————— **11s** 37ms/step - accuracy: 0.6112 - f1_score: 0.6020

- loss: 1.3786 - precision: 0.8474

Test Accuracy: 62.29%

Test Loss: 1.3537427186965942

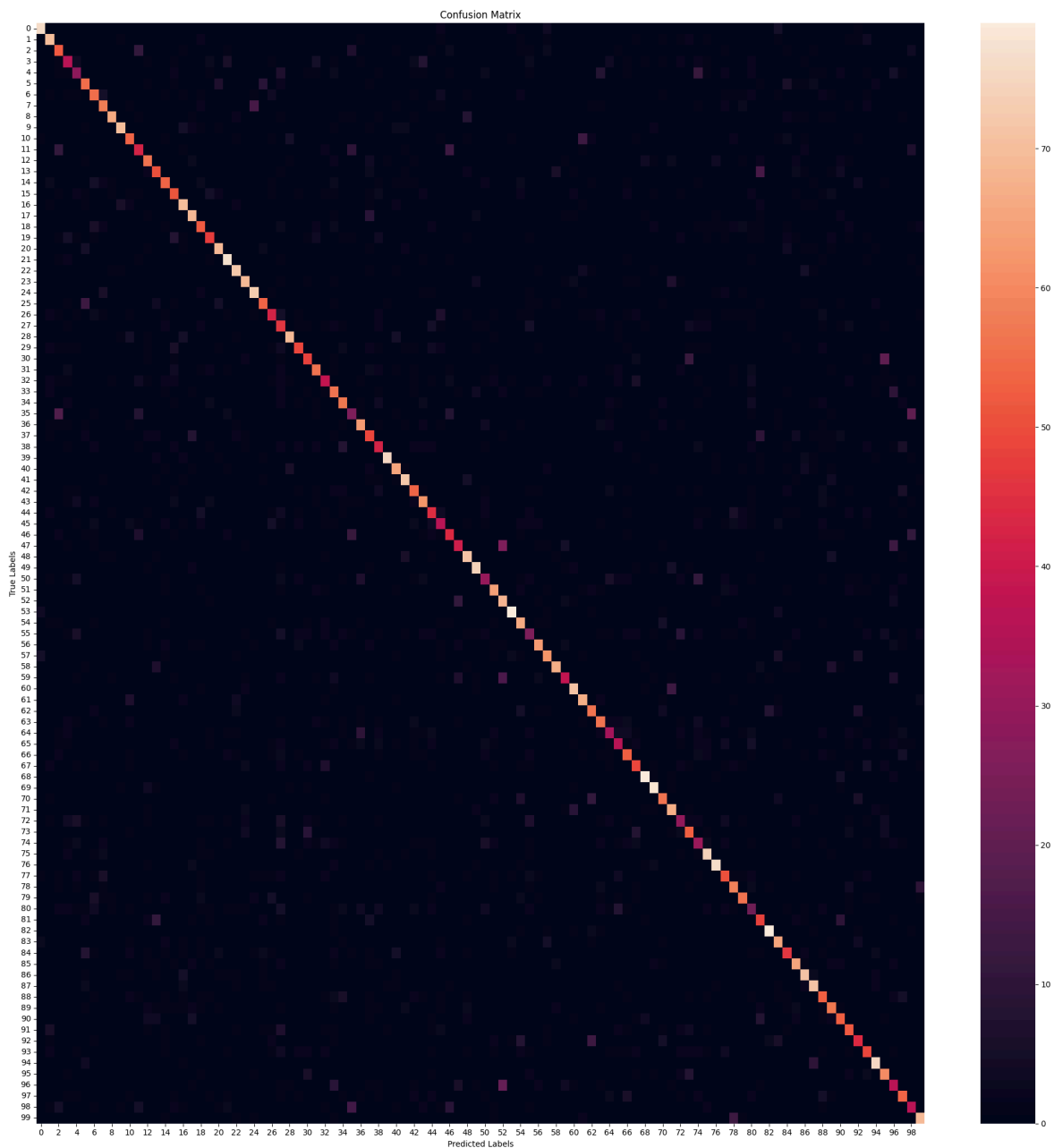
Test Precision: 85.99%

Test F1 Scores (Per Class): [84.44444 72.53885 53.06122 44.31137 34.355824 5
6.994812 57.894737

64.088394	79.99999	81.39534	60.67415	48.554905	61.87845	56.818176
62.427734	55.737698	71.428566	71.87499	57.45856	57.48503	75.82417
79.38144	77.41935	76.24309	78.07486	64.28571	51.219505	38.81856
79.31035	60.869564	56.140347	65.49707	46.62576	54.36893	61.956512
29.447845	67.37968	52.68817	46.92737	88.2353	70.27026	78.260864
60.33519	67.02702	50.279324	44.31137	50.28571	51.25	73.57513
80.43478	37.735844	72.31638	60.98654	86.81318	65.346535	31.055897
73.68421	70.05649	78.82352	51.00671	76.34408	72.04301	61.202175
61.87845	42.58064	44.444443	58.88888	54.748596	84.946236	82.53968
64.36781	70.15707	36.708855	57.92349	33.879776	77.48691	84.91619
56.497173	63.101597	66.666664	27.380949	50.526314	86.81318	67.357506
54.117645	70.27026	74.34555	74.73684	56.382973	60.96256	59.21787
59.090908	51.764698	56.32183	81.52173	63.21243	39.3617	59.66851
42.04545	76.24309					

Average Test F1 Scores:62.01

282/282 ————— **11s** 27ms/step



	precision	recall	f1-score	support
beaver	0.84	0.84	0.84	90
dolphin	0.68	0.78	0.73	90
otter	0.50	0.58	0.53	90
seal	0.47	0.41	0.44	90
whale	0.38	0.31	0.34	90
aquarium fish	0.53	0.61	0.57	90
flatfish	0.55	0.61	0.58	90
ray	0.64	0.64	0.64	90
shark	0.88	0.73	0.80	90
trout	0.85	0.78	0.81	90
orchids	0.61	0.60	0.61	90
poppies	0.51	0.47	0.49	90
roses	0.62	0.62	0.62	90
sunflowers	0.58	0.56	0.57	90
tulips	0.65	0.60	0.62	90
bottles	0.55	0.57	0.56	90
bowls	0.66	0.78	0.71	90
cans	0.68	0.77	0.72	90
cups	0.57	0.58	0.57	90
plates	0.62	0.52	0.57	90
apples	0.75	0.77	0.76	90
mushrooms	0.74	0.86	0.79	90
oranges	0.75	0.80	0.77	90
pears	0.76	0.77	0.76	90
sweet peppers	0.75	0.81	0.78	90
clock	0.69	0.60	0.64	90
computer keyboard	0.56	0.47	0.51	90
lamp	0.31	0.51	0.39	90
telephone	0.82	0.77	0.79	90
television	0.69	0.54	0.61	90
bed	0.59	0.53	0.56	90
chair	0.69	0.62	0.65	90
couch	0.52	0.42	0.47	90
table	0.48	0.62	0.54	90
wardrobe	0.61	0.63	0.62	90
bee	0.34	0.28	0.30	90
beetle	0.65	0.70	0.67	90
butterfly	0.51	0.54	0.53	90
caterpillar	0.47	0.47	0.47	90
cockroach	0.94	0.83	0.88	90
bear	0.68	0.72	0.70	90
leopard	0.77	0.80	0.78	90
lion	0.61	0.60	0.60	90
tiger	0.65	0.69	0.67	90
wolf	0.51	0.50	0.50	90
bridge	0.47	0.40	0.43	90
castle	0.52	0.49	0.50	90
house	0.59	0.46	0.51	90
road	0.69	0.79	0.74	90
skyscraper	0.79	0.82	0.80	90
cloud	0.43	0.33	0.38	90
forest	0.74	0.71	0.72	90
mountain	0.51	0.76	0.61	90
plain	0.86	0.88	0.87	90
sea	0.59	0.73	0.65	90
camel	0.35	0.28	0.31	90
cattle	0.78	0.70	0.74	90
chimpanzee	0.71	0.69	0.70	90

elephant	0.84	0.74	0.79	90
kangaroo	0.66	0.42	0.51	90
fox	0.74	0.79	0.76	90
porcupine	0.70	0.74	0.72	90
possum	0.60	0.62	0.61	90
raccoon	0.62	0.62	0.62	90
skunk	0.51	0.37	0.43	90
crab	0.50	0.40	0.44	90
lobster	0.59	0.59	0.59	90
snail	0.56	0.54	0.55	90
spider	0.82	0.88	0.85	90
worm	0.79	0.87	0.83	90
baby	0.67	0.62	0.64	90
boy	0.66	0.74	0.70	90
girl	0.43	0.32	0.37	90
man	0.57	0.59	0.58	90
woman	0.33	0.34	0.34	90
crocodile	0.73	0.82	0.77	90
dinosaur	0.85	0.84	0.85	90
lizard	0.57	0.56	0.56	90
snake	0.61	0.66	0.63	90
turtle	0.72	0.62	0.67	90
hamster	0.29	0.26	0.27	90
mouse	0.48	0.53	0.51	90
rabbit	0.86	0.88	0.87	90
shrew	0.63	0.72	0.67	90
squirrel	0.57	0.51	0.54	90
maple	0.68	0.72	0.70	90
oak	0.70	0.79	0.74	90
palm	0.71	0.79	0.75	90
pine	0.54	0.59	0.56	90
willow	0.59	0.63	0.61	90
bicycle	0.60	0.59	0.59	90
bus	0.60	0.58	0.59	90
motorcycle	0.55	0.49	0.52	90
pickup truck	0.58	0.54	0.56	90
train	0.80	0.83	0.82	90
lawn-mower	0.59	0.68	0.63	90
rocket	0.38	0.41	0.39	90
streetcar	0.59	0.60	0.60	90
tank	0.43	0.41	0.42	90
tractor	0.76	0.77	0.76	90
accuracy			0.62	9000
macro avg	0.62	0.62	0.62	9000
weighted avg	0.62	0.62	0.62	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

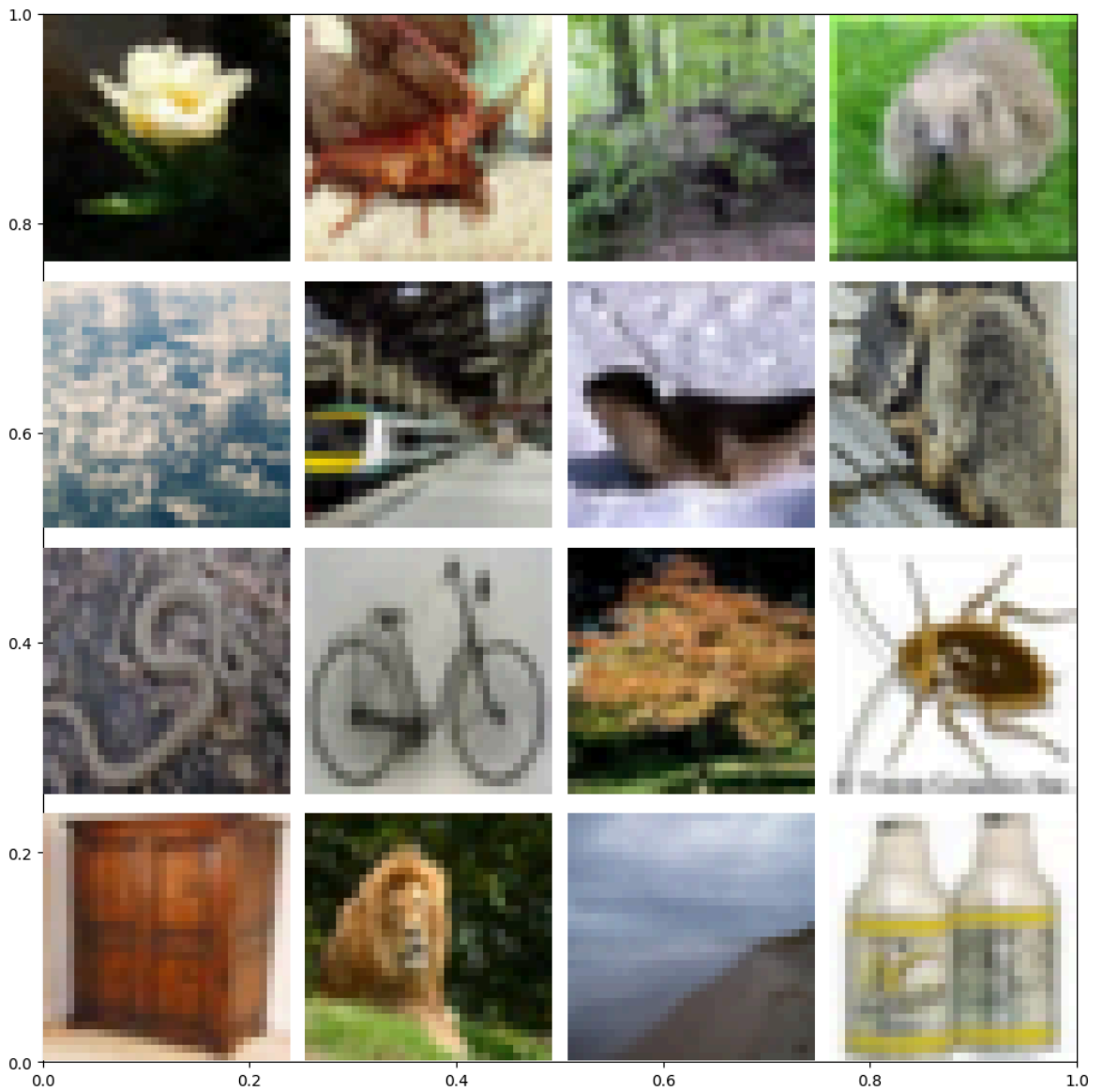
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

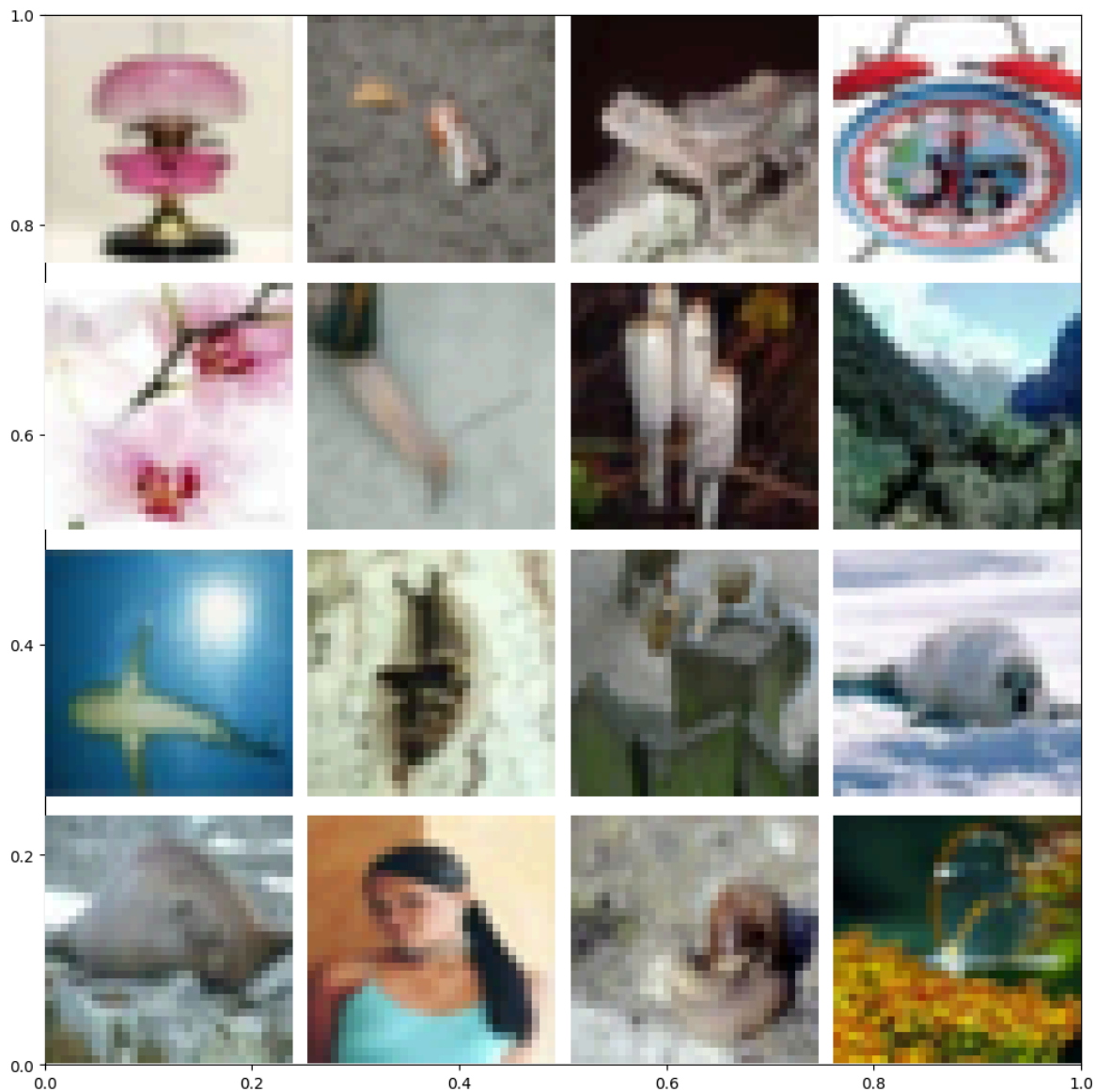
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[522].shape: (1,)
imgs[522].shape: (32, 32, 3)
```

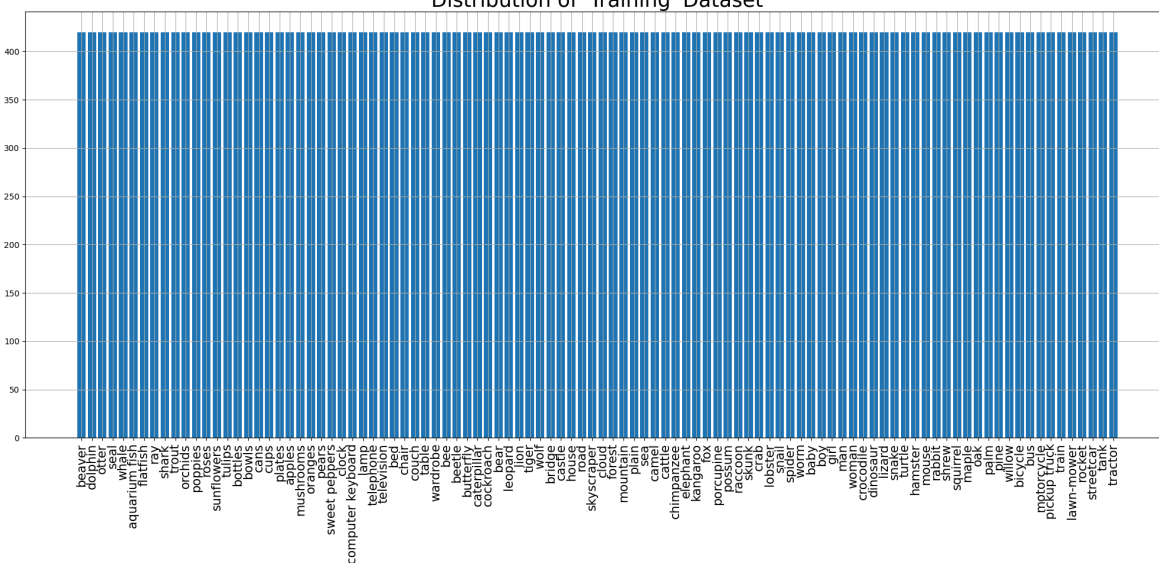


```
labels[787].shape: (1,)  
imgs[787].shape: (32, 32, 3)
```

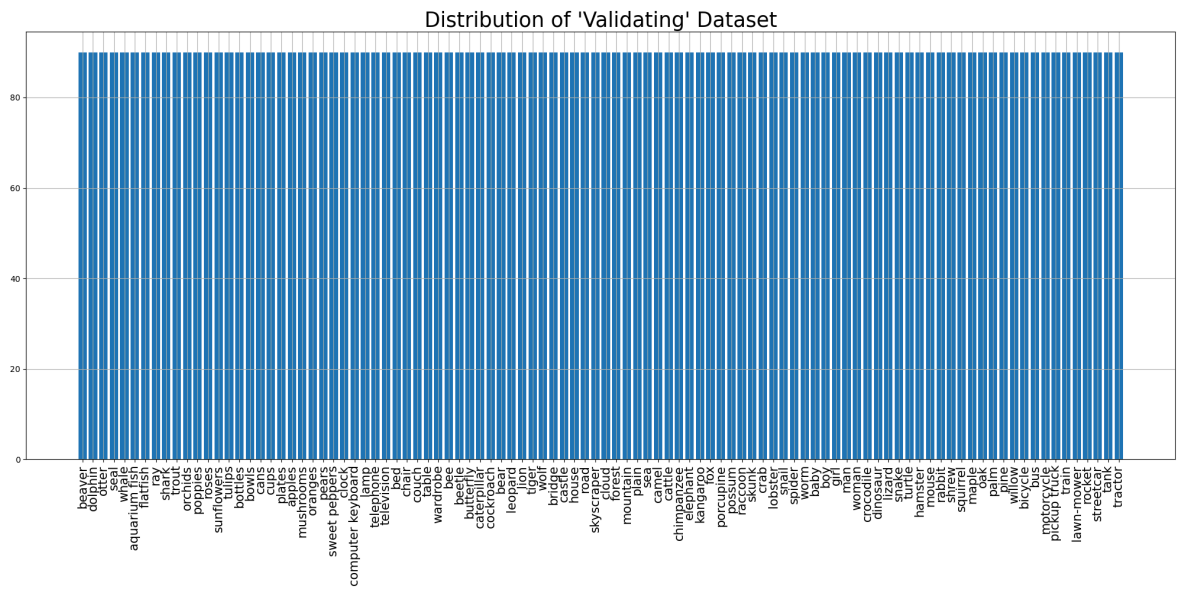


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

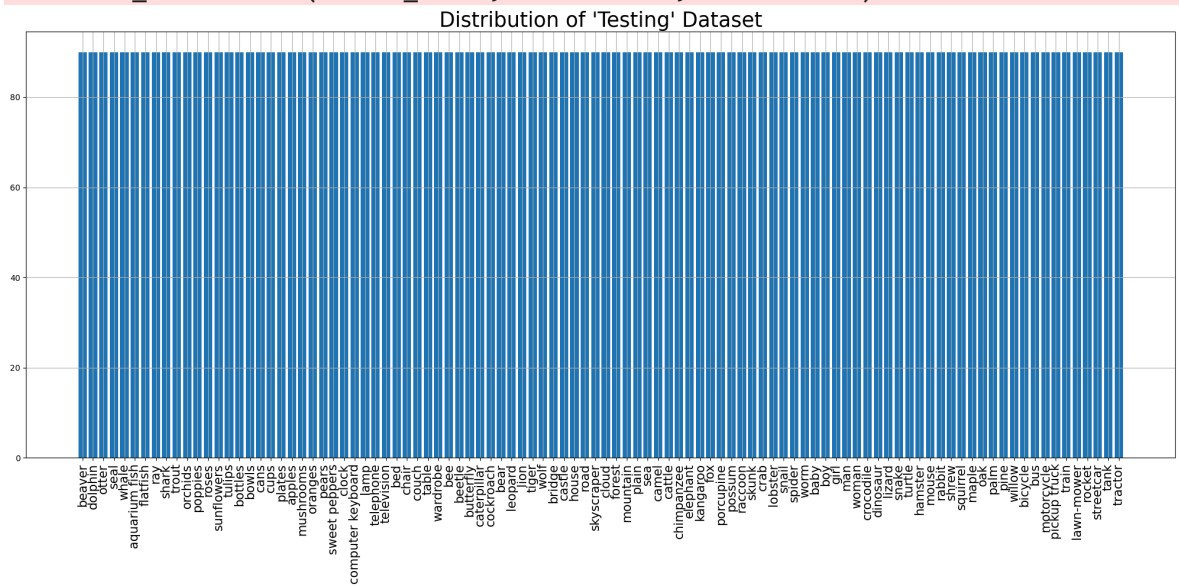
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_3"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None , 2048)
dense_6 (Dense)	(None , 512)
dropout_3 (Dropout)	(None , 512)
dense_7 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— 0s 28ms/step - accuracy: 0.1863 - f1_score: 0.1778 - loss: 3.6512 - precision: 0.5610
Epoch 1: val_loss improved from inf to 1.79046, saving model to best_model_v2.keras as
657/657 ————— 40s 44ms/step - accuracy: 0.1864 - f1_score: 0.1779 - loss: 3.6502 - precision: 0.5611 - val_accuracy: 0.5363 - val_f1_score: 0.5252 - val_loss: 1.7905 - val_precision: 0.8627
Epoch 2/25
656/657 ————— 0s 23ms/step - accuracy: 0.3866 - f1_score: 0.3780 - loss: 2.3105 - precision: 0.7092
Epoch 2: val_loss improved from 1.79046 to 1.68157, saving model to best_model_v2.keras
657/657 ————— 19s 29ms/step - accuracy: 0.3866 - f1_score: 0.3780 - loss: 2.3103 - precision: 0.7092 - val_accuracy: 0.5540 - val_f1_score: 0.5454 - val_loss: 1.6816 - val_precision: 0.8708
Epoch 3/25
655/657 ————— 0s 23ms/step - accuracy: 0.4162 - f1_score: 0.4093 - loss: 2.1767 - precision: 0.7187
Epoch 3: val_loss improved from 1.68157 to 1.64207, saving model to best_model_v2.keras
657/657 ————— 19s 29ms/step - accuracy: 0.4163 - f1_score: 0.4094 - loss: 2.1765 - precision: 0.7187 - val_accuracy: 0.5651 - val_f1_score: 0.5564 - val_loss: 1.6421 - val_precision: 0.8733
Epoch 4/25
656/657 ————— 0s 23ms/step - accuracy: 0.4285 - f1_score: 0.4210 - loss: 2.0923 - precision: 0.7358
Epoch 4: val_loss improved from 1.64207 to 1.61963, saving model to best_model_v2.keras
657/657 ————— 19s 29ms/step - accuracy: 0.4285 - f1_score: 0.4210 - loss: 2.0923 - precision: 0.7357 - val_accuracy: 0.5679 - val_f1_score: 0.5572 - val_loss: 1.6196 - val_precision: 0.8639
Epoch 5/25
656/657 ————— 0s 23ms/step - accuracy: 0.4322 - f1_score: 0.4248 - loss: 2.0897 - precision: 0.7311
Epoch 5: val_loss improved from 1.61963 to 1.59403, saving model to best_model_v2.keras
657/657 ————— 19s 29ms/step - accuracy: 0.4322 - f1_score: 0.4248 - loss: 2.0896 - precision: 0.7311 - val_accuracy: 0.5721 - val_f1_score: 0.5626 - val_loss: 1.5940 - val_precision: 0.8595
Epoch 6/25
655/657 ————— 0s 23ms/step - accuracy: 0.4404 - f1_score: 0.4330 - loss: 2.0678 - precision: 0.7321
Epoch 6: val_loss did not improve from 1.59403
657/657 ————— 18s 28ms/step - accuracy: 0.4404 - f1_score: 0.4330 - loss: 2.0677 - precision: 0.7321 - val_accuracy: 0.5652 - val_f1_score: 0.5546 - val_loss: 1.6155 - val_precision: 0.8564
Epoch 7/25
655/657 ————— 0s 23ms/step - accuracy: 0.4384 - f1_score: 0.4311 - loss: 2.0513 - precision: 0.7424
Epoch 7: val_loss did not improve from 1.59403
657/657 ————— 18s 28ms/step - accuracy: 0.4384 - f1_score: 0.4311 - loss: 2.0512 - precision: 0.7423 - val_accuracy: 0.5650 - val_f1_score: 0.5545 - val_loss: 1.6077 - val_precision: 0.8513
Epoch 8/25
655/657 ————— 0s 23ms/step - accuracy: 0.4466 - f1_score: 0.4397 - loss: 2.0470 - precision: 0.7329
Epoch 8: val_loss did not improve from 1.59403
657/657 ————— 18s 28ms/step - accuracy: 0.4466 - f1_score: 0.4398 - loss: 2.0469 - precision: 0.7330 - val_accuracy: 0.5690 - val_f1_score: 0.5587

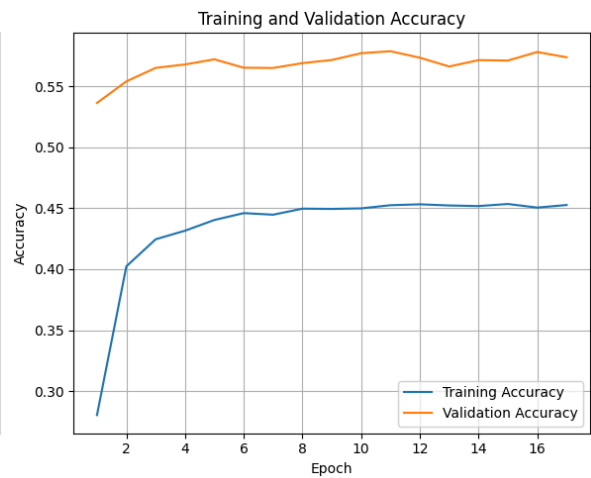
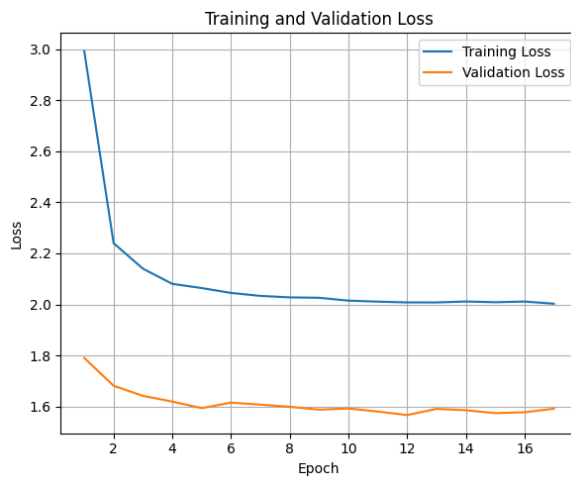
- val_loss: 1.5993 - val_precision: 0.8585
Epoch 9/25
655/657 ————— 0s 23ms/step - accuracy: 0.4428 - f1_score: 0.4366 -
loss: 2.0513 - precision: 0.7218
Epoch 9: val_loss improved from 1.59403 to 1.58793, saving model to best_model_v
2.keras
657/657 ————— 19s 29ms/step - accuracy: 0.4429 - f1_score: 0.4366
- loss: 2.0512 - precision: 0.7218 - val_accuracy: 0.5716 - val_f1_score: 0.5630
- val_loss: 1.5879 - val_precision: 0.8616
Epoch 10/25
657/657 ————— 0s 23ms/step - accuracy: 0.4464 - f1_score: 0.4395 -
loss: 2.0321 - precision: 0.7288
Epoch 10: val_loss did not improve from 1.58793
657/657 ————— 18s 28ms/step - accuracy: 0.4464 - f1_score: 0.4395
- loss: 2.0321 - precision: 0.7288 - val_accuracy: 0.5771 - val_f1_score: 0.5661
- val_loss: 1.5927 - val_precision: 0.8687
Epoch 11/25
657/657 ————— 0s 23ms/step - accuracy: 0.4459 - f1_score: 0.4399 -
loss: 2.0286 - precision: 0.7351
Epoch 11: val_loss improved from 1.58793 to 1.58054, saving model to best_model_v
2.keras
657/657 ————— 19s 29ms/step - accuracy: 0.4459 - f1_score: 0.4400
- loss: 2.0285 - precision: 0.7351 - val_accuracy: 0.5788 - val_f1_score: 0.5711
- val_loss: 1.5805 - val_precision: 0.8505
Epoch 12/25
655/657 ————— 0s 23ms/step - accuracy: 0.4507 - f1_score: 0.4431 -
loss: 2.0186 - precision: 0.7338
Epoch 12: val_loss improved from 1.58054 to 1.56681, saving model to best_model_v
2.keras
657/657 ————— 19s 29ms/step - accuracy: 0.4507 - f1_score: 0.4432
- loss: 2.0185 - precision: 0.7338 - val_accuracy: 0.5734 - val_f1_score: 0.5645
- val_loss: 1.5668 - val_precision: 0.8507
Epoch 13/25
655/657 ————— 0s 23ms/step - accuracy: 0.4511 - f1_score: 0.4446 -
loss: 2.0242 - precision: 0.7407
Epoch 13: val_loss did not improve from 1.56681
657/657 ————— 18s 28ms/step - accuracy: 0.4511 - f1_score: 0.4446
- loss: 2.0241 - precision: 0.7407 - val_accuracy: 0.5662 - val_f1_score: 0.5568
- val_loss: 1.5910 - val_precision: 0.8487
Epoch 14/25
655/657 ————— 0s 23ms/step - accuracy: 0.4514 - f1_score: 0.4448 -
loss: 2.0280 - precision: 0.7283
Epoch 14: val_loss did not improve from 1.56681
657/657 ————— 18s 28ms/step - accuracy: 0.4514 - f1_score: 0.4448
- loss: 2.0279 - precision: 0.7283 - val_accuracy: 0.5714 - val_f1_score: 0.5622
- val_loss: 1.5858 - val_precision: 0.8605
Epoch 15/25
657/657 ————— 0s 23ms/step - accuracy: 0.4500 - f1_score: 0.4436 -
loss: 2.0243 - precision: 0.7332
Epoch 15: val_loss did not improve from 1.56681
657/657 ————— 18s 28ms/step - accuracy: 0.4500 - f1_score: 0.4436
- loss: 2.0242 - precision: 0.7332 - val_accuracy: 0.5711 - val_f1_score: 0.5603
- val_loss: 1.5744 - val_precision: 0.8510
Epoch 16/25
655/657 ————— 0s 23ms/step - accuracy: 0.4454 - f1_score: 0.4381 -
loss: 2.0278 - precision: 0.7376
Epoch 16: val_loss did not improve from 1.56681
657/657 ————— 18s 28ms/step - accuracy: 0.4454 - f1_score: 0.4381
- loss: 2.0277 - precision: 0.7376 - val_accuracy: 0.5781 - val_f1_score: 0.5702
- val_loss: 1.5778 - val_precision: 0.8601

Epoch 17/25

657/657 ————— **0s** 23ms/step - accuracy: 0.4488 - f1_score: 0.4426 -
loss: 2.0237 - precision: 0.7324

Epoch 17: val_loss did not improve from 1.56681

657/657 ————— **18s** 28ms/step - accuracy: 0.4488 - f1_score: 0.4426
- loss: 2.0237 - precision: 0.7324 - val_accuracy: 0.5738 - val_f1_score: 0.5645
- val_loss: 1.5918 - val_precision: 0.8553



141/141 ————— **11s** 37ms/step - accuracy: 0.5632 - f1_score: 0.5498
- loss: 1.5820 - precision: 0.8470

Test Accuracy: 57.32%

Test Loss: 1.5581715106964111

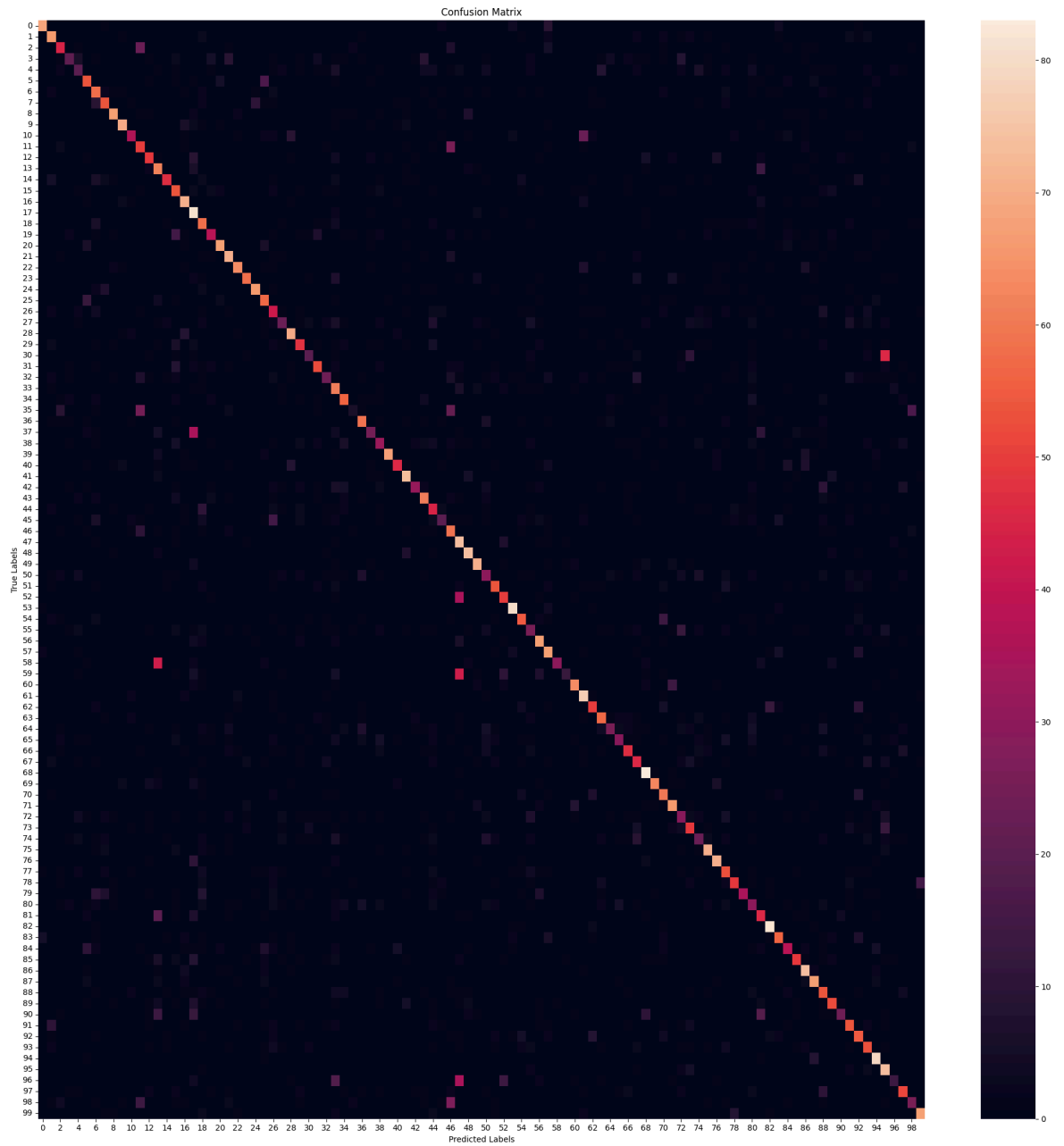
Test Precision: 85.07%

Test F1 Scores (Per Class): [82.142845 67.34693 53.254433 33.599995 27.5862 5
5.958546 52.25224

61.714287 77.456635 80.459755 50.704224 44.247784 59.756096 46.09665
63.51351 49.090904 70.99999 51.75718 49.785408 49.673195 77.906975
70.64676 76.646706 72.04968 76.74419 56.43564 40.38461 32.85714
74.8663 55.813946 31.746029 61.90476 34.53237 44.202892 61.538452
11.42857 64.83515 38.167934 40.49079 85. 61.33333 73.73737
48.48484 65.24063 44.999996 31.249996 45.038162 47.28434 75.89743
76.756744 33.898304 64.670654 56.179775 85.10638 60.773476 31.70731
67.33668 64.423065 49.586773 22.857141 74.8538 69.99999 59.523804
62.983418 38.80597 42.335762 58.749992 47.66839 78.30189 79.24527
59.113293 70.21276 33.33333 54.143642 30.263153 75.53191 76.756744
54.081623 61.63522 53.030293 31.914886 45.098038 82.828285 65.14285
47.204964 64.47368 70.14217 72.82608 49.76958 61.176468 39.669422
55.958546 54.187183 54.639168 79.79797 56.488544 21.138208 59.30232
36.619713 74.86033]

Average Test F1 Scores:56.46

282/282 ————— **11s** 27ms/step



	precision	recall	f1-score	support
beaver	0.88	0.77	0.82	90
dolphin	0.62	0.73	0.67	90
otter	0.57	0.50	0.53	90
seal	0.60	0.23	0.34	90
whale	0.36	0.22	0.28	90
aquarium fish	0.52	0.60	0.56	90
flatfish	0.44	0.64	0.52	90
ray	0.64	0.60	0.62	90
shark	0.81	0.74	0.77	90
trout	0.83	0.78	0.80	90
orchids	0.69	0.40	0.51	90
poppies	0.37	0.56	0.44	90
roses	0.66	0.54	0.60	90
sunflowers	0.35	0.69	0.46	90
tulips	0.81	0.52	0.64	90
bottles	0.42	0.60	0.49	90
bowls	0.65	0.79	0.71	90
cans	0.36	0.90	0.52	90
cups	0.41	0.64	0.50	90
plates	0.60	0.42	0.50	90
apples	0.82	0.74	0.78	90
mushrooms	0.64	0.79	0.71	90
oranges	0.83	0.71	0.77	90
pears	0.82	0.64	0.72	90
sweet peppers	0.80	0.73	0.77	90
clock	0.51	0.63	0.56	90
computer keyboard	0.36	0.47	0.40	90
lamp	0.46	0.26	0.33	90
telephone	0.72	0.78	0.75	90
television	0.59	0.53	0.56	90
bed	0.56	0.22	0.32	90
chair	0.67	0.58	0.62	90
couch	0.49	0.27	0.35	90
table	0.33	0.68	0.44	90
wardrobe	0.61	0.62	0.62	90
bee	0.40	0.07	0.11	90
beetle	0.64	0.66	0.65	90
butterfly	0.61	0.28	0.38	90
caterpillar	0.45	0.37	0.40	90
cockroach	0.97	0.76	0.85	90
bear	0.77	0.51	0.61	90
leopard	0.68	0.81	0.74	90
lion	0.78	0.36	0.49	90
tiger	0.63	0.68	0.65	90
wolf	0.41	0.50	0.45	90
bridge	0.53	0.22	0.31	90
castle	0.34	0.66	0.45	90
house	0.33	0.82	0.47	90
road	0.70	0.82	0.76	90
skyscraper	0.75	0.79	0.77	90
cloud	0.34	0.33	0.34	90
forest	0.70	0.60	0.65	90
mountain	0.57	0.56	0.56	90
plain	0.82	0.89	0.85	90
sea	0.60	0.61	0.61	90
camel	0.35	0.29	0.32	90
cattle	0.61	0.74	0.67	90
chimpanzee	0.57	0.74	0.64	90

elephant	0.97	0.33	0.50	90
kangaroo	0.80	0.13	0.23	90
fox	0.79	0.71	0.75	90
porcupine	0.59	0.86	0.70	90
possum	0.64	0.56	0.60	90
raccoon	0.63	0.63	0.63	90
skunk	0.59	0.29	0.39	90
crab	0.62	0.32	0.42	90
lobster	0.67	0.52	0.59	90
snail	0.45	0.51	0.48	90
spider	0.68	0.92	0.78	90
worm	0.91	0.70	0.79	90
baby	0.53	0.67	0.59	90
boy	0.67	0.73	0.70	90
girl	0.35	0.32	0.33	90
man	0.54	0.54	0.54	90
woman	0.37	0.26	0.30	90
crocodile	0.72	0.79	0.76	90
dinosaur	0.75	0.79	0.77	90
lizard	0.50	0.59	0.54	90
snake	0.71	0.54	0.62	90
turtle	0.83	0.39	0.53	90
hamster	0.30	0.33	0.32	90
mouse	0.40	0.51	0.45	90
rabbit	0.76	0.91	0.83	90
shrew	0.67	0.62	0.64	90
squirrel	0.54	0.42	0.47	90
maple	0.79	0.54	0.64	90
oak	0.61	0.82	0.70	90
palm	0.71	0.74	0.73	90
pine	0.43	0.60	0.50	90
willow	0.65	0.58	0.61	90
bicycle	0.77	0.27	0.40	90
bus	0.52	0.60	0.56	90
motorcycle	0.48	0.61	0.54	90
pickup truck	0.51	0.59	0.55	90
train	0.73	0.88	0.80	90
lawn-mower	0.43	0.82	0.56	90
rocket	0.39	0.14	0.21	90
streetcar	0.62	0.57	0.59	90
tank	0.50	0.29	0.37	90
tractor	0.75	0.74	0.75	90
accuracy			0.57	9000
macro avg	0.60	0.57	0.56	9000
weighted avg	0.60	0.57	0.56	9000

Model: "sequential_3"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None , 2048)
dense_6 (Dense)	(None , 512)
dropout_3 (Dropout)	(None , 512)
dense_7 (Dense)	(None , 100)



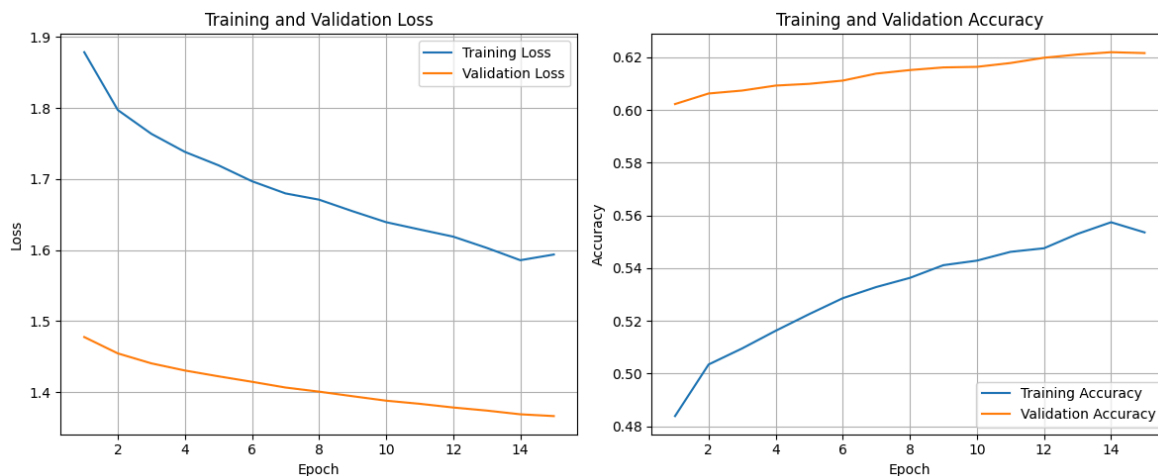
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1312/1314 ————— 0s 26ms/step - accuracy: 0.4712 - f1_score: 0.4633
- loss: 1.9297 - precision: 0.7421
Epoch 1: val_loss improved from inf to 1.47768, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 56s 34ms/step - accuracy: 0.4713 - f1_score: 0.4633 - loss: 1.9296 - precision: 0.7421 - val_accuracy: 0.6022 - val_f1_score: 0.5961 - val_loss: 1.4777 - val_precision: 0.8655
Epoch 2/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.4982 - f1_score: 0.4919 - loss: 1.8191 - precision: 0.7647
Epoch 2: val_loss improved from 1.47768 to 1.45467, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.4982 - f1_score: 0.4920 - loss: 1.8190 - precision: 0.7647 - val_accuracy: 0.6062 - val_f1_score: 0.6011 - val_loss: 1.4547 - val_precision: 0.8614
Epoch 3/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.5035 - f1_score: 0.4975 - loss: 1.7843 - precision: 0.7700
Epoch 3: val_loss improved from 1.45467 to 1.44066, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5035 - f1_score: 0.4975 - loss: 1.7842 - precision: 0.7700 - val_accuracy: 0.6073 - val_f1_score: 0.6029 - val_loss: 1.4407 - val_precision: 0.8583
Epoch 4/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.5150 - f1_score: 0.5092 - loss: 1.7526 - precision: 0.7791
Epoch 4: val_loss improved from 1.44066 to 1.43061, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5150 - f1_score: 0.5092 - loss: 1.7525 - precision: 0.7791 - val_accuracy: 0.6092 - val_f1_score: 0.6051 - val_loss: 1.4306 - val_precision: 0.8593
Epoch 5/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.5183 - f1_score: 0.5129 - loss: 1.7333 - precision: 0.7783
Epoch 5: val_loss improved from 1.43061 to 1.42244, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5183 - f1_score: 0.5129 - loss: 1.7333 - precision: 0.7783 - val_accuracy: 0.6099 - val_f1_score: 0.6059 - val_loss: 1.4224 - val_precision: 0.8570
Epoch 6/15
1312/1314 ————— 0s 23ms/step - accuracy: 0.5265 - f1_score: 0.5211 - loss: 1.7092 - precision: 0.7857
Epoch 6: val_loss improved from 1.42244 to 1.41473, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5265 - f1_score: 0.5211 - loss: 1.7091 - precision: 0.7857 - val_accuracy: 0.6111 - val_f1_score: 0.6072 - val_loss: 1.4147 - val_precision: 0.8586
Epoch 7/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.5290 - f1_score: 0.5238 - loss: 1.6938 - precision: 0.7877
Epoch 7: val_loss improved from 1.41473 to 1.40666, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5290 - f1_score: 0.5239 - loss: 1.6938 - precision: 0.7877 - val_accuracy: 0.6138 - val_f1_score: 0.6096 - val_loss: 1.4067 - val_precision: 0.8557
Epoch 8/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.5349 - f1_score: 0.5294 - loss: 1.6788 - precision: 0.7896
Epoch 8: val_loss improved from 1.40666 to 1.40086, saving model to best_model_fine_tune_v2.keras

```
ne_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5349 - f1_score: 0.529
4 - loss: 1.6788 - precision: 0.7896 - val_accuracy: 0.6151 - val_f1_score: 0.611
3 - val_loss: 1.4009 - val_precision: 0.8576
Epoch 9/15
1313/1314 ————— 0s 23ms/step - accuracy: 0.5384 - f1_score: 0.5335
- loss: 1.6658 - precision: 0.7923
Epoch 9: val_loss improved from 1.40086 to 1.39442, saving model to best_model_fi
ne_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5384 - f1_score: 0.533
5 - loss: 1.6658 - precision: 0.7923 - val_accuracy: 0.6161 - val_f1_score: 0.612
4 - val_loss: 1.3944 - val_precision: 0.8549
Epoch 10/15
1312/1314 ————— 0s 23ms/step - accuracy: 0.5399 - f1_score: 0.5347
- loss: 1.6487 - precision: 0.7957
Epoch 10: val_loss improved from 1.39442 to 1.38814, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5399 - f1_score: 0.534
7 - loss: 1.6487 - precision: 0.7957 - val_accuracy: 0.6163 - val_f1_score: 0.612
4 - val_loss: 1.3881 - val_precision: 0.8542
Epoch 11/15
1312/1314 ————— 0s 23ms/step - accuracy: 0.5473 - f1_score: 0.5419
- loss: 1.6295 - precision: 0.8010
Epoch 11: val_loss improved from 1.38814 to 1.38375, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.5473 - f1_score: 0.541
9 - loss: 1.6295 - precision: 0.8010 - val_accuracy: 0.6178 - val_f1_score: 0.613
9 - val_loss: 1.3838 - val_precision: 0.8544
Epoch 12/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.5463 - f1_score: 0.5413
- loss: 1.6260 - precision: 0.8033
Epoch 12: val_loss improved from 1.38375 to 1.37855, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.5463 - f1_score: 0.541
3 - loss: 1.6260 - precision: 0.8033 - val_accuracy: 0.6198 - val_f1_score: 0.616
3 - val_loss: 1.3786 - val_precision: 0.8524
Epoch 13/15
1312/1314 ————— 0s 23ms/step - accuracy: 0.5543 - f1_score: 0.5492
- loss: 1.6051 - precision: 0.8026
Epoch 13: val_loss improved from 1.37855 to 1.37422, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.5543 - f1_score: 0.549
2 - loss: 1.6051 - precision: 0.8026 - val_accuracy: 0.6210 - val_f1_score: 0.617
3 - val_loss: 1.3742 - val_precision: 0.8530
Epoch 14/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.5561 - f1_score: 0.5510
- loss: 1.5928 - precision: 0.8057
Epoch 14: val_loss improved from 1.37422 to 1.36899, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.5561 - f1_score: 0.551
0 - loss: 1.5928 - precision: 0.8057 - val_accuracy: 0.6219 - val_f1_score: 0.618
4 - val_loss: 1.3690 - val_precision: 0.8507
Epoch 15/15
1314/1314 ————— 0s 23ms/step - accuracy: 0.5536 - f1_score: 0.5490
- loss: 1.5928 - precision: 0.8005
Epoch 15: val_loss improved from 1.36899 to 1.36648, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.5536 - f1_score: 0.549
0 - loss: 1.5928 - precision: 0.8005 - val_accuracy: 0.6216 - val_f1_score: 0.617
9 - val_loss: 1.3665 - val_precision: 0.8532
```



141/141 ————— **11s 40ms/step** - accuracy: 0.6077 - f1_score: 0.5990

- loss: 1.3772 - precision: 0.8401

Test Accuracy: 62.27%

Test Loss: 1.3533506393432617

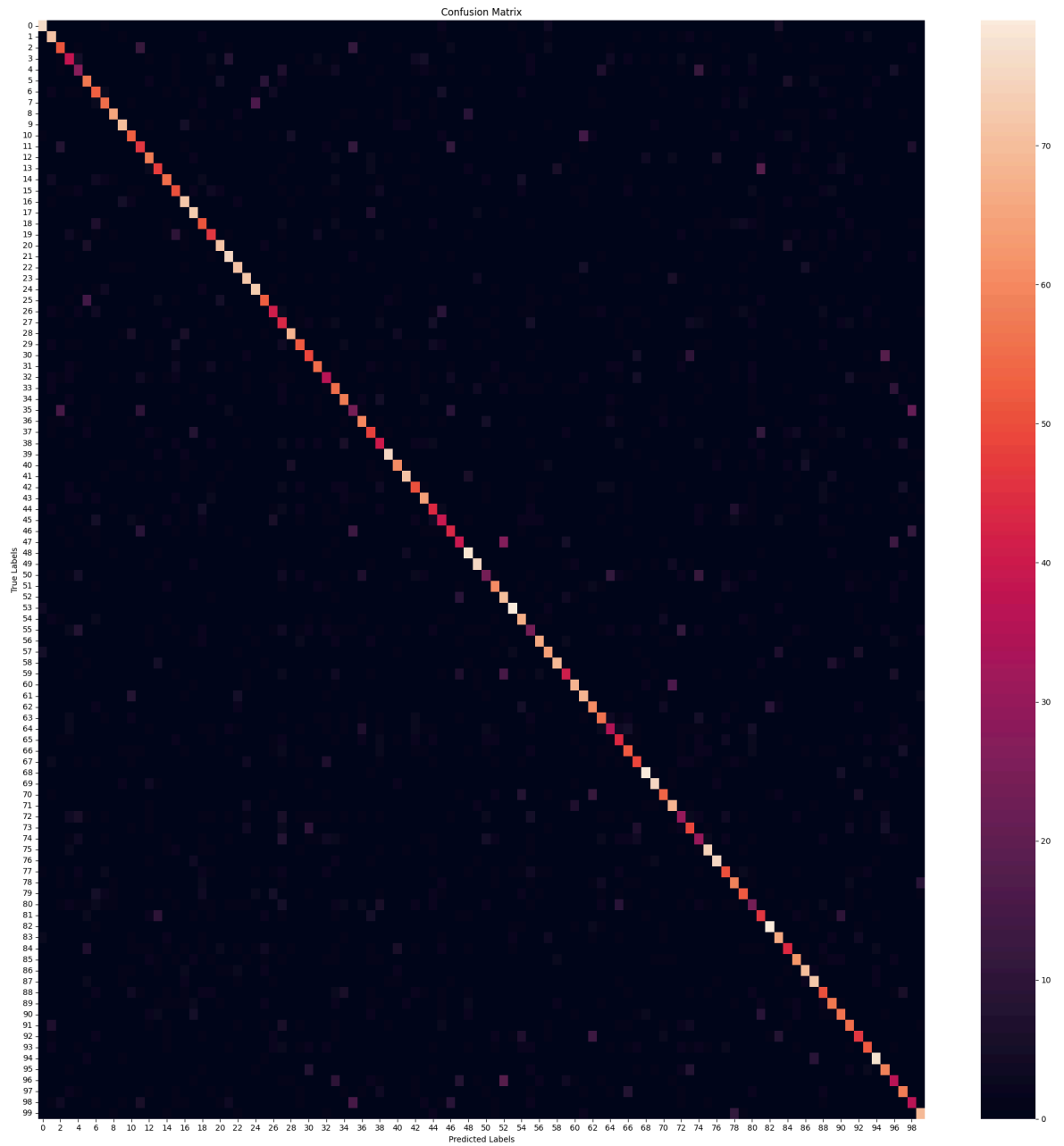
Test Precision: 85.40%

Test F1 Scores (Per Class): [84.44444 71.71716 56.66666 43.181812 32.335327 5
9.685856 54.35897

66.26506	77.8443	79.54544	59.21787	50.	63.387978	56.28742
65.47618	54.64481	73.846146	73.36682	54.838707	54.117645	78.453026
76.923065	76.756744	77.41935	77.65957	62.352936	49.382713	37.391296
78.16091	61.53846	57.64705	66.66666	44.72049	51.886784	63.73626
28.235287	66.29834	53.93258	43.010746	87.71929	68.15642	80.681816
56.818176	71.91011	49.438198	45.882347	49.142857	49.056595	76.84728
81.08108	34.0136	69.71428	62.780266	87.29281	65.686264	29.629627
76.300575	70.329666	80.701744	53.24675	76.83616	71.57894	63.87434
61.202175	41.212116	52.380943	57.45856	55.999992	84.946236	81.52173
63.157898	70.46631	36.58536	55.05617	32.78688	79.14438	81.96721
57.471264	63.440853	60.465115	28.571424	47.66839	87.29281	68.39378
51.764698	69.27374	74.074066	76.190475	54.945045	61.62162	60.869556
60.773476	55.75757	59.428566	82.7957	62.433857	40.437157	62.702698
41.379307	76.24309					

Average Test F1 Scores:62.00

282/282 ————— **11s 28ms/step**



	precision	recall	f1-score	support
beaver	0.84	0.84	0.84	90
dolphin	0.66	0.79	0.72	90
otter	0.57	0.57	0.57	90
seal	0.44	0.42	0.43	90
whale	0.35	0.30	0.32	90
aquarium fish	0.56	0.63	0.60	90
flatfish	0.50	0.59	0.54	90
ray	0.72	0.61	0.66	90
shark	0.84	0.72	0.78	90
trout	0.81	0.78	0.80	90
orchids	0.60	0.59	0.59	90
poppies	0.49	0.51	0.50	90
roses	0.62	0.64	0.63	90
sunflowers	0.61	0.52	0.56	90
tulips	0.71	0.61	0.65	90
bottles	0.54	0.56	0.55	90
bowls	0.69	0.80	0.74	90
cans	0.67	0.81	0.73	90
cups	0.53	0.57	0.55	90
plates	0.57	0.51	0.54	90
apples	0.78	0.79	0.78	90
mushrooms	0.71	0.83	0.77	90
oranges	0.75	0.79	0.77	90
pears	0.75	0.80	0.77	90
sweet peppers	0.74	0.81	0.78	90
clock	0.66	0.59	0.62	90
computer keyboard	0.56	0.46	0.50	90
lamp	0.31	0.48	0.38	90
telephone	0.81	0.76	0.78	90
television	0.66	0.58	0.62	90
bed	0.60	0.54	0.57	90
chair	0.73	0.61	0.67	90
couch	0.51	0.40	0.45	90
table	0.45	0.61	0.52	90
wardrobe	0.63	0.64	0.64	90
bee	0.30	0.27	0.28	90
beetle	0.66	0.67	0.66	90
butterfly	0.55	0.53	0.54	90
caterpillar	0.42	0.44	0.43	90
cockroach	0.93	0.83	0.88	90
bear	0.69	0.68	0.68	90
leopard	0.83	0.79	0.81	90
lion	0.58	0.56	0.57	90
tiger	0.73	0.71	0.72	90
wolf	0.50	0.49	0.49	90
bridge	0.49	0.43	0.46	90
castle	0.51	0.48	0.49	90
house	0.57	0.43	0.49	90
road	0.69	0.87	0.77	90
skyscraper	0.79	0.83	0.81	90
cloud	0.43	0.27	0.33	90
forest	0.72	0.68	0.70	90
mountain	0.53	0.78	0.63	90
plain	0.87	0.88	0.87	90
sea	0.58	0.74	0.65	90
camel	0.33	0.27	0.30	90
cattle	0.80	0.73	0.76	90
chimpanzee	0.70	0.71	0.70	90

elephant	0.85	0.77	0.81	90
kangaroo	0.64	0.46	0.53	90
fox	0.78	0.76	0.77	90
porcupine	0.68	0.76	0.72	90
possum	0.60	0.68	0.64	90
raccoon	0.60	0.62	0.61	90
skunk	0.45	0.38	0.41	90
crab	0.56	0.49	0.52	90
lobster	0.57	0.58	0.57	90
snail	0.58	0.54	0.56	90
spider	0.82	0.88	0.85	90
worm	0.80	0.83	0.82	90
baby	0.67	0.60	0.63	90
boy	0.66	0.76	0.70	90
girl	0.41	0.33	0.37	90
man	0.56	0.54	0.55	90
woman	0.32	0.33	0.33	90
crocodile	0.76	0.82	0.79	90
dinosaur	0.81	0.83	0.82	90
lizard	0.60	0.56	0.57	90
snake	0.61	0.66	0.63	90
turtle	0.63	0.58	0.60	90
hamster	0.31	0.27	0.29	90
mouse	0.45	0.51	0.48	90
rabbit	0.87	0.88	0.87	90
shrew	0.64	0.73	0.68	90
squirrel	0.55	0.49	0.52	90
maple	0.70	0.69	0.69	90
oak	0.71	0.78	0.74	90
palm	0.73	0.80	0.76	90
pine	0.54	0.56	0.55	90
willow	0.60	0.63	0.62	90
bicycle	0.60	0.62	0.61	90
bus	0.60	0.61	0.61	90
motorcycle	0.62	0.51	0.56	90
pickup truck	0.61	0.58	0.59	90
train	0.80	0.86	0.83	90
lawn-mower	0.60	0.66	0.63	90
rocket	0.40	0.41	0.40	90
streetcar	0.61	0.64	0.63	90
tank	0.43	0.40	0.41	90
tractor	0.76	0.77	0.76	90
accuracy			0.62	9000
macro avg	0.62	0.62	0.62	9000
weighted avg	0.62	0.62	0.62	9000

Dropout(0.8), weight_decay(0.9), l2(0.9), train_augmentation - over-regularising

```
In [7]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
```

```

X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
print(f"X_temp.shape: {X_temp.shape}\n")

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
                'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
                'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
                'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
                'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
                'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
                'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
                'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
                'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
                'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
                'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))

```

```

        .map(augment_dataset, num_parallel_calls=tf.data.experimental
        .batch(batch_size)
        .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(0.9) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.8), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):

```

```

print(type(sample)) # Should be <class 'tuple'>
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'tensor'
print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=0.9
        loss='categorical_crossentropy',
        metrics=['accuracy', 'precision', 'f1_score']))

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                                patience=5, # Num. epochs with no improvement
                                restore_best_weights=True)
#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                                factor=0.1, # Reduce lr by a factor
#                                patience=3, # Num epochs w/ no improvement
#                                min_lr=1e-6, # Min lr
#                                verbose=1)
#tensorboard = TensorBoard(log_dir='./logs', # Logs directory
#                            histogram_freq=1, # Logs histograms for weights/acc
#                            write_graph=True, # Logs graph of model
#                            write_images=True) # Log images like weight histograms
checkpoint = ModelCheckpoint('best_model_v3.keras', # Path where the best model is saved
                             monitor='val_loss', # What to monitor (validation loss)
                             save_best_only=True, # Only save model when val loss is best
                             mode='min', # Looking to minimize the validation loss
                             verbose=1) # Print out message when saving model
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v3.keras', # Path where the best model is saved
                                       monitor='val_loss', # What to monitor (validation loss)
                                       save_best_only=True, # Only save model when val loss is best
                                       mode='min', # Looking to minimize the validation loss
                                       verbose=1) # Print out message when saving model
#csv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                    batch_size=batch_size, callbacks=[early_stopping, checkpoint])

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']

```

```

val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v3.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<Fine-Tune>>>>>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

```

```

model.summary()

#### <<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                           .batch(batch_size)
                           .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v3.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))

```



```
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

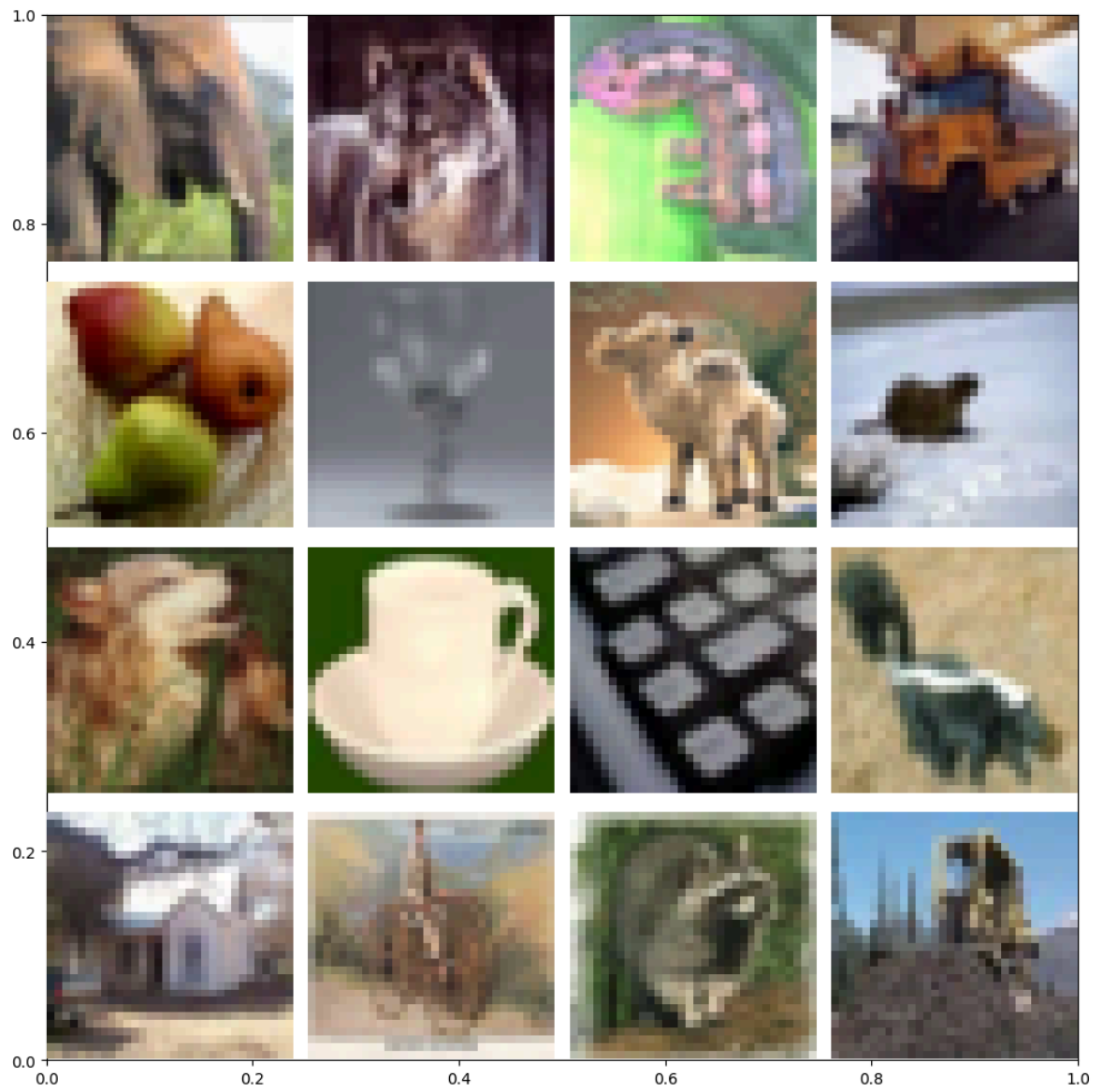
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

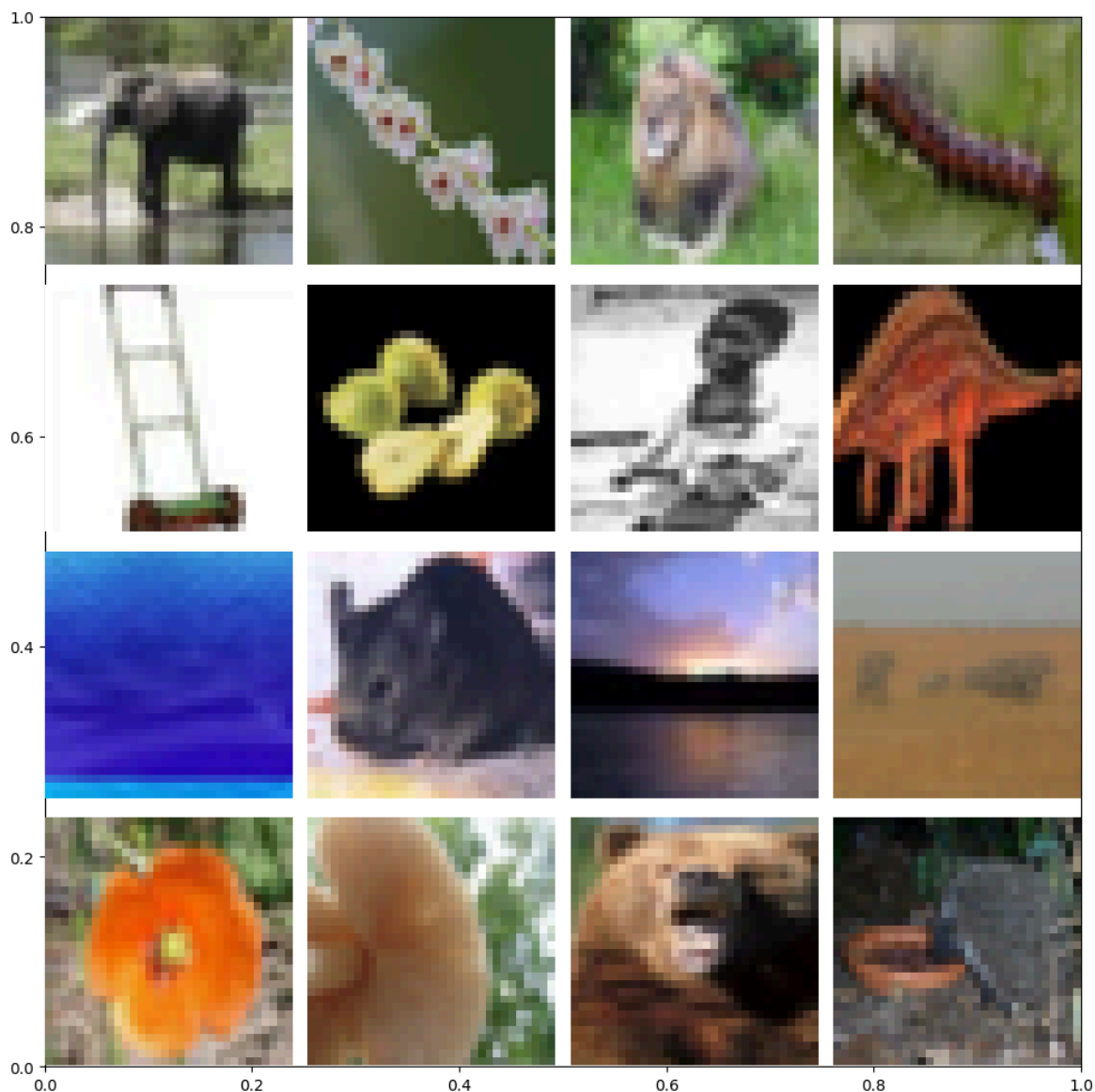
y_test.shape: (9000, 1)

labels[8252].shape: (1,)

imgs[8252].shape: (32, 32, 3)

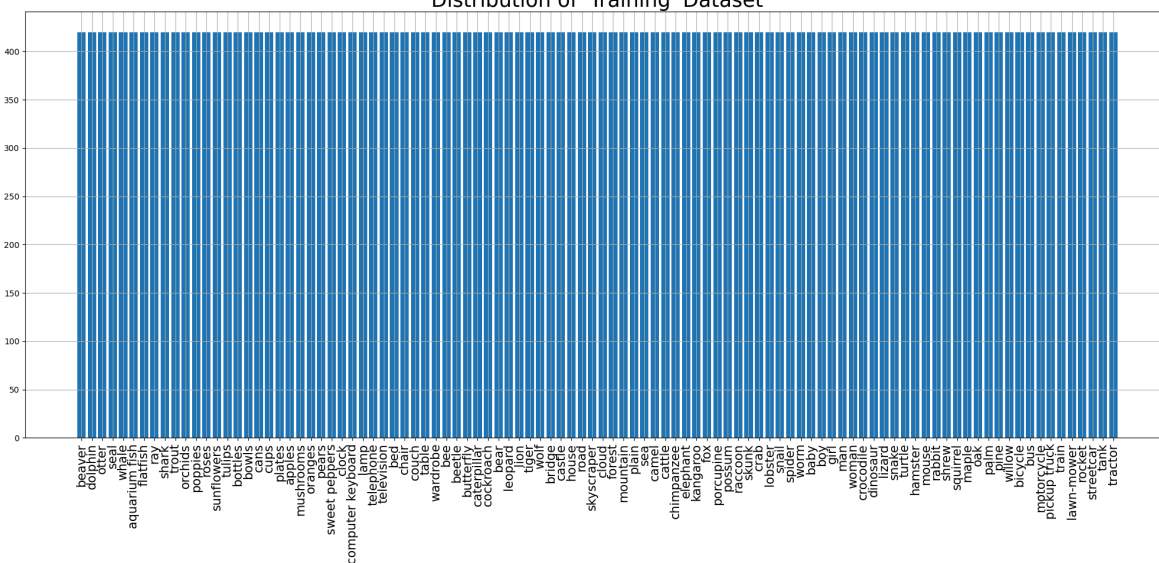


```
labels[6553].shape: (1,)  
imgs[6553].shape: (32, 32, 3)
```

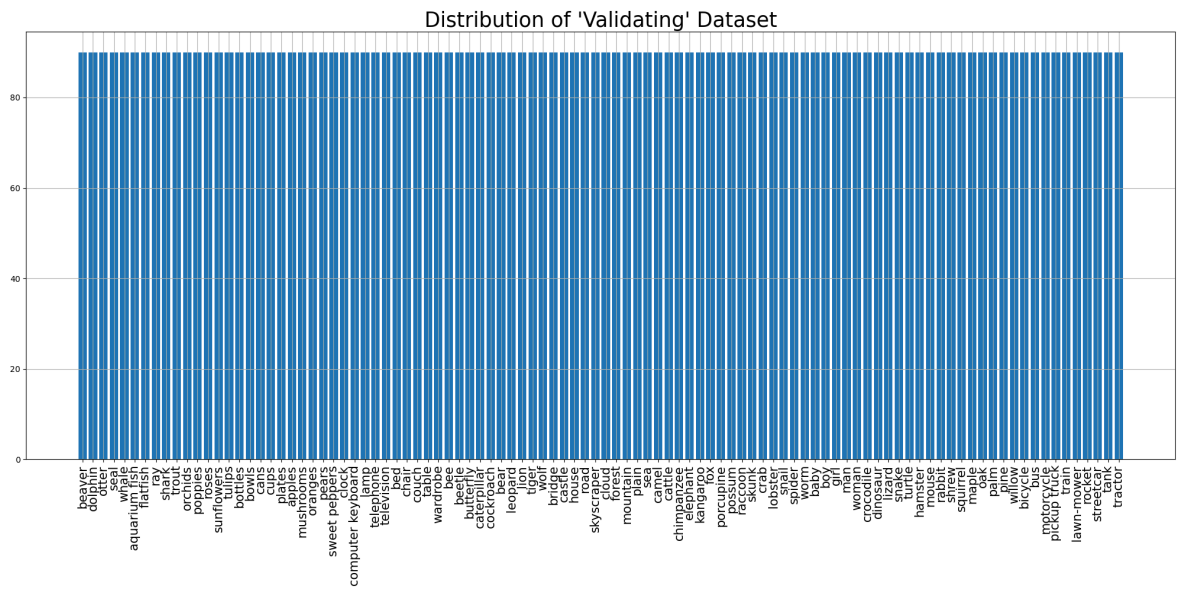


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

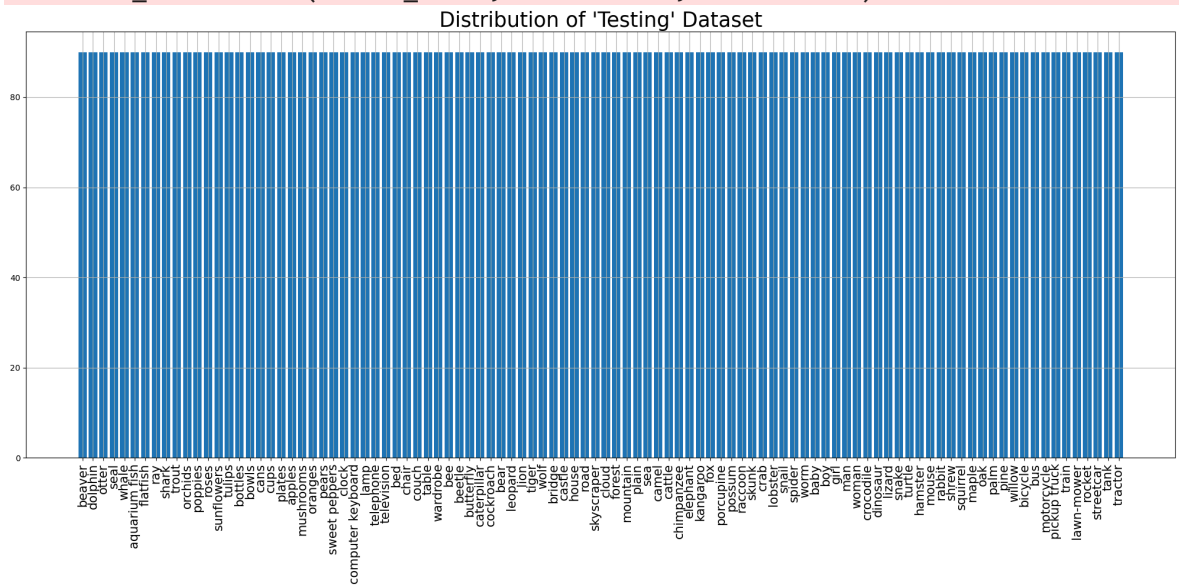
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_ConcatenateDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

```
<class 'tuple'>
```

2

```
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
```

(64, 224, 224, 3)

(64, 100)

Model input shape: (None, 224, 224, 3)

Model output shape: (None, 100)

2

```
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
```

(64, 224, 224, 3) (64, 100)

Model: "sequential_4"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_4 (GlobalAveragePooling2D)	(None , 2048)
dense_8 (Dense)	(None , 512)
dropout_4 (Dropout)	(None , 512)
dense_9 (Dense)	(None , 100)

◀  ▶

Total params: [24,665,188](#) (94.09 MB)

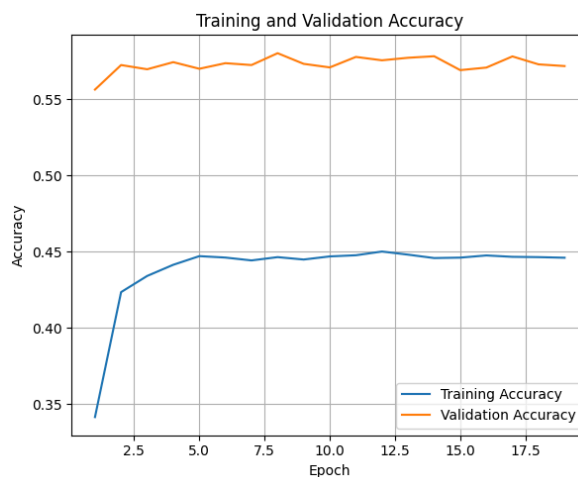
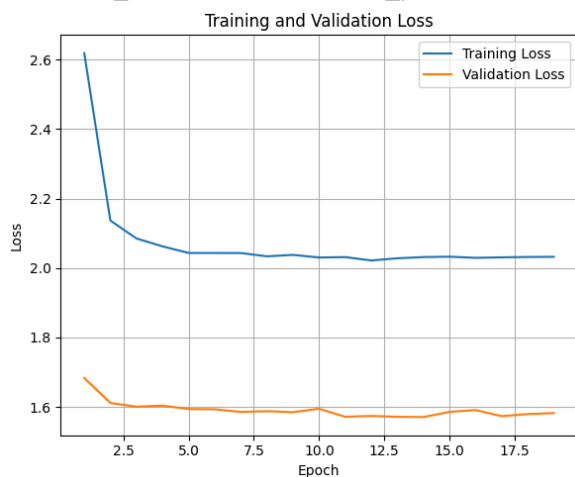
Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
1312/1314 ————— 0s 26ms/step - accuracy: 0.2546 - f1_score: 0.2476
- loss: 3.2023 - precision: 0.6290
Epoch 1: val_loss improved from inf to 1.68343, saving model to best_model_v3.keras
1314/1314 ————— 55s 34ms/step - accuracy: 0.2548 - f1_score: 0.2478
8 - loss: 3.2009 - precision: 0.6292 - val_accuracy: 0.5563 - val_f1_score: 0.5467
7 - val_loss: 1.6834 - val_precision: 0.8685
Epoch 2/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.4215 - f1_score: 0.4149
- loss: 2.1536 - precision: 0.7285
Epoch 2: val_loss improved from 1.68343 to 1.61151, saving model to best_model_v3.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.4215 - f1_score: 0.4149
9 - loss: 2.1535 - precision: 0.7285 - val_accuracy: 0.5724 - val_f1_score: 0.5632
2 - val_loss: 1.6115 - val_precision: 0.8717
Epoch 3/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.4300 - f1_score: 0.4242
- loss: 2.0942 - precision: 0.7291
Epoch 3: val_loss improved from 1.61151 to 1.60088, saving model to best_model_v3.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.4300 - f1_score: 0.4242
2 - loss: 2.0942 - precision: 0.7291 - val_accuracy: 0.5697 - val_f1_score: 0.5611
1 - val_loss: 1.6009 - val_precision: 0.8560
Epoch 4/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.4422 - f1_score: 0.4358
- loss: 2.0644 - precision: 0.7343
Epoch 4: val_loss did not improve from 1.60088
1314/1314 ————— 34s 26ms/step - accuracy: 0.4422 - f1_score: 0.4358
8 - loss: 2.0644 - precision: 0.7343 - val_accuracy: 0.5743 - val_f1_score: 0.5648
8 - val_loss: 1.6039 - val_precision: 0.8590
Epoch 5/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.4462 - f1_score: 0.4396
- loss: 2.0460 - precision: 0.7397
Epoch 5: val_loss improved from 1.60088 to 1.59399, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.4462 - f1_score: 0.4396
6 - loss: 2.0460 - precision: 0.7397 - val_accuracy: 0.5700 - val_f1_score: 0.5594
4 - val_loss: 1.5940 - val_precision: 0.8577
Epoch 6/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.4468 - f1_score: 0.4411
- loss: 2.0399 - precision: 0.7358
Epoch 6: val_loss improved from 1.59399 to 1.59333, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.4468 - f1_score: 0.4411
1 - loss: 2.0399 - precision: 0.7358 - val_accuracy: 0.5737 - val_f1_score: 0.5678
8 - val_loss: 1.5933 - val_precision: 0.8497
Epoch 7/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.4432 - f1_score: 0.4378
- loss: 2.0453 - precision: 0.7305
Epoch 7: val_loss improved from 1.59333 to 1.58593, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.4432 - f1_score: 0.4378
8 - loss: 2.0453 - precision: 0.7305 - val_accuracy: 0.5724 - val_f1_score: 0.5621
1 - val_loss: 1.5859 - val_precision: 0.8564
Epoch 8/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.4437 - f1_score: 0.4376
- loss: 2.0377 - precision: 0.7333
Epoch 8: val_loss did not improve from 1.58593
1314/1314 ————— 34s 26ms/step - accuracy: 0.4437 - f1_score: 0.437

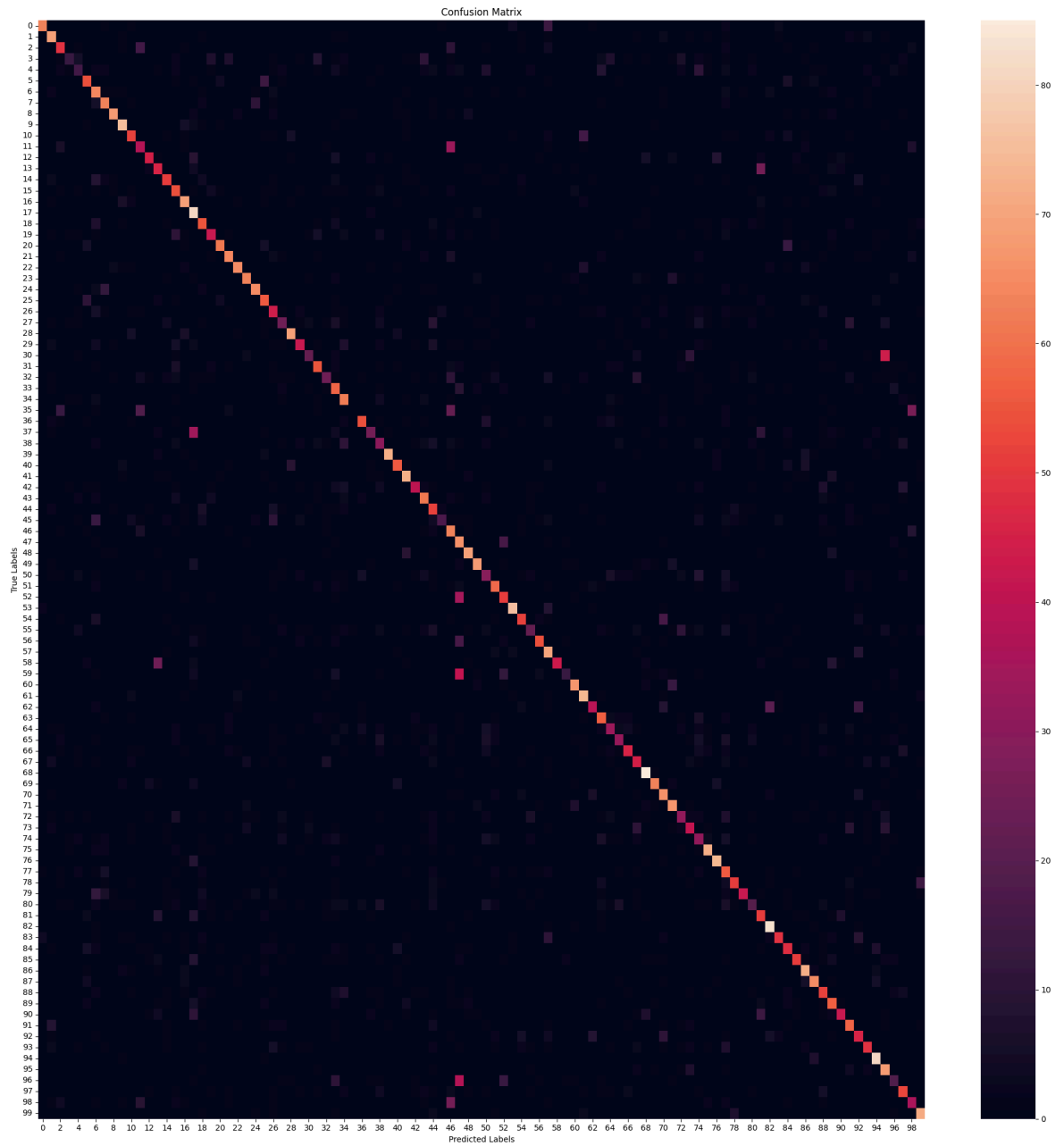
6 - loss: 2.0377 - precision: 0.7333 - val_accuracy: 0.5802 - val_f1_score: 0.569
7 - val_loss: 1.5880 - val_precision: 0.8656
Epoch 9/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.4443 - f1_score: 0.4379
- loss: 2.0378 - precision: 0.7336
Epoch 9: val_loss improved from 1.58593 to 1.58499, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.4443 - f1_score: 0.437
9 - loss: 2.0378 - precision: 0.7336 - val_accuracy: 0.5732 - val_f1_score: 0.562
5 - val_loss: 1.5850 - val_precision: 0.8503
Epoch 10/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.4470 - f1_score: 0.4404
- loss: 2.0306 - precision: 0.7351
Epoch 10: val_loss did not improve from 1.58499
1314/1314 ————— 34s 26ms/step - accuracy: 0.4470 - f1_score: 0.440
4 - loss: 2.0306 - precision: 0.7351 - val_accuracy: 0.5709 - val_f1_score: 0.559
6 - val_loss: 1.5949 - val_precision: 0.8521
Epoch 11/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.4481 - f1_score: 0.4412
- loss: 2.0288 - precision: 0.7373
Epoch 11: val_loss improved from 1.58499 to 1.57185, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.4481 - f1_score: 0.441
2 - loss: 2.0288 - precision: 0.7373 - val_accuracy: 0.5778 - val_f1_score: 0.568
4 - val_loss: 1.5718 - val_precision: 0.8585
Epoch 12/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.4515 - f1_score: 0.4451
- loss: 2.0224 - precision: 0.7371
Epoch 12: val_loss did not improve from 1.57185
1314/1314 ————— 34s 26ms/step - accuracy: 0.4515 - f1_score: 0.445
1 - loss: 2.0224 - precision: 0.7371 - val_accuracy: 0.5756 - val_f1_score: 0.566
5 - val_loss: 1.5743 - val_precision: 0.8488
Epoch 13/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.4469 - f1_score: 0.4405
- loss: 2.0272 - precision: 0.7333
Epoch 13: val_loss improved from 1.57185 to 1.57174, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.4469 - f1_score: 0.440
5 - loss: 2.0272 - precision: 0.7333 - val_accuracy: 0.5772 - val_f1_score: 0.567
0 - val_loss: 1.5717 - val_precision: 0.8625
Epoch 14/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.4462 - f1_score: 0.4399
- loss: 2.0240 - precision: 0.7308
Epoch 14: val_loss improved from 1.57174 to 1.57098, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.4462 - f1_score: 0.439
9 - loss: 2.0240 - precision: 0.7308 - val_accuracy: 0.5782 - val_f1_score: 0.570
0 - val_loss: 1.5710 - val_precision: 0.8574
Epoch 15/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.4469 - f1_score: 0.4406
- loss: 2.0277 - precision: 0.7368
Epoch 15: val_loss did not improve from 1.57098
1314/1314 ————— 34s 26ms/step - accuracy: 0.4469 - f1_score: 0.440
6 - loss: 2.0277 - precision: 0.7368 - val_accuracy: 0.5691 - val_f1_score: 0.559
5 - val_loss: 1.5861 - val_precision: 0.8523
Epoch 16/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.4474 - f1_score: 0.4411
- loss: 2.0253 - precision: 0.7332
Epoch 16: val_loss did not improve from 1.57098
1314/1314 ————— 34s 26ms/step - accuracy: 0.4474 - f1_score: 0.441

1 - loss: 2.0253 - precision: 0.7332 - val_accuracy: 0.5708 - val_f1_score: 0.5607 - val_loss: 1.5911 - val_precision: 0.8551
 Epoch 17/25
 1313/1314 ————— 0s 23ms/step - accuracy: 0.4455 - f1_score: 0.4392 - loss: 2.0317 - precision: 0.7318
 Epoch 17: val_loss did not improve from 1.57098
 1314/1314 ————— 34s 26ms/step - accuracy: 0.4455 - f1_score: 0.4392 - loss: 2.0317 - precision: 0.7318 - val_accuracy: 0.5781 - val_f1_score: 0.5675 - val_loss: 1.5737 - val_precision: 0.8583
 Epoch 18/25
 1314/1314 ————— 0s 23ms/step - accuracy: 0.4476 - f1_score: 0.4413 - loss: 2.0235 - precision: 0.7297
 Epoch 18: val_loss did not improve from 1.57098
 1314/1314 ————— 34s 26ms/step - accuracy: 0.4476 - f1_score: 0.4413 - loss: 2.0235 - precision: 0.7297 - val_accuracy: 0.5729 - val_f1_score: 0.5656 - val_loss: 1.5796 - val_precision: 0.8595
 Epoch 19/25
 1313/1314 ————— 0s 23ms/step - accuracy: 0.4452 - f1_score: 0.4396 - loss: 2.0301 - precision: 0.7364
 Epoch 19: val_loss did not improve from 1.57098
 1314/1314 ————— 34s 26ms/step - accuracy: 0.4452 - f1_score: 0.4396 - loss: 2.0301 - precision: 0.7364 - val_accuracy: 0.5718 - val_f1_score: 0.5617 - val_loss: 1.5825 - val_precision: 0.8602



141/141  **11s** 41ms/step - accuracy: 0.5647 - f1_score: 0.5507
- loss: 1.5889 - precision: 0.8484
Test Accuracy: 57.69%
Test Loss: 1.5593595504760742
Test Precision: 85.85%
Test F1 Scores (Per Class): [78.75 68.31682 54.748596 23.214283 22.222218 5
9.66851 50.39369
65.968575 79.53216 79.787224 60.1156 40.217384 60.810806 48.93617
60.714275 51.923065 71.13402 55.10203 57.291664 52.1739 74.390236
71.1111 77.38094 72.41379 74.71264 59.259247 44.897953 32.653057
77.09496 54.90196 34.92063 62.14689 35.03649 47.154465 57.4074
0. 66.666664 37.878784 33.33333 86.74699 65.49707 73.846146
55.555546 63.21243 45.21739 29.565212 42.424236 41.379303 76.66666
76.83616 32.95454 63.440853 53.403137 85.39326 62.275444 32.352936
68.78981 59.574463 63.703697 23.214283 76.13636 73.26733 52.
58.163254 41.249996 42.10526 60.927155 48.648643 80.95238 78.26086
60.829483 69.791664 38.993706 49.10179 31.155773 78.260864 81.31868
45.344128 62.1118 60.431652 26.95035 48.571423 76.85184 64.47368
51.336895 67.105255 69.565216 73.863625 53.60824 56.999992 57.142853
57.71144 48.167538 59.39393 79.41175 55.284554 25.714281 58.88888
40.449436 75.13227]
Average Test F1 Scores:56.92

282/282  **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.90	0.70	0.79	90
dolphin	0.62	0.77	0.68	90
otter	0.56	0.54	0.55	90
seal	0.59	0.14	0.23	90
whale	0.33	0.17	0.22	90
aquarium fish	0.59	0.60	0.60	90
flatfish	0.39	0.71	0.50	90
ray	0.62	0.70	0.66	90
shark	0.85	0.76	0.80	90
trout	0.77	0.83	0.80	90
orchids	0.63	0.58	0.60	90
poppies	0.40	0.42	0.41	90
roses	0.78	0.50	0.61	90
sunflowers	0.47	0.51	0.49	90
tulips	0.65	0.57	0.61	90
bottles	0.46	0.60	0.52	90
bowls	0.66	0.77	0.71	90
cans	0.40	0.90	0.55	90
cups	0.53	0.61	0.57	90
plates	0.59	0.47	0.52	90
apples	0.82	0.68	0.74	90
mushrooms	0.71	0.71	0.71	90
oranges	0.83	0.72	0.77	90
pears	0.75	0.70	0.72	90
sweet peppers	0.77	0.72	0.75	90
clock	0.57	0.62	0.59	90
computer keyboard	0.42	0.49	0.45	90
lamp	0.42	0.27	0.33	90
telephone	0.78	0.77	0.77	90
television	0.67	0.47	0.55	90
bed	0.61	0.24	0.35	90
chair	0.63	0.61	0.62	90
couch	0.51	0.27	0.35	90
table	0.37	0.64	0.47	90
wardrobe	0.49	0.69	0.57	90
bee	0.00	0.00	0.00	90
beetle	0.75	0.60	0.67	90
butterfly	0.60	0.28	0.38	90
caterpillar	0.33	0.33	0.33	90
cockroach	0.95	0.80	0.87	90
bear	0.69	0.62	0.65	90
leopard	0.69	0.80	0.74	90
lion	0.74	0.44	0.56	90
tiger	0.59	0.68	0.63	90
wolf	0.37	0.58	0.45	90
bridge	0.68	0.19	0.30	90
castle	0.31	0.70	0.43	90
house	0.29	0.73	0.41	90
road	0.77	0.77	0.77	90
skyscraper	0.78	0.76	0.77	90
cloud	0.34	0.32	0.33	90
forest	0.62	0.66	0.64	90
mountain	0.50	0.57	0.53	90
plain	0.86	0.84	0.85	90
sea	0.68	0.58	0.62	90
camel	0.48	0.24	0.32	90
cattle	0.81	0.60	0.69	90
chimpanzee	0.48	0.78	0.60	90

elephant	0.96	0.48	0.64	90
kangaroo	0.59	0.14	0.23	90
fox	0.78	0.74	0.76	90
porcupine	0.66	0.82	0.73	90
possum	0.65	0.43	0.52	90
raccoon	0.54	0.63	0.58	90
skunk	0.47	0.37	0.41	90
crab	0.52	0.36	0.42	90
lobster	0.75	0.51	0.61	90
snail	0.47	0.50	0.49	90
spider	0.71	0.94	0.81	90
worm	0.88	0.70	0.78	90
baby	0.52	0.73	0.61	90
boy	0.66	0.74	0.70	90
girl	0.45	0.34	0.39	90
man	0.53	0.46	0.49	90
woman	0.28	0.34	0.31	90
crocodile	0.77	0.80	0.78	90
dinosaur	0.80	0.82	0.81	90
lizard	0.36	0.62	0.45	90
snake	0.71	0.57	0.63	90
turtle	0.86	0.47	0.60	90
hamster	0.37	0.21	0.27	90
mouse	0.42	0.57	0.49	90
rabbit	0.66	0.92	0.77	90
shrew	0.79	0.54	0.64	90
squirrel	0.49	0.53	0.51	90
maple	0.82	0.57	0.67	90
oak	0.62	0.80	0.70	90
palm	0.76	0.72	0.74	90
pine	0.50	0.58	0.54	90
willow	0.52	0.63	0.57	90
bicycle	0.74	0.47	0.57	90
bus	0.52	0.64	0.58	90
motorcycle	0.46	0.51	0.48	90
pickup truck	0.65	0.54	0.59	90
train	0.71	0.90	0.79	90
lawn-mower	0.44	0.76	0.55	90
rocket	0.36	0.20	0.26	90
streetcar	0.59	0.59	0.59	90
tank	0.41	0.40	0.40	90
tractor	0.72	0.79	0.75	90
accuracy			0.58	9000
macro avg	0.60	0.58	0.57	9000
weighted avg	0.60	0.58	0.57	9000

Model: "sequential_4"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_4 (GlobalAveragePooling2D)	(None, 2048)
dense_8 (Dense)	(None, 512)
dropout_4 (Dropout)	(None, 512)
dense_9 (Dense)	(None, 100)



Total params: 24,665,188 (94.09 MB)
Trainable params: 1,100,388 (4.20 MB)
Non-trainable params: 23,564,800 (89.89 MB)

Epoch 1/15
1969/1971 ————— **0s** 25ms/step - accuracy: 0.4721 - f1_score: 0.4648
- loss: 1.9266 - precision: 0.7484
Epoch 1: val_loss improved from inf to 1.46425, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **70s** 30ms/step - accuracy: 0.4721 - f1_score: 0.4649 - loss: 1.9265 - precision: 0.7484 - val_accuracy: 0.6007 - val_f1_score: 0.5958 - val_loss: 1.4642 - val_precision: 0.8615
Epoch 2/15
1969/1971 ————— **0s** 24ms/step - accuracy: 0.5023 - f1_score: 0.4963
- loss: 1.7935 - precision: 0.7744
Epoch 2: val_loss improved from 1.46425 to 1.44226, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.5023 - f1_score: 0.4963 - loss: 1.7934 - precision: 0.7744 - val_accuracy: 0.6048 - val_f1_score: 0.6001 - val_loss: 1.4423 - val_precision: 0.8571
Epoch 3/15
1971/1971 ————— **0s** 24ms/step - accuracy: 0.5149 - f1_score: 0.5088
- loss: 1.7524 - precision: 0.7783
Epoch 3: val_loss improved from 1.44226 to 1.42835, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.5149 - f1_score: 0.5088 - loss: 1.7524 - precision: 0.7783 - val_accuracy: 0.6072 - val_f1_score: 0.6025 - val_loss: 1.4284 - val_precision: 0.8553
Epoch 4/15
1970/1971 ————— **0s** 24ms/step - accuracy: 0.5222 - f1_score: 0.5165
- loss: 1.7228 - precision: 0.7867
Epoch 4: val_loss improved from 1.42835 to 1.41493, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.5222 - f1_score: 0.5165 - loss: 1.7228 - precision: 0.7867 - val_accuracy: 0.6110 - val_f1_score: 0.6067 - val_loss: 1.4149 - val_precision: 0.8564
Epoch 5/15
1970/1971 ————— **0s** 24ms/step - accuracy: 0.5322 - f1_score: 0.5269
- loss: 1.6915 - precision: 0.7917
Epoch 5: val_loss improved from 1.41493 to 1.40334, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.5322 - f1_score: 0.5269 - loss: 1.6914 - precision: 0.7917 - val_accuracy: 0.6130 - val_f1_score: 0.6086 - val_loss: 1.4033 - val_precision: 0.8558
Epoch 6/15
1969/1971 ————— **0s** 24ms/step - accuracy: 0.5360 - f1_score: 0.5305
- loss: 1.6630 - precision: 0.7924
Epoch 6: val_loss improved from 1.40334 to 1.39403, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.5360 - f1_score: 0.5305 - loss: 1.6630 - precision: 0.7924 - val_accuracy: 0.6144 - val_f1_score: 0.6105 - val_loss: 1.3940 - val_precision: 0.8547
Epoch 7/15
1971/1971 ————— **0s** 24ms/step - accuracy: 0.5390 - f1_score: 0.5339
- loss: 1.6501 - precision: 0.7968
Epoch 7: val_loss improved from 1.39403 to 1.38518, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.5390 - f1_score: 0.5339 - loss: 1.6500 - precision: 0.7968 - val_accuracy: 0.6168 - val_f1_score: 0.6130 - val_loss: 1.3852 - val_precision: 0.8537
Epoch 8/15
1969/1971 ————— **0s** 24ms/step - accuracy: 0.5444 - f1_score: 0.5393
- loss: 1.6313 - precision: 0.7991
Epoch 8: val_loss improved from 1.38518 to 1.37847, saving model to best_model_fine_tune_v3.keras

ne_tune_v3.keras

1971/1971 ————— 50s 26ms/step - accuracy: 0.5444 - f1_score: 0.5393 - loss: 1.6313 - precision: 0.7991 - val_accuracy: 0.6190 - val_f1_score: 0.6154 - val_loss: 1.3785 - val_precision: 0.8528

Epoch 9/15

1970/1971 ————— 0s 24ms/step - accuracy: 0.5480 - f1_score: 0.5430 - loss: 1.6153 - precision: 0.7995

Epoch 9: val_loss improved from 1.37847 to 1.37100, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 26ms/step - accuracy: 0.5480 - f1_score: 0.5430 - loss: 1.6152 - precision: 0.7995 - val_accuracy: 0.6199 - val_f1_score: 0.6164 - val_loss: 1.3710 - val_precision: 0.8530

Epoch 10/15

1970/1971 ————— 0s 23ms/step - accuracy: 0.5524 - f1_score: 0.5473 - loss: 1.5997 - precision: 0.8041

Epoch 10: val_loss improved from 1.37100 to 1.36508, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.5524 - f1_score: 0.5473 - loss: 1.5997 - precision: 0.8041 - val_accuracy: 0.6223 - val_f1_score: 0.6188 - val_loss: 1.3651 - val_precision: 0.8521

Epoch 11/15

1971/1971 ————— 0s 23ms/step - accuracy: 0.5582 - f1_score: 0.5535 - loss: 1.5858 - precision: 0.8073

Epoch 11: val_loss improved from 1.36508 to 1.35918, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 26ms/step - accuracy: 0.5582 - f1_score: 0.5535 - loss: 1.5858 - precision: 0.8073 - val_accuracy: 0.6233 - val_f1_score: 0.6200 - val_loss: 1.3592 - val_precision: 0.8509

Epoch 12/15

1971/1971 ————— 0s 24ms/step - accuracy: 0.5589 - f1_score: 0.5539 - loss: 1.5731 - precision: 0.8081

Epoch 12: val_loss improved from 1.35918 to 1.35371, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 26ms/step - accuracy: 0.5589 - f1_score: 0.5539 - loss: 1.5731 - precision: 0.8081 - val_accuracy: 0.6237 - val_f1_score: 0.6206 - val_loss: 1.3537 - val_precision: 0.8489

Epoch 13/15

1969/1971 ————— 0s 23ms/step - accuracy: 0.5642 - f1_score: 0.5594 - loss: 1.5610 - precision: 0.8079

Epoch 13: val_loss improved from 1.35371 to 1.34899, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 50s 25ms/step - accuracy: 0.5642 - f1_score: 0.5594 - loss: 1.5610 - precision: 0.8079 - val_accuracy: 0.6248 - val_f1_score: 0.6217 - val_loss: 1.3490 - val_precision: 0.8483

Epoch 14/15

1969/1971 ————— 0s 23ms/step - accuracy: 0.5642 - f1_score: 0.5595 - loss: 1.5471 - precision: 0.8112

Epoch 14: val_loss improved from 1.34899 to 1.34432, saving model to best_model_fine_tune_v3.keras

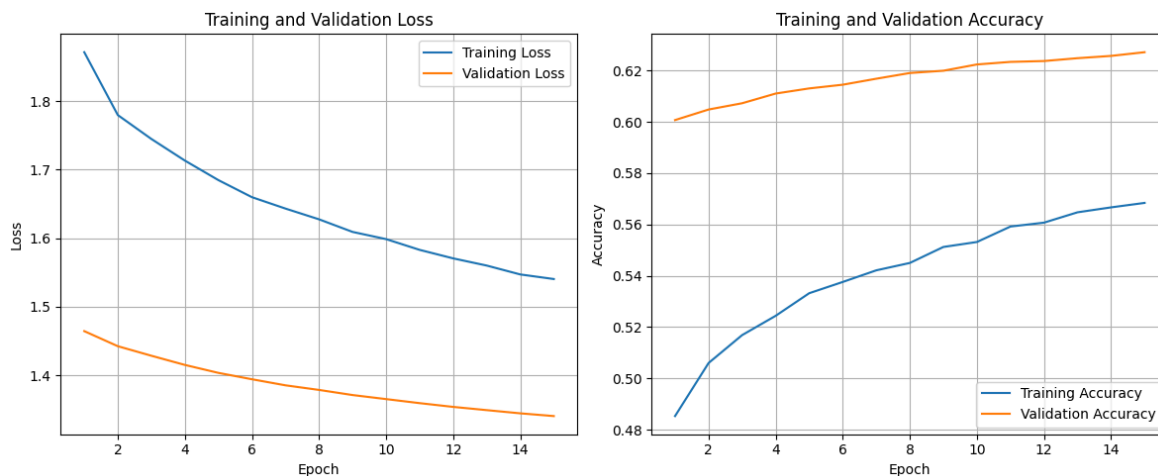
1971/1971 ————— 50s 25ms/step - accuracy: 0.5642 - f1_score: 0.5595 - loss: 1.5471 - precision: 0.8112 - val_accuracy: 0.6257 - val_f1_score: 0.6227 - val_loss: 1.3443 - val_precision: 0.8490

Epoch 15/15

1970/1971 ————— 0s 24ms/step - accuracy: 0.5670 - f1_score: 0.5625 - loss: 1.5395 - precision: 0.8130

Epoch 15: val_loss improved from 1.34432 to 1.34033, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.5670 - f1_score: 0.5625 - loss: 1.5395 - precision: 0.8130 - val_accuracy: 0.6271 - val_f1_score: 0.6243 - val_loss: 1.3403 - val_precision: 0.8489



141/141 ————— **11s** 38ms/step - accuracy: 0.6165 - f1_score: 0.6084

- loss: 1.3475 - precision: 0.8457

Test Accuracy: 62.90%

Test Loss: 1.3231642246246338

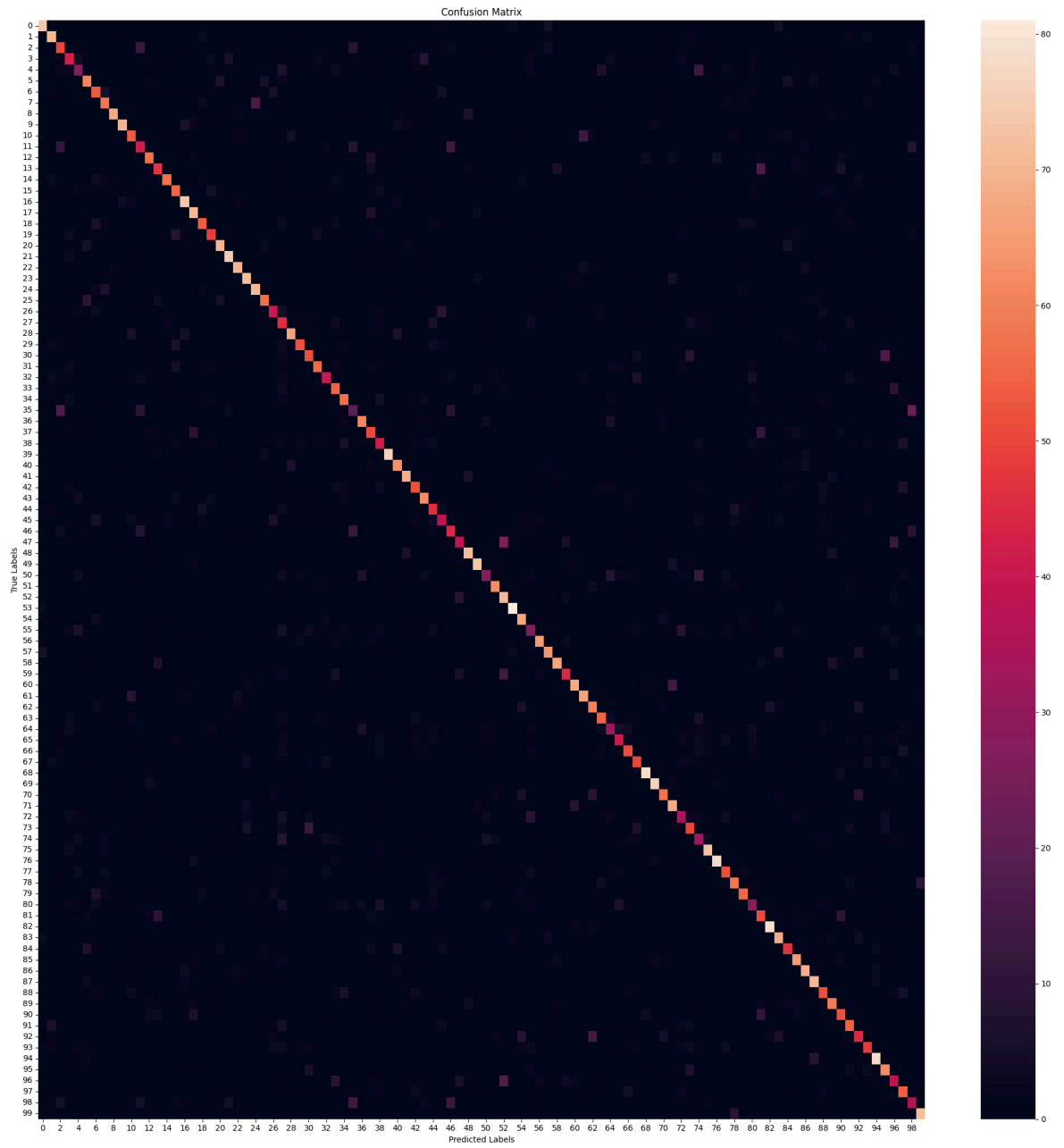
Test Precision: 85.69%

Test F1 Scores (Per Class): [83.14606 72.82051 51.813465 48.044685 33.540367 64.55026 54.35897

64.80447 79.99999 81.87134 59.999996 47.999996 62.29508 55.294113
67.45561 59.459454 74.371864 71.71716 59.340656 58.682625 77.34807
77.7202 77.34807 76.190475 76.34408 67.45561 47.904186 38.33333
77.01149 63.803684 58.100555 67.06587 50.632904 52.88461 63.33333
23.899366 68.53932 53.475933 45.901634 89.534874 67.37968 77.9661
58.426964 67.02702 48.42105 44.705875 49.162006 49.350643 73.846146
81.31868 35.999996 68.508286 63.39285 88.52458 66.666664 32.298134
76.02339 71.50838 77.906975 56.774185 75.55555 72.04301 64.17112
61.797745 40.764324 50.61728 58.620686 57.471264 83.422455 82.608696
66.66666 70.83333 42.23602 55.865913 33.507847 79.347824 85.71427
58.757053 64.088394 65.88235 32.558136 52.30769 88.76403 70.83333
54.651157 70.65217 75.82417 75.13227 56.521736 64.480865 58.100555
60.439552 51.764698 57.309933 83.422455 64.921455 42.857136 59.340656
39.999996 77.00535]

Average Test F1 Scores:62.66

282/282 ————— **11s** 27ms/step



	precision	recall	f1-score	support
beaver	0.84	0.82	0.83	90
dolphin	0.68	0.79	0.73	90
otter	0.49	0.56	0.52	90
seal	0.48	0.48	0.48	90
whale	0.38	0.30	0.34	90
aquarium fish	0.62	0.68	0.65	90
flatfish	0.50	0.59	0.54	90
ray	0.65	0.64	0.65	90
shark	0.85	0.76	0.80	90
trout	0.86	0.78	0.82	90
orchids	0.60	0.59	0.59	90
poppies	0.49	0.47	0.48	90
roses	0.61	0.63	0.62	90
sunflowers	0.59	0.52	0.55	90
tulips	0.72	0.63	0.67	90
bottles	0.58	0.61	0.59	90
bowls	0.68	0.82	0.74	90
cans	0.66	0.79	0.72	90
cups	0.59	0.60	0.59	90
plates	0.64	0.54	0.59	90
apples	0.77	0.78	0.77	90
mushrooms	0.73	0.83	0.78	90
oranges	0.77	0.78	0.77	90
pears	0.73	0.80	0.76	90
sweet peppers	0.74	0.79	0.76	90
clock	0.72	0.63	0.67	90
computer keyboard	0.52	0.44	0.48	90
lamp	0.31	0.51	0.38	90
telephone	0.80	0.74	0.77	90
television	0.71	0.58	0.64	90
bed	0.58	0.58	0.58	90
chair	0.73	0.62	0.67	90
couch	0.59	0.44	0.51	90
table	0.47	0.61	0.53	90
wardrobe	0.63	0.63	0.63	90
bee	0.28	0.21	0.24	90
beetle	0.69	0.68	0.69	90
butterfly	0.52	0.56	0.53	90
caterpillar	0.45	0.47	0.46	90
cockroach	0.94	0.84	0.89	90
bear	0.65	0.70	0.67	90
leopard	0.79	0.77	0.78	90
lion	0.59	0.58	0.58	90
tiger	0.65	0.69	0.67	90
wolf	0.46	0.51	0.48	90
bridge	0.47	0.42	0.45	90
castle	0.49	0.49	0.49	90
house	0.59	0.42	0.49	90
road	0.69	0.80	0.74	90
skyscraper	0.80	0.82	0.81	90
cloud	0.45	0.30	0.36	90
forest	0.69	0.69	0.69	90
mountain	0.53	0.79	0.63	90
plain	0.87	0.90	0.89	90
sea	0.60	0.74	0.67	90
camel	0.37	0.29	0.32	90
cattle	0.80	0.72	0.76	90
chimpanzee	0.72	0.71	0.72	90

elephant	0.82	0.74	0.78	90
kangaroo	0.68	0.49	0.57	90
fox	0.76	0.76	0.76	90
porcupine	0.70	0.74	0.72	90
possum	0.62	0.67	0.64	90
raccoon	0.62	0.61	0.62	90
skunk	0.48	0.36	0.41	90
crab	0.57	0.46	0.51	90
lobster	0.61	0.57	0.59	90
snail	0.60	0.56	0.57	90
spider	0.80	0.87	0.83	90
worm	0.81	0.84	0.83	90
baby	0.70	0.63	0.67	90
boy	0.67	0.76	0.71	90
girl	0.48	0.38	0.42	90
man	0.56	0.56	0.56	90
woman	0.32	0.36	0.34	90
crocodile	0.78	0.81	0.79	90
dinosaur	0.85	0.87	0.86	90
lizard	0.60	0.58	0.59	90
snake	0.64	0.64	0.64	90
turtle	0.70	0.62	0.66	90
hamster	0.34	0.31	0.33	90
mouse	0.49	0.57	0.52	90
rabbit	0.90	0.88	0.89	90
shrew	0.66	0.76	0.70	90
squirrel	0.57	0.52	0.54	90
maple	0.69	0.72	0.71	90
oak	0.74	0.77	0.75	90
palm	0.72	0.79	0.75	90
pine	0.55	0.58	0.57	90
willow	0.63	0.66	0.64	90
bicycle	0.58	0.58	0.58	90
bus	0.59	0.61	0.60	90
motorcycle	0.56	0.49	0.52	90
pickup truck	0.60	0.54	0.57	90
train	0.80	0.87	0.83	90
lawn-mower	0.61	0.69	0.65	90
rocket	0.42	0.43	0.43	90
streetcar	0.59	0.60	0.59	90
tank	0.41	0.39	0.40	90
tractor	0.74	0.80	0.77	90
accuracy			0.63	9000
macro avg	0.63	0.63	0.63	9000
weighted avg	0.63	0.63	0.63	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

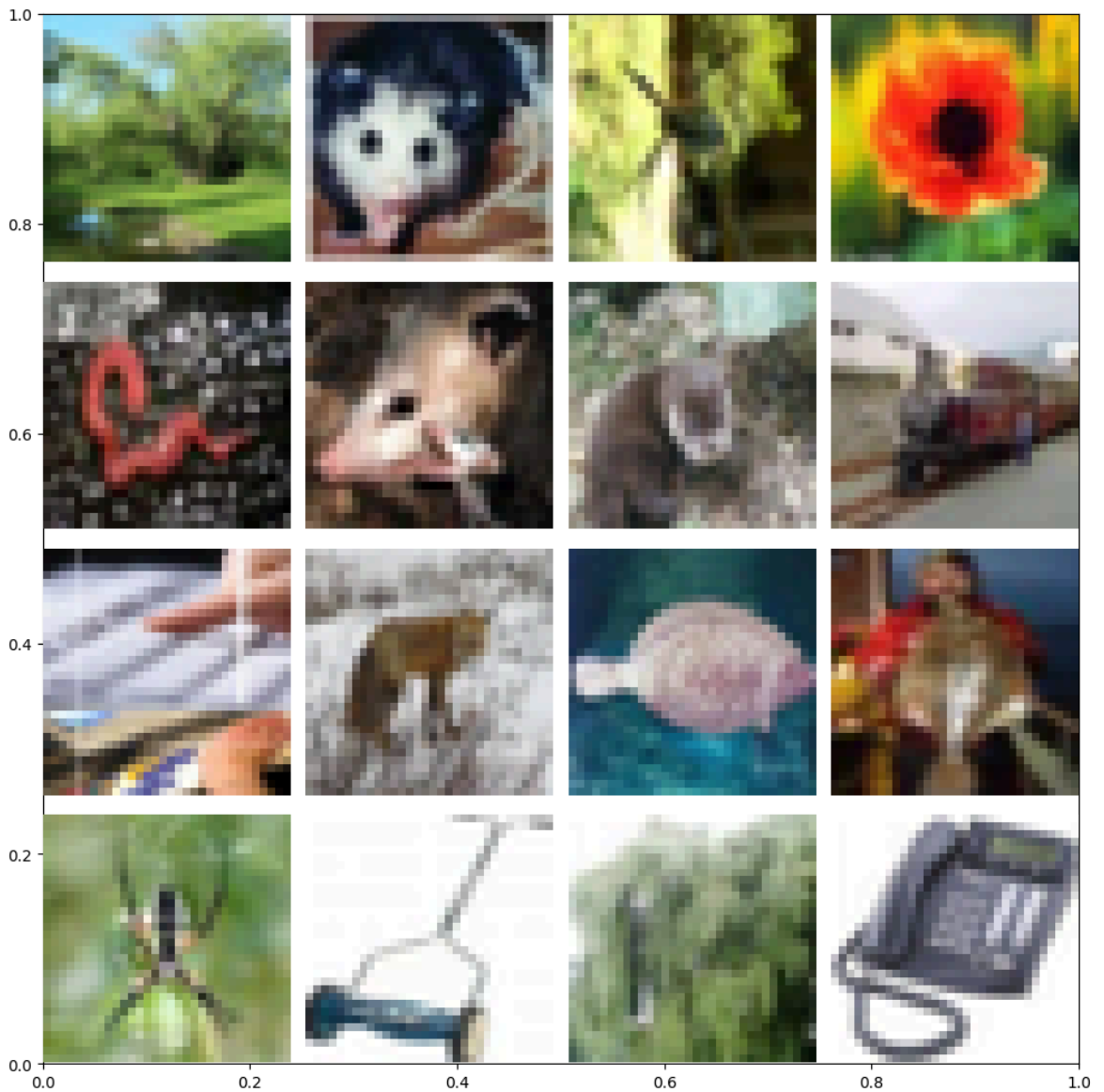
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

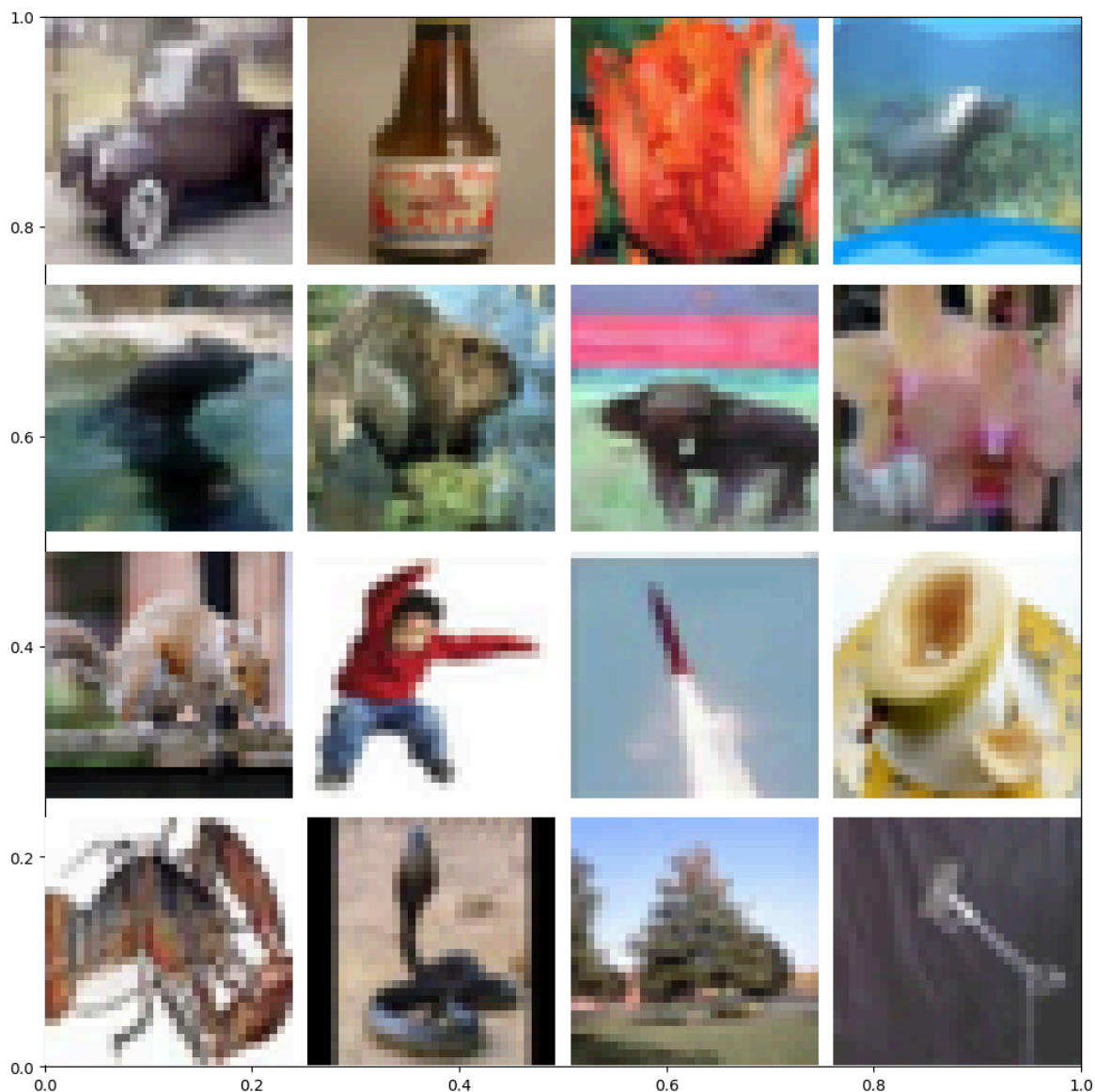
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[3796].shape: (1,)  
imgs[3796].shape: (32, 32, 3)
```

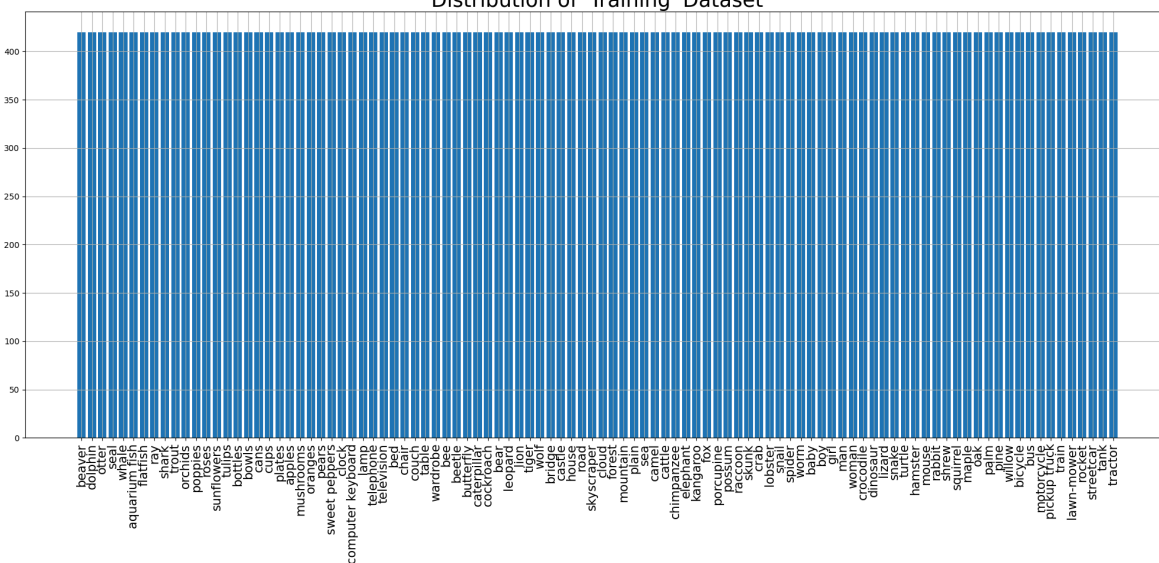


```
labels[30338].shape: (1,)  
imgs[30338].shape: (32, 32, 3)
```

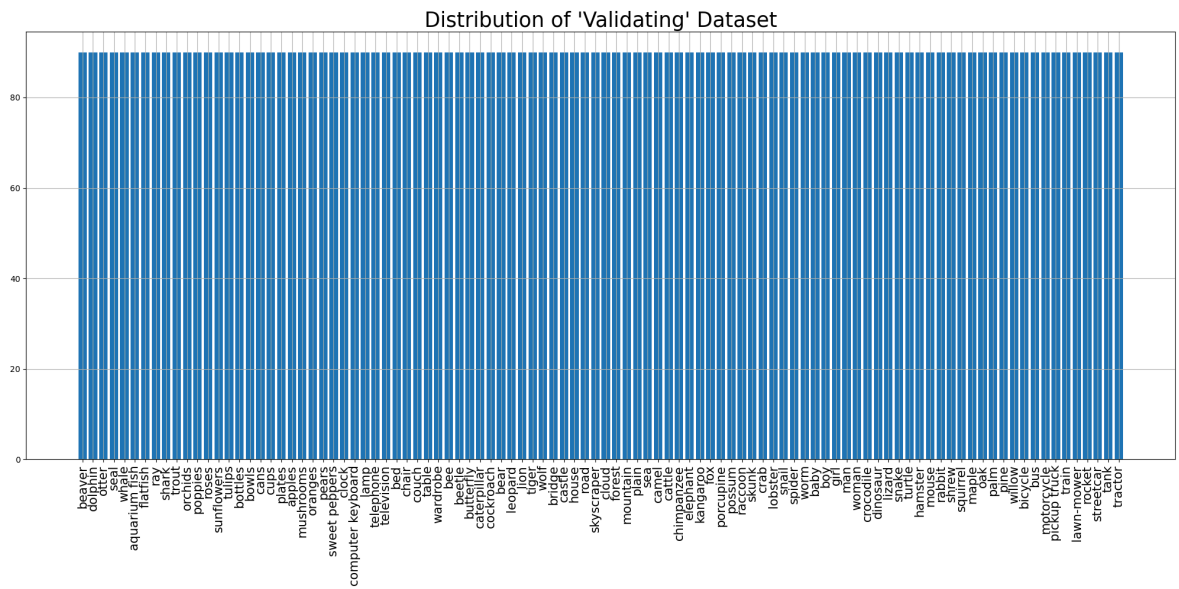


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

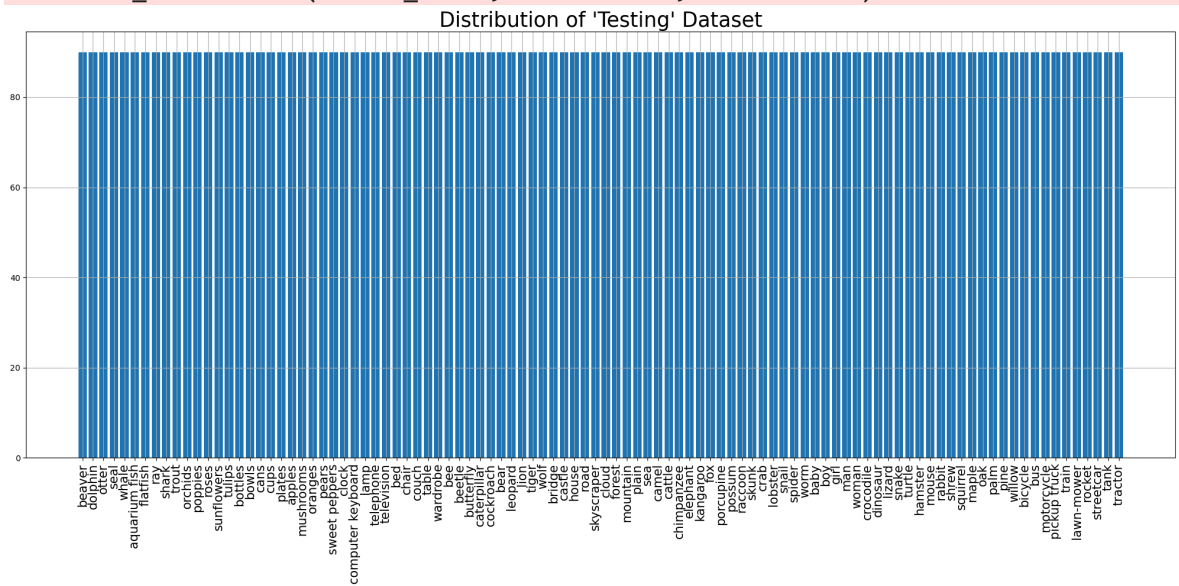
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_ConcatenateDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

```
<class 'tuple'>
```

2

```
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
```

(64, 224, 224, 3)

(64, 100)

Model input shape: (None, 224, 224, 3)

Model output shape: (None, 100)

2

```
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
```

(64, 224, 224, 3) (64, 100)

Model: "sequential_5"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_5 (GlobalAveragePooling2D)	(None , 2048)
dense_10 (Dense)	(None , 512)
dropout_5 (Dropout)	(None , 512)
dense_11 (Dense)	(None , 100)

◀  ▶

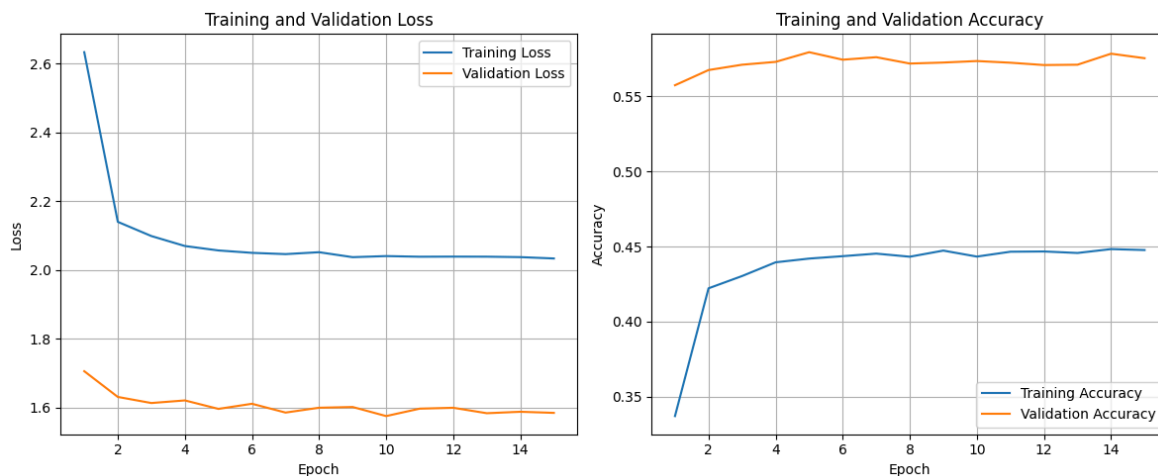
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
1313/1314 ————— 0s 26ms/step - accuracy: 0.2482 - f1_score: 0.2399
- loss: 3.2151 - precision: 0.6366
Epoch 1: val_loss improved from inf to 1.70520, saving model to best_model_v3.keras as
1314/1314 ————— 54s 34ms/step - accuracy: 0.2484 - f1_score: 0.240
0 - loss: 3.2142 - precision: 0.6367 - val_accuracy: 0.5574 - val_f1_score: 0.546
8 - val_loss: 1.7052 - val_precision: 0.8685
Epoch 2/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.4166 - f1_score: 0.4097
- loss: 2.1577 - precision: 0.7195
Epoch 2: val_loss improved from 1.70520 to 1.63043, saving model to best_model_v3.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.4166 - f1_score: 0.409
7 - loss: 2.1576 - precision: 0.7195 - val_accuracy: 0.5676 - val_f1_score: 0.556
7 - val_loss: 1.6304 - val_precision: 0.8621
Epoch 3/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.4292 - f1_score: 0.4223
- loss: 2.1058 - precision: 0.7252
Epoch 3: val_loss improved from 1.63043 to 1.61273, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.4292 - f1_score: 0.422
3 - loss: 2.1058 - precision: 0.7252 - val_accuracy: 0.5711 - val_f1_score: 0.560
1 - val_loss: 1.6127 - val_precision: 0.8664
Epoch 4/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.4382 - f1_score: 0.4318
- loss: 2.0721 - precision: 0.7314
Epoch 4: val_loss did not improve from 1.61273
1314/1314 ————— 34s 26ms/step - accuracy: 0.4382 - f1_score: 0.431
8 - loss: 2.0721 - precision: 0.7315 - val_accuracy: 0.5730 - val_f1_score: 0.564
9 - val_loss: 1.6203 - val_precision: 0.8721
Epoch 5/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.4407 - f1_score: 0.4339
- loss: 2.0620 - precision: 0.7325
Epoch 5: val_loss improved from 1.61273 to 1.59568, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.4407 - f1_score: 0.433
9 - loss: 2.0620 - precision: 0.7325 - val_accuracy: 0.5794 - val_f1_score: 0.567
6 - val_loss: 1.5957 - val_precision: 0.8614
Epoch 6/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.4439 - f1_score: 0.4372
- loss: 2.0463 - precision: 0.7343
Epoch 6: val_loss did not improve from 1.59568
1314/1314 ————— 34s 26ms/step - accuracy: 0.4439 - f1_score: 0.437
2 - loss: 2.0463 - precision: 0.7343 - val_accuracy: 0.5744 - val_f1_score: 0.564
7 - val_loss: 1.6106 - val_precision: 0.8730
Epoch 7/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.4462 - f1_score: 0.4402
- loss: 2.0431 - precision: 0.7300
Epoch 7: val_loss improved from 1.59568 to 1.58497, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.4462 - f1_score: 0.440
2 - loss: 2.0431 - precision: 0.7300 - val_accuracy: 0.5761 - val_f1_score: 0.566
9 - val_loss: 1.5850 - val_precision: 0.8594
Epoch 8/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.4438 - f1_score: 0.4375
- loss: 2.0441 - precision: 0.7277
Epoch 8: val_loss did not improve from 1.58497
1314/1314 ————— 34s 26ms/step - accuracy: 0.4438 - f1_score: 0.437
5 - loss: 2.0441 - precision: 0.7277 - val_accuracy: 0.5719 - val_f1_score: 0.563

3 - val_loss: 1.5990 - val_precision: 0.8678
Epoch 9/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.4483 - f1_score: 0.4419
- loss: 2.0346 - precision: 0.7376
Epoch 9: val_loss did not improve from 1.58497
1314/1314 ————— 34s 26ms/step - accuracy: 0.4483 - f1_score: 0.441
9 - loss: 2.0346 - precision: 0.7375 - val_accuracy: 0.5726 - val_f1_score: 0.563
3 - val_loss: 1.6012 - val_precision: 0.8719
Epoch 10/25
1312/1314 ————— 0s 23ms/step - accuracy: 0.4433 - f1_score: 0.4369
- loss: 2.0378 - precision: 0.7371
Epoch 10: val_loss improved from 1.58497 to 1.57480, saving model to best_model_v3.keras
1314/1314 ————— 35s 26ms/step - accuracy: 0.4433 - f1_score: 0.436
9 - loss: 2.0378 - precision: 0.7371 - val_accuracy: 0.5736 - val_f1_score: 0.563
4 - val_loss: 1.5748 - val_precision: 0.8560
Epoch 11/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.4462 - f1_score: 0.4396
- loss: 2.0388 - precision: 0.7319
Epoch 11: val_loss did not improve from 1.57480
1314/1314 ————— 34s 26ms/step - accuracy: 0.4462 - f1_score: 0.439
6 - loss: 2.0388 - precision: 0.7319 - val_accuracy: 0.5724 - val_f1_score: 0.559
5 - val_loss: 1.5964 - val_precision: 0.8626
Epoch 12/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.4462 - f1_score: 0.4396
- loss: 2.0360 - precision: 0.7264
Epoch 12: val_loss did not improve from 1.57480
1314/1314 ————— 34s 26ms/step - accuracy: 0.4462 - f1_score: 0.439
6 - loss: 2.0360 - precision: 0.7265 - val_accuracy: 0.5709 - val_f1_score: 0.560
8 - val_loss: 1.5989 - val_precision: 0.8705
Epoch 13/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.4454 - f1_score: 0.4383
- loss: 2.0417 - precision: 0.7307
Epoch 13: val_loss did not improve from 1.57480
1314/1314 ————— 34s 26ms/step - accuracy: 0.4454 - f1_score: 0.438
3 - loss: 2.0417 - precision: 0.7307 - val_accuracy: 0.5711 - val_f1_score: 0.564
2 - val_loss: 1.5831 - val_precision: 0.8563
Epoch 14/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.4489 - f1_score: 0.4432
- loss: 2.0293 - precision: 0.7352
Epoch 14: val_loss did not improve from 1.57480
1314/1314 ————— 34s 26ms/step - accuracy: 0.4489 - f1_score: 0.443
2 - loss: 2.0293 - precision: 0.7352 - val_accuracy: 0.5784 - val_f1_score: 0.568
3 - val_loss: 1.5873 - val_precision: 0.8608
Epoch 15/25
1313/1314 ————— 0s 23ms/step - accuracy: 0.4456 - f1_score: 0.4394
- loss: 2.0357 - precision: 0.7321
Epoch 15: val_loss did not improve from 1.57480
1314/1314 ————— 34s 26ms/step - accuracy: 0.4456 - f1_score: 0.439
4 - loss: 2.0357 - precision: 0.7321 - val_accuracy: 0.5754 - val_f1_score: 0.566
6 - val_loss: 1.5841 - val_precision: 0.8593



141/141 ————— 11s 37ms/step - accuracy: 0.5662 - f1_score: 0.5526

- loss: 1.5855 - precision: 0.8538

Test Accuracy: 57.90%

Test Loss: 1.5625771284103394

Test Precision: 86.69%

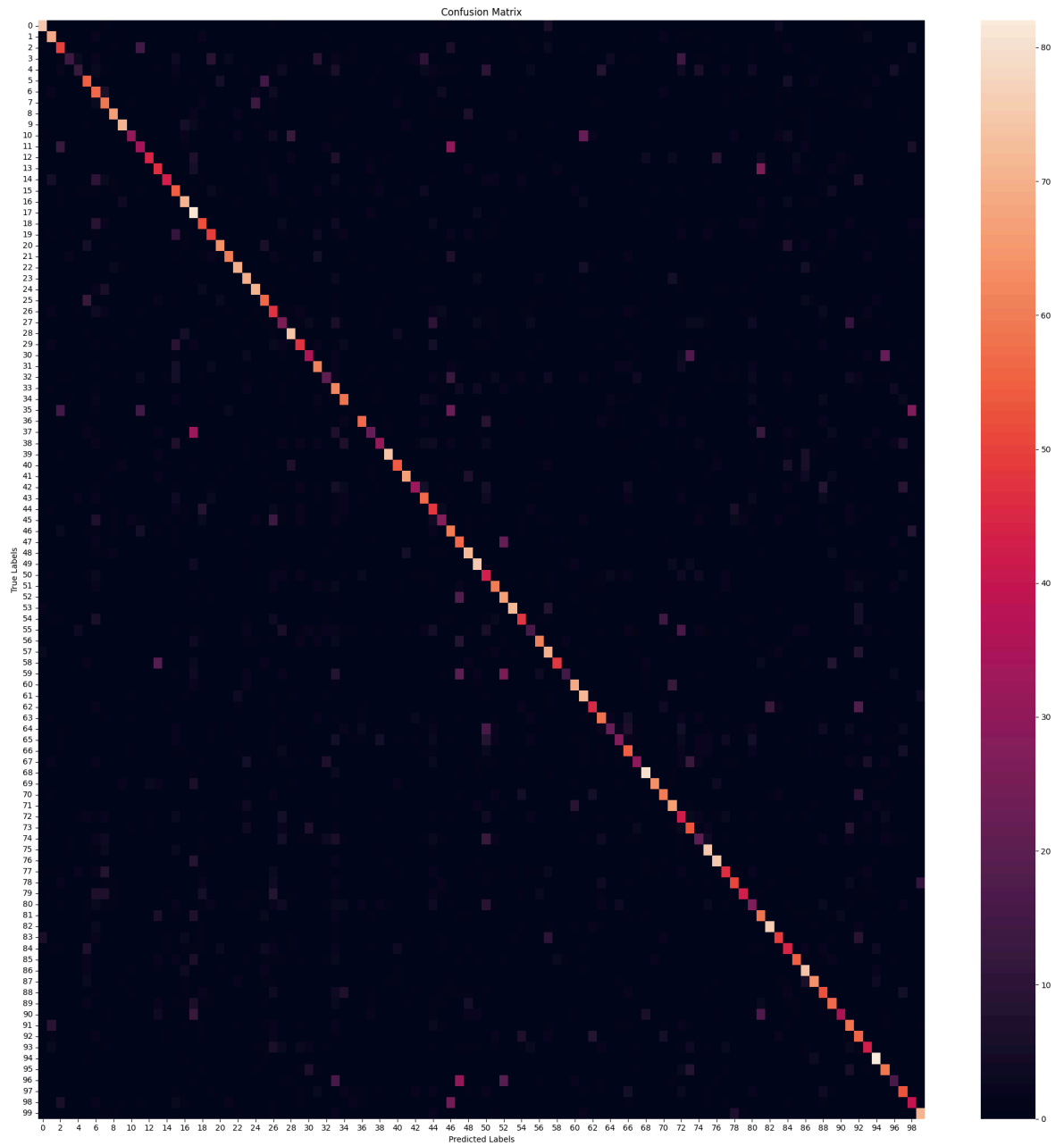
Test F1 Scores (Per Class): [84.26965 70.40816 51.813465 23.214283 20.68965

1 56.994812

45.967735	58.41583	75.706215	82.08092	46.15384	40.96385
63.30935	50.549446	60.56338	48.24561	72.82051	55.290092
54.92227	56.64739	75.73964	73.61964	78.65168	70.35175
76.756744	59.067356	40.34334	28.901731	75.12689	56.6265
48.684204	62.5641	30.656927	41.891888	61.052628	1.9801961
70.37037	33.58778	43.66197	86.549706	65.454544	77.01149
48.1203	61.290314	43.43891	36.486485	44.776115	47.899155
70.93597	77.7202	35.53718	65.93406	58.51528	85.2071
57.142853	24.61538	71.345024	66.35071	66.666664	27.272722
76.24309	67.92453	56.96202	61.702126	35.199997	40.2985
56.41025	42.335762	83.33333	79.01234	63.157898	70.65217
38.00904	53.807102	27.777771	75.75757	78.947365	55.75757
62.576683	59.722214	30.58823	52.017933	83.060104	59.39393
46.56084	67.90123	68.20277	76.646706	53.60824	61.956512
53.435104	55.238087	48.27586	49.71098	82.	62.433857
24.61538	56.684486	43.71584	77.173904	]	

Average Test F1 Scores:57.06

282/282 ————— 11s 28ms/step



	precision	recall	f1-score	support
beaver	0.85	0.83	0.84	90
dolphin	0.65	0.77	0.70	90
otter	0.49	0.56	0.52	90
seal	0.59	0.14	0.23	90
whale	0.46	0.13	0.21	90
aquarium fish	0.53	0.61	0.57	90
flatfish	0.36	0.63	0.46	90
ray	0.53	0.66	0.58	90
shark	0.77	0.74	0.76	90
trout	0.86	0.79	0.82	90
orchids	0.75	0.33	0.46	90
poppies	0.45	0.38	0.41	90
roses	0.90	0.49	0.63	90
sunflowers	0.50	0.51	0.51	90
tulips	0.83	0.48	0.61	90
bottles	0.40	0.61	0.48	90
bowls	0.68	0.79	0.73	90
cans	0.40	0.90	0.55	90
cups	0.51	0.58	0.54	90
plates	0.59	0.54	0.57	90
apples	0.81	0.71	0.76	90
mushrooms	0.82	0.67	0.74	90
oranges	0.80	0.78	0.79	90
pears	0.64	0.78	0.70	90
sweet peppers	0.75	0.79	0.77	90
clock	0.55	0.63	0.59	90
computer keyboard	0.33	0.52	0.40	90
lamp	0.30	0.28	0.29	90
telephone	0.69	0.82	0.75	90
television	0.62	0.52	0.57	90
bed	0.59	0.40	0.48	90
chair	0.58	0.68	0.63	90
couch	0.45	0.23	0.31	90
table	0.30	0.69	0.42	90
wardrobe	0.58	0.64	0.61	90
bee	0.09	0.01	0.02	90
beetle	0.79	0.63	0.70	90
butterfly	0.54	0.24	0.34	90
caterpillar	0.60	0.34	0.44	90
cockroach	0.91	0.82	0.87	90
bear	0.72	0.60	0.65	90
leopard	0.80	0.74	0.77	90
lion	0.74	0.36	0.48	90
tiger	0.59	0.63	0.61	90
wolf	0.36	0.53	0.43	90
bridge	0.47	0.30	0.36	90
castle	0.34	0.67	0.45	90
house	0.39	0.63	0.48	90
road	0.64	0.80	0.71	90
skyscraper	0.74	0.83	0.78	90
cloud	0.28	0.48	0.36	90
forest	0.65	0.67	0.66	90
mountain	0.48	0.74	0.59	90
plain	0.91	0.80	0.85	90
sea	0.62	0.53	0.57	90
camel	0.40	0.18	0.25	90
cattle	0.75	0.68	0.71	90
chimpanzee	0.58	0.78	0.66	90

elephant	0.89	0.53	0.67	90
kangaroo	0.75	0.17	0.27	90
fox	0.76	0.77	0.76	90
porcupine	0.59	0.80	0.68	90
possum	0.66	0.50	0.57	90
raccoon	0.59	0.64	0.62	90
skunk	0.63	0.24	0.35	90
crab	0.61	0.30	0.40	90
lobster	0.52	0.61	0.56	90
snail	0.62	0.32	0.42	90
spider	0.78	0.89	0.83	90
worm	0.89	0.71	0.79	90
baby	0.60	0.67	0.63	90
boy	0.69	0.73	0.71	90
girl	0.32	0.47	0.38	90
man	0.49	0.59	0.54	90
woman	0.37	0.22	0.28	90
crocodile	0.70	0.83	0.76	90
dinosaur	0.75	0.83	0.79	90
lizard	0.61	0.51	0.56	90
snake	0.70	0.57	0.63	90
turtle	0.80	0.48	0.60	90
hamster	0.33	0.29	0.31	90
mouse	0.44	0.64	0.52	90
rabbit	0.82	0.84	0.83	90
shrew	0.65	0.54	0.59	90
squirrel	0.44	0.49	0.47	90
maple	0.76	0.61	0.68	90
oak	0.58	0.82	0.68	90
palm	0.83	0.71	0.77	90
pine	0.50	0.58	0.54	90
willow	0.61	0.63	0.62	90
bicycle	0.85	0.39	0.53	90
bus	0.48	0.64	0.55	90
motorcycle	0.39	0.62	0.48	90
pickup truck	0.52	0.48	0.50	90
train	0.75	0.91	0.82	90
lawn-mower	0.60	0.66	0.62	90
rocket	0.40	0.18	0.25	90
streetcar	0.55	0.59	0.57	90
tank	0.43	0.44	0.43	90
tractor	0.76	0.79	0.77	90
accuracy			0.58	9000
macro avg	0.60	0.58	0.57	9000
weighted avg	0.60	0.58	0.57	9000

Model: "sequential_5"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_5 (GlobalAveragePooling2D)	(None , 2048)
dense_10 (Dense)	(None , 512)
dropout_5 (Dropout)	(None , 512)
dense_11 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1971/1971 ————— **0s** 25ms/step - accuracy: 0.4693 - f1_score: 0.4615
- loss: 1.9428 - precision: 0.7495
Epoch 1: val_loss improved from inf to 1.47285, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **70s** 30ms/step - accuracy: 0.4693 - f1_score: 0.4615
- loss: 1.9427 - precision: 0.7495 - val_accuracy: 0.6004 - val_f1_score: 0.5954
- val_loss: 1.4728 - val_precision: 0.8663
Epoch 2/15
1970/1971 ————— **0s** 24ms/step - accuracy: 0.5007 - f1_score: 0.4945
- loss: 1.8098 - precision: 0.7713
Epoch 2: val_loss improved from 1.47285 to 1.44951, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.5007 - f1_score: 0.4945
- loss: 1.8098 - precision: 0.7713 - val_accuracy: 0.6066 - val_f1_score: 0.6016
- val_loss: 1.4495 - val_precision: 0.8627
Epoch 3/15
1969/1971 ————— **0s** 24ms/step - accuracy: 0.5128 - f1_score: 0.5074
- loss: 1.7628 - precision: 0.7783
Epoch 3: val_loss improved from 1.44951 to 1.43526, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.5128 - f1_score: 0.5074
- loss: 1.7628 - precision: 0.7783 - val_accuracy: 0.6082 - val_f1_score: 0.6038
- val_loss: 1.4353 - val_precision: 0.8606
Epoch 4/15
1971/1971 ————— **0s** 24ms/step - accuracy: 0.5192 - f1_score: 0.5136
- loss: 1.7362 - precision: 0.7868
Epoch 4: val_loss improved from 1.43526 to 1.42166, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.5192 - f1_score: 0.5136
- loss: 1.7362 - precision: 0.7868 - val_accuracy: 0.6119 - val_f1_score: 0.6076
- val_loss: 1.4217 - val_precision: 0.8584
Epoch 5/15
1969/1971 ————— **0s** 24ms/step - accuracy: 0.5261 - f1_score: 0.5206
- loss: 1.7082 - precision: 0.7855
Epoch 5: val_loss improved from 1.42166 to 1.41086, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **50s** 26ms/step - accuracy: 0.5261 - f1_score: 0.5206
- loss: 1.7081 - precision: 0.7855 - val_accuracy: 0.6152 - val_f1_score: 0.6111
- val_loss: 1.4109 - val_precision: 0.8588
Epoch 6/15
1969/1971 ————— **0s** 24ms/step - accuracy: 0.5338 - f1_score: 0.5286
- loss: 1.6853 - precision: 0.7934
Epoch 6: val_loss improved from 1.41086 to 1.40180, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **50s** 26ms/step - accuracy: 0.5338 - f1_score: 0.5286
- loss: 1.6853 - precision: 0.7934 - val_accuracy: 0.6162 - val_f1_score: 0.6121
- val_loss: 1.4018 - val_precision: 0.8572
Epoch 7/15
1969/1971 ————— **0s** 24ms/step - accuracy: 0.5361 - f1_score: 0.5304
- loss: 1.6703 - precision: 0.7956
Epoch 7: val_loss improved from 1.40180 to 1.39306, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **50s** 26ms/step - accuracy: 0.5361 - f1_score: 0.5304
- loss: 1.6702 - precision: 0.7956 - val_accuracy: 0.6178 - val_f1_score: 0.6138
- val_loss: 1.3931 - val_precision: 0.8558
Epoch 8/15
1970/1971 ————— **0s** 24ms/step - accuracy: 0.5423 - f1_score: 0.5371
- loss: 1.6446 - precision: 0.7984
Epoch 8: val_loss improved from 1.39306 to 1.38608, saving model to best_model_fine_tune_v3.keras

ne_tune_v3.keras

1971/1971 ————— **50s** 26ms/step - accuracy: 0.5423 - f1_score: 0.537
1 - loss: 1.6446 - precision: 0.7984 - val_accuracy: 0.6169 - val_f1_score: 0.612
9 - val_loss: 1.3861 - val_precision: 0.8557

Epoch 9/15

1969/1971 ————— **0s** 24ms/step - accuracy: 0.5462 - f1_score: 0.5413
- loss: 1.6268 - precision: 0.8018

Epoch 9: val_loss improved from 1.38608 to 1.37918, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **50s** 26ms/step - accuracy: 0.5462 - f1_score: 0.541
3 - loss: 1.6268 - precision: 0.8018 - val_accuracy: 0.6192 - val_f1_score: 0.615
3 - val_loss: 1.3792 - val_precision: 0.8559

Epoch 10/15

1969/1971 ————— **0s** 24ms/step - accuracy: 0.5508 - f1_score: 0.5456
- loss: 1.6131 - precision: 0.8022

Epoch 10: val_loss improved from 1.37918 to 1.37258, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **50s** 26ms/step - accuracy: 0.5508 - f1_score: 0.545
6 - loss: 1.6131 - precision: 0.8022 - val_accuracy: 0.6189 - val_f1_score: 0.615
2 - val_loss: 1.3726 - val_precision: 0.8553

Epoch 11/15

1969/1971 ————— **0s** 23ms/step - accuracy: 0.5503 - f1_score: 0.5456
- loss: 1.6017 - precision: 0.8053

Epoch 11: val_loss improved from 1.37258 to 1.36601, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **50s** 26ms/step - accuracy: 0.5503 - f1_score: 0.545
6 - loss: 1.6017 - precision: 0.8053 - val_accuracy: 0.6201 - val_f1_score: 0.616
5 - val_loss: 1.3660 - val_precision: 0.8549

Epoch 12/15

1971/1971 ————— **0s** 23ms/step - accuracy: 0.5580 - f1_score: 0.5530
- loss: 1.5810 - precision: 0.8100

Epoch 12: val_loss improved from 1.36601 to 1.36051, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **50s** 26ms/step - accuracy: 0.5580 - f1_score: 0.553
0 - loss: 1.5810 - precision: 0.8100 - val_accuracy: 0.6218 - val_f1_score: 0.618
3 - val_loss: 1.3605 - val_precision: 0.8541

Epoch 13/15

1969/1971 ————— **0s** 23ms/step - accuracy: 0.5626 - f1_score: 0.5576
- loss: 1.5708 - precision: 0.8096

Epoch 13: val_loss improved from 1.36051 to 1.35587, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **50s** 25ms/step - accuracy: 0.5626 - f1_score: 0.557
6 - loss: 1.5708 - precision: 0.8096 - val_accuracy: 0.6217 - val_f1_score: 0.618
4 - val_loss: 1.3559 - val_precision: 0.8532

Epoch 14/15

1969/1971 ————— **0s** 23ms/step - accuracy: 0.5628 - f1_score: 0.5580
- loss: 1.5631 - precision: 0.8116

Epoch 14: val_loss improved from 1.35587 to 1.35147, saving model to best_model_fine_tune_v3.keras

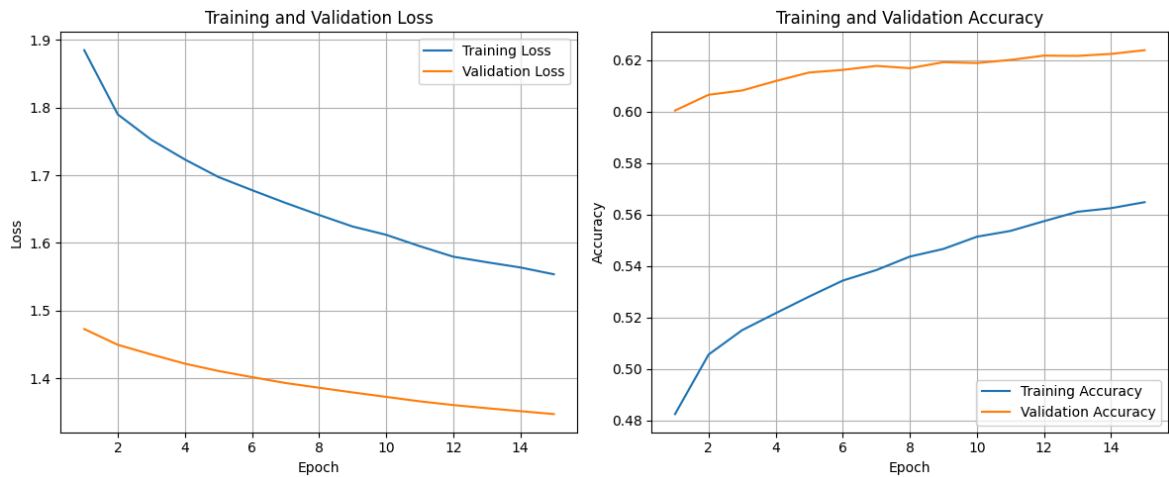
1971/1971 ————— **51s** 26ms/step - accuracy: 0.5628 - f1_score: 0.558
0 - loss: 1.5631 - precision: 0.8116 - val_accuracy: 0.6224 - val_f1_score: 0.619
1 - val_loss: 1.3515 - val_precision: 0.8532

Epoch 15/15

1970/1971 ————— **0s** 24ms/step - accuracy: 0.5645 - f1_score: 0.5597
- loss: 1.5551 - precision: 0.8108

Epoch 15: val_loss improved from 1.35147 to 1.34716, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **51s** 26ms/step - accuracy: 0.5645 - f1_score: 0.559
7 - loss: 1.5551 - precision: 0.8108 - val_accuracy: 0.6239 - val_f1_score: 0.620
5 - val_loss: 1.3472 - val_precision: 0.8539



141/141 ————— **11s** 38ms/step - accuracy: 0.6179 - f1_score: 0.6087

- loss: 1.3524 - precision: 0.8443

Test Accuracy: 62.78%

Test Loss: 1.3295519351959229

Test Precision: 85.86%

Test F1 Scores (Per Class): [82.872925 72.916664 54.34782 47.953213 29.999996 6
1.458332 54.639168

63.905323 78.362564 81.14285 58.757053 49.704136 65.90909 55.421684
66.27218 58.064507 72.72727 74.99999 58.20105 60.60605 75.82417
76.8421 77.77777 73.79678 78.30688 62.352936 50.602406 39.495792
79.768776 65.454544 55.813946 68.263466 41.249996 54.54545 65.217384
28.235287 67.74193 54.143642 45.977005 88.888885 66.66666 78.61271
60.1156 69.31818 47.31182 42.76729 49.71751 48.10126 76.381905
81.08108 33.333332 69.66292 64.57398 85.08286 66.666664 36.249996
75.14451 67.0391 76.300575 51.282047 77.27273 71.65774 63.541664
62.499992 42.857136 54.54545 55.6701 55.75757 87.09677 84.656075
65.088745 70.83333 39.999996 59.459454 32.460724 78.30688 85.56149
57.303364 62.983418 64.77273 32.608692 51.515137 88.888885 68.0203
51.461983 69.27374 77.173904 76.59574 58.24175 63.157898 58.56353
62.29508 52.380943 56.179775 83.87096 64.58333 42.391296 61.62162
40.449436 75.138115]

Average Test F1 Scores:62.51

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.82	0.83	0.83	90
dolphin	0.69	0.79	0.74	90
otter	0.53	0.56	0.54	90
seal	0.51	0.46	0.48	90
whale	0.35	0.27	0.30	90
aquarium fish	0.58	0.66	0.61	90
flatfish	0.50	0.59	0.54	90
ray	0.68	0.60	0.64	90
shark	0.83	0.74	0.78	90
trout	0.84	0.79	0.81	90
orchids	0.60	0.58	0.59	90
poppies	0.53	0.47	0.50	90
roses	0.67	0.64	0.66	90
sunflowers	0.61	0.51	0.55	90
tulips	0.71	0.62	0.66	90
bottles	0.56	0.60	0.58	90
bowls	0.67	0.80	0.73	90
cans	0.71	0.81	0.76	90
cups	0.56	0.61	0.58	90
plates	0.67	0.56	0.61	90
apples	0.75	0.77	0.76	90
mushrooms	0.73	0.81	0.77	90
oranges	0.78	0.78	0.78	90
pears	0.71	0.77	0.74	90
sweet peppers	0.75	0.82	0.78	90
clock	0.66	0.59	0.62	90
computer keyboard	0.55	0.47	0.51	90
lamp	0.32	0.52	0.39	90
telephone	0.83	0.77	0.80	90
television	0.72	0.60	0.65	90
bed	0.59	0.53	0.56	90
chair	0.74	0.63	0.68	90
couch	0.46	0.37	0.41	90
table	0.48	0.63	0.55	90
wardrobe	0.64	0.67	0.65	90
bee	0.30	0.27	0.28	90
beetle	0.66	0.70	0.68	90
butterfly	0.54	0.54	0.54	90
caterpillar	0.48	0.44	0.46	90
cockroach	0.94	0.84	0.89	90
bear	0.65	0.69	0.67	90
leopard	0.82	0.76	0.79	90
lion	0.63	0.58	0.60	90
tiger	0.71	0.68	0.69	90
wolf	0.46	0.49	0.47	90
bridge	0.49	0.38	0.43	90
castle	0.51	0.49	0.50	90
house	0.56	0.42	0.48	90
road	0.70	0.84	0.76	90
skyscraper	0.79	0.83	0.81	90
cloud	0.42	0.28	0.33	90
forest	0.70	0.69	0.70	90
mountain	0.54	0.80	0.64	90
plain	0.85	0.86	0.85	90
sea	0.60	0.76	0.67	90
camel	0.41	0.32	0.36	90
cattle	0.78	0.72	0.75	90
chimpanzee	0.67	0.67	0.67	90

elephant	0.80	0.73	0.76	90
kangaroo	0.62	0.44	0.52	90
fox	0.80	0.76	0.78	90
porcupine	0.69	0.74	0.72	90
possum	0.60	0.68	0.64	90
raccoon	0.64	0.61	0.62	90
skunk	0.52	0.37	0.43	90
crab	0.60	0.50	0.55	90
lobster	0.52	0.60	0.56	90
snail	0.61	0.51	0.56	90
spider	0.84	0.90	0.87	90
worm	0.81	0.89	0.85	90
baby	0.70	0.61	0.65	90
boy	0.67	0.76	0.71	90
girl	0.46	0.36	0.40	90
man	0.58	0.61	0.59	90
woman	0.31	0.34	0.32	90
crocodile	0.75	0.82	0.78	90
dinosaur	0.82	0.89	0.86	90
lizard	0.58	0.57	0.57	90
snake	0.63	0.63	0.63	90
turtle	0.66	0.63	0.65	90
hamster	0.32	0.33	0.33	90
mouse	0.47	0.57	0.52	90
rabbit	0.89	0.89	0.89	90
shrew	0.63	0.74	0.68	90
squirrel	0.54	0.49	0.51	90
maple	0.70	0.69	0.69	90
oak	0.76	0.79	0.77	90
palm	0.73	0.80	0.77	90
pine	0.58	0.59	0.58	90
willow	0.60	0.67	0.63	90
bicycle	0.58	0.58	0.58	90
bus	0.61	0.63	0.62	90
motorcycle	0.56	0.49	0.52	90
pickup truck	0.57	0.56	0.56	90
train	0.81	0.87	0.84	90
lawn-mower	0.61	0.69	0.65	90
rocket	0.41	0.43	0.42	90
streetcar	0.60	0.63	0.62	90
tank	0.41	0.40	0.40	90
tractor	0.75	0.76	0.75	90
accuracy			0.63	9000
macro avg	0.63	0.63	0.63	9000
weighted avg	0.63	0.63	0.63	9000

In [7]: